GIÁO TRÌNH TỰ HỌC DÀNH CHO NGƯỜI MỚI BẮT ĐẦU

PYTHON TỪ TRONG CHĂN

TẬP 1

Hiểu bản chất — Thử bằng đôi tay — Tự xây lời giải

Môi trường, biến, kiểu dữ liệu, nhập xuất, điều kiện, vòng lặp và hàm

Nguyễn Huỳnh Đăng Nhựt

Bản thiết kế đồng bộ — 2026

Lời nói đầu

Chào mừng bạn đến với thế giới lập trình!

Python hiện là một trong những ngôn ngữ lập trình phổ biến nhất thế giới. Không chỉ sở hữu cú pháp đơn giản, dễ đọc và gần gũi với tiếng Anh, Python còn là nền tảng của rất nhiều lĩnh vực công nghệ hiện đại như Trí tuệ nhân tạo (Artificial Intelligence), Khoa học dữ liệu (Data Science), Phân tích dữ liệu (Data Analytics), Tự động hóa (Automation), Phát triển Web, An ninh mạng và Điện toán đám mây. Tuy nhiên, học Python không chỉ là học cách viết các dòng lệnh; điều quan trọng hơn là xây dựng tư duy giải quyết vấn đề và hiểu được bản chất của lập trình.

Với triết lý **"Hiểu bản chất trước khi ghi nhớ cú pháp"**, cuốn sách *Python từ trong chăn* được xây dựng theo phương pháp học tập trực quan và thực hành liên tục. Mỗi chương đều được trình bày theo một cấu trúc thống nhất:

Khái niệm $\longrightarrow$ Ví dụ minh họa $\longrightarrow$ Giải thích chi tiết $\longrightarrow$ Ghi nhớ nhanh $\longrightarrow$ Lỗi thường gặp $\longrightarrow$ Bài tập thực hành

Cách tiếp cận này giúp người học không chỉ biết *viết được chương trình*, mà còn hiểu *vì sao chương trình hoạt động*, từ đó hình thành nền tảng vững chắc để tiếp tục học các chủ đề nâng cao.

Cuốn sách hướng đến nhiều đối tượng độc giả: học sinh, sinh viên, người mới bắt đầu lập trình, giáo viên, cũng như bất kỳ ai mong muốn sử dụng Python để phục vụ học tập, nghiên cứu hoặc công việc.

Tôi là một người yêu thích và đam mê ngành Khoa học máy tính, lập trình và công nghệ nói chung. Trong quá trình học tập và thực hành, tôi đã nhận ra rằng lập trình không phải là một năng khiếu đặc biệt, mà là một kỹ năng có thể rèn luyện qua thời gian. Chính những trải nghiệm đó đã thôi thúc tôi biên soạn tài liệu này với mong muốn mang đến một người bạn đồng hành hữu ích cho những ai đang bước những bước đầu tiên trên hành trình chinh phục khoa học máy tính.

Nội dung cuốn sách được tổng hợp từ nhiều tài liệu uy tín, kết hợp với kinh nghiệm học tập, thực hành và phương pháp giảng giải hướng đến việc giúp người đọc hiểu sâu thay vì chỉ ghi nhớ. Tôi hy vọng rằng *Python từ trong chăn* sẽ không chỉ giúp bạn học Python, mà còn mở ra cánh cửa để khám phá thế giới của khoa học máy tính, phát triển phần mềm và trí tuệ nhân tạo.

Chỉ cần kiên trì học tập, thực hành đều đặn và không ngại mắc lỗi, bất kỳ ai cũng có thể tiến bộ và trở thành một lập trình viên giỏi.

Xin cảm ơn bạn đã lựa chọn *Python từ trong chăn*. Chúc bạn có một hành trình học tập thật nhiều niềm vui, khám phá và thành công!

Nguyễn Huỳnh Đăng Nhựt
Tác giả

Mục lục

Lời nói đầu **iii**

I Khởi hành cùng Python 1

1 Làm quen với Python - Tại sao lại là Python? 3

1.1 Từ một tình huống thật đến mô hình có thể kiểm tra 3
1.1.1 Bước 1: khoanh ranh giới của bài toán 4
1.1.2 Bước 2: viết quan hệ trước khi viết cú pháp 4
1.1.3 Bước 3: xác định bất biến 5
1.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi 5
1.3 Một ca làm mẫu đi đủ các bước 6
1.3.1 Bước 1: phát biểu lại yêu cầu 6
1.3.2 Bước 2: dựng ví dụ nhỏ bằng tay 6
1.3.3 Bước 3: chọn biểu diễn dữ liệu 6
1.3.4 Bước 4: viết thuật toán bằng động từ 7
1.3.5 Bước 5: chứng minh đủ dùng 7
1.4 Kiểm thử là thí nghiệm của mô hình 7
1.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm 7
1.6 Vận dụng và mở rộng mô hình 8
1.7 Bài tập ba tầng: từ hiểu đến tự thiết kế 8
1.8 Python là gì? Tại sao nên học Python? 8
1.9 Cài đặt Python, IDE và cách chạy chương trình 9
1.9.1 Cài đặt Python 9
1.9.2 Chọn IDE (Môi trường phát triển) 10
1.9.3 Chạy chương trình đầu tiên 12
1.10 Bài tập luyện tập 13
1.10.1 Bài tập cơ bản 13
1.10.2 Bài tập về tư duy lập trình 14
1.10.3 Bài tập mở rộng 16
1.11 Tóm tắt cuối chương 17
1.12 Thử thách cuối chương 18

2 Nền tảng đầu tiên - Biến, Kiểu dữ liệu và Toán tử 21

2.1 Từ một tình huống thật đến mô hình có thể kiểm tra 22

2.1.1 Bước 1: khoanh ranh giới của bài toán . . . 22
2.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 22
2.1.3 Bước 3: xác định bất biến . . . 23
2.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 23
2.3 Một ca làm mẫu đi đủ các bước . . . 24
2.3.1 Bước 1: phát biểu lại yêu cầu . . . 24
2.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 24
2.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 25
2.3.4 Bước 4: viết thuật toán bằng động từ . . . 25
2.3.5 Bước 5: chứng minh đủ dùng . . . 25
2.4 Kiểm thử là thí nghiệm của mô hình . . . 25
2.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 26
2.6 Vận dụng và mở rộng mô hình . . . 26
2.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 26
2.8 Biến (Variables) - Những chiếc hộp chứa dữ liệu . . . 27
2.8.1 Quy tắc đặt tên biến . . . 27
2.8.2 Gán lại giá trị cho biến . . . 28
2.9 Kiểu dữ liệu (Data Types) . . . 28
2.9.1 Số nguyên (`int`) . . . 28
2.9.2 Số thực (`float`) . . . 29
2.9.3 Chuỗi ký tự (`str`) . . . 29
2.9.4 Giá trị logic (`bool`) . . . 30
2.10 Toán tử (Operators) . . . 30
2.10.1 Toán tử số học . . . 31
2.10.2 Thứ tự ưu tiên của toán tử . . . 32
2.10.3 Toán tử so sánh . . . 32
2.10.4 Toán tử logic . . . 33
2.10.5 Ý nghĩa thực hành của toán tử . . . 34
2.11 Ép kiểu dữ liệu (Type Casting) . . . 34
2.11.1 Ép kiểu từ chuỗi sang số . . . 35
2.11.2 Ép kiểu giữa số nguyên và số thực . . . 35
2.11.3 Ép kiểu sang chuỗi . . . 36
2.11.4 Vì sao ép kiểu quan trọng? . . . 36
2.11.5 Mối liên hệ giữa ép kiểu và tư duy toán học . . . 36
2.12 Bài tập luyện tập . . . 37
2.13 Tóm tắt cuối chương . . . 38

3 Tương tác với chương trình - Nhập và Xuất dữ liệu 39
3.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 40
3.1.1 Bước 1: khoanh ranh giới của bài toán . . . 40
3.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 40
3.1.3 Bước 3: xác định bất biến . . . 41
3.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 41
3.3 Một ca làm mẫu đi đủ các bước . . . 42

3.3.1 Bước 1: phát biểu lại yêu cầu . . . 42
3.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 42
3.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 42
3.3.4 Bước 4: viết thuật toán bằng động từ . . . 43
3.3.5 Bước 5: chứng minh đủ dùng . . . 43
3.4 Kiểm thử là thí nghiệm của mô hình . . . 43
3.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 44
3.6 Vận dụng và mở rộng mô hình . . . 44
3.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 44
3.8 Hàm nhập liệu `input()` . . . 45
3.8.1 Cách hiểu `input()` theo tư duy bài toán . . . 45
3.8.2 Tại sao `input()` luôn trả về chuỗi? . . . 45
3.9 Hàm xuất liệu `print()` nâng cao . . . 46
3.9.1 `sep` - ký tự phân cách . . . 46
3.9.2 `end` - ký tự kết thúc dòng . . . 47
3.10 Định dạng chuỗi với `f-string` . . . 47
3.10.1 Vì sao `f-string` quan trọng? . . . 47
3.11 Nhập dữ liệu số và ép kiểu . . . 48
3.12 Kết hợp nhập, xử lý và xuất dữ liệu . . . 48
3.13 Bài tập luyện tập . . . 49
3.14 Tóm tắt cuối chương . . . 49

4 Rẽ nhánh chương trình - Câu lệnh điều kiện 51
4.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 52
4.1.1 Bước 1: khoanh ranh giới của bài toán . . . 52
4.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 52
4.1.3 Bước 3: xác định bất biến . . . 53
4.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 53
4.3 Một ca làm mẫu đi đủ các bước . . . 54
4.3.1 Bước 1: phát biểu lại yêu cầu . . . 54
4.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 54
4.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 55
4.3.4 Bước 4: viết thuật toán bằng động từ . . . 55
4.3.5 Bước 5: chứng minh đủ dùng . . . 55
4.4 Kiểm thử là thí nghiệm của mô hình . . . 55
4.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 56
4.6 Vận dụng và mở rộng mô hình . . . 56
4.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 56
4.8 Ý nghĩa của câu lệnh điều kiện . . . 57
4.9 Câu lệnh `if` . . . 57
4.9.1 Cấu trúc tổng quát của `if` . . . 58
4.9.2 Vai trò của điều kiện trong tư duy giải toán . . . 58
4.10 Quy tắc thụt lề trong Python . . . 59
4.10.1 Ý nghĩa của thụt lề . . . 60

4.11 Câu lệnh `if-else` . . . 61
4.12 Câu lệnh `if-elif-else` . . . 62
4.12.1 Vì sao thứ tự các điều kiện lại quan trọng? . . . 64
4.12.2 Ý nghĩa của `elif` . . . 64
4.12.3 Nhận diện khi nào cần nhiều nhánh . . . 64
4.13 Kết hợp điều kiện với toán tử logic . . . 65
4.13.1 Ba phép logic cơ bản . . . 66
4.14 Ứng dụng của câu lệnh điều kiện . . . 66
4.15 Bài tập luyện tập . . . 67
4.16 Tóm tắt cuối chương . . . 68

5 Lặp lại công việc - Vòng lặp 71
5.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 72
5.1.1 Bước 1: khoanh ranh giới của bài toán . . . 72
5.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 72
5.1.3 Bước 3: xác định bất biến . . . 73
5.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 73
5.3 Một ca làm mẫu đi đủ các bước . . . 74
5.3.1 Bước 1: phát biểu lại yêu cầu . . . 74
5.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 74
5.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 75
5.3.4 Bước 4: viết thuật toán bằng động từ . . . 75
5.3.5 Bước 5: chứng minh đủ dùng . . . 75
5.4 Kiểm thử là thí nghiệm của mô hình . . . 75
5.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 76
5.6 Vận dụng và mở rộng mô hình . . . 76
5.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 76
5.8 Ý nghĩa của vòng lặp . . . 77
5.8.1 Một hình dung đơn giản . . . 78
5.9 Vòng lặp `for` và hàm `range()` . . . 78
5.9.1 Hàm `range()` . . . 80
5.9.2 Tư duy đằng sau `range()` . . . 80
5.9.3 Ứng dụng của vòng lặp `for` . . . 81
5.9.4 Cảm giác quen thuộc với người học Toán . . . 81
5.10 Vòng lặp `while` . . . 82
5.10.1 Bản chất của vòng lặp `while` . . . 83
5.10.2 Sai lầm thường gặp với `while` . . . 84
5.10.3 Tư duy kiểm tra liên tục . . . 84
5.11 `break` và `continue` . . . 85
5.11.1 Khi nào dùng `break`, khi nào dùng `continue`? . . . 86
5.11.2 Hình ảnh tư duy . . . 86
5.12 Ứng dụng của vòng lặp . . . 87
5.13 Bài tập luyện tập . . . 88
5.14 Tóm tắt cuối chương . . . 88

6 Tổ chức mã nguồn - Hàm (Functions) 91
6.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 92
6.1.1 Bước 1: khoanh ranh giới của bài toán . . . 93
6.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 93
6.1.3 Bước 3: xác định bất biến . . . 94
6.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 94
6.3 Một ca làm mẫu đi đủ các bước . . . 95
6.3.1 Bước 1: phát biểu lại yêu cầu . . . 95
6.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 95
6.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 95
6.3.4 Bước 4: viết thuật toán bằng động từ . . . 96
6.3.5 Bước 5: chứng minh đủ dùng . . . 96
6.4 Kiểm thử là thí nghiệm của mô hình . . . 96
6.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 96
6.6 Vận dụng và mở rộng mô hình . . . 97
6.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 97
6.8 Ý nghĩa của hàm . . . 97
6.8.1 Tư duy chia nhỏ bài toán . . . 99
6.9 Định nghĩa hàm với `def` . . . 99
6.9.1 Luồng thực thi của chương trình . . . 100
6.9.2 Quy tắc đặt tên hàm . . . 100
6.9.3 Mỗi hàm nên thực hiện một nhiệm vụ . . . 101
6.9.4 Liên hệ với Toán học . . . 101
6.10 Tham số (Parameters) và Đối số (Arguments) . . . 102
6.10.1 Hàm có nhiều tham số . . . 103
6.10.2 Sự tương ứng giữa tham số và đối số . . . 103
6.10.3 Luồng truyền dữ liệu vào hàm . . . 104
6.10.4 Liên hệ với hàm số trong Toán học . . . 104
6.10.5 Vì sao nên sử dụng tham số? . . . 104
6.11 Lệnh `return` – Trả về giá trị . . . 105
6.11.1 `print()` và `return` khác nhau như thế nào? . . . 106
6.11.2 Hàm kết thúc ngay khi gặp `return` . . . 107
6.11.3 Một hàm có thể trả về nhiều kiểu dữ liệu . . . 107
6.11.4 Khi nào nên dùng `return`? . . . 108
6.12 Bài tập luyện tập . . . 108

II Từ cú pháp đến cách giải một bài toán 111

7 Tư duy thuật toán - Trước code là một bản thiết kế 113
7.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 114
7.1.1 Bước 1: khoanh ranh giới của bài toán . . . 114
7.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 114
7.1.3 Bước 3: xác định bất biến . . . 115

7.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 115
7.3 Một ca làm mẫu đi đủ các bước . . . 116
7.3.1 Bước 1: phát biểu lại yêu cầu . . . 116
7.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 116
7.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 116
7.3.4 Bước 4: viết thuật toán bằng động từ . . . 117
7.3.5 Bước 5: chứng minh đủ dùng . . . 117
7.4 Kiểm thử là thí nghiệm của mô hình . . . 117
7.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 118
7.6 Vận dụng và mở rộng mô hình . . . 118
7.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 118
7.8 Phòng thực hành sau khi đã hoàn tất mô hình . . . 119
7.9 Tóm tắt cuối chương . . . 119

8 Truy vết, gỡ lỗi và kiểm thử - Học cách chứng minh chương trình 121
8.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 122
8.1.1 Bước 1: khoanh ranh giới của bài toán . . . 122
8.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 122
8.1.3 Bước 3: xác định bất biến . . . 123
8.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 123
8.3 Một ca làm mẫu đi đủ các bước . . . 124
8.3.1 Bước 1: phát biểu lại yêu cầu . . . 124
8.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 124
8.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 124
8.3.4 Bước 4: viết thuật toán bằng động từ . . . 125
8.3.5 Bước 5: chứng minh đủ dùng . . . 125
8.4 Kiểm thử là thí nghiệm của mô hình . . . 125
8.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 126
8.6 Vận dụng và mở rộng mô hình . . . 126
8.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 126
8.8 Phòng thực hành sau khi đã hoàn tất mô hình . . . 127
8.9 Tóm tắt cuối chương . . . 127

9 Mô hình hóa từ Toán học đến Python 129
9.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 129
9.1.1 Bước 1: khoanh ranh giới của bài toán . . . 130
9.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 130
9.1.3 Bước 3: xác định bất biến . . . 131
9.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 131
9.3 Một ca làm mẫu đi đủ các bước . . . 132
9.3.1 Bước 1: phát biểu lại yêu cầu . . . 132
9.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 132
9.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 132
9.3.4 Bước 4: viết thuật toán bằng động từ . . . 133

9.3.5 Bước 5: chứng minh đủ dùng . . . 133
9.4 Kiểm thử là thí nghiệm của mô hình . . . 133
9.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 133
9.6 Vận dụng và mở rộng mô hình . . . 134
9.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 134
9.8 Phòng thực hành sau khi đã hoàn tất mô hình . . . 134
9.9 Tóm tắt cuối chương . . . 135

10 Chia nhỏ bài toán - Từ một khối lớn đến các phần có hợp đồng 137
10.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 138
10.1.1 Bước 1: khoanh ranh giới của bài toán . . . 138
10.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 138
10.1.3 Bước 3: xác định bất biến . . . 139
10.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 139
10.3 Một ca làm mẫu đi đủ các bước . . . 140
10.3.1 Bước 1: phát biểu lại yêu cầu . . . 140
10.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 140
10.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 140
10.3.4 Bước 4: viết thuật toán bằng động từ . . . 141
10.3.5 Bước 5: chứng minh đủ dùng . . . 141
10.4 Kiểm thử là thí nghiệm của mô hình . . . 141
10.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 141
10.6 Vận dụng và mở rộng mô hình . . . 142
10.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 142
10.8 Phòng thực hành sau khi đã hoàn tất mô hình . . . 142
10.9 Tóm tắt cuối chương . . . 143

11 Dữ liệu hợp lệ - Đừng để lỗi đi sâu vào chương trình 145
11.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 146
11.1.1 Bước 1: khoanh ranh giới của bài toán . . . 146
11.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 146
11.1.3 Bước 3: xác định bất biến . . . 147
11.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 147
11.3 Một ca làm mẫu đi đủ các bước . . . 148
11.3.1 Bước 1: phát biểu lại yêu cầu . . . 148
11.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 148
11.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 148
11.3.4 Bước 4: viết thuật toán bằng động từ . . . 149
11.3.5 Bước 5: chứng minh đủ dùng . . . 149
11.4 Kiểm thử là thí nghiệm của mô hình . . . 149
11.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 149
11.6 Vận dụng và mở rộng mô hình . . . 150
11.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 150
11.8 Phòng thực hành sau khi đã hoàn tất mô hình . . . 150

11.9 Tóm tắt cuối chương . . . 151

12 Dự án cuối Tập 1 - Máy luyện Toán trong terminal 153
12.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 154
12.1.1 Bước 1: khoanh ranh giới của bài toán . . . 154
12.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 154
12.1.3 Bước 3: xác định bất biến . . . 155
12.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 155
12.3 Một ca làm mẫu đi đủ các bước . . . 156
12.3.1 Bước 1: phát biểu lại yêu cầu . . . 156
12.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 156
12.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 156
12.3.4 Bước 4: viết thuật toán bằng động từ . . . 157
12.3.5 Bước 5: chứng minh đủ dùng . . . 157
12.4 Kiểm thử là thí nghiệm của mô hình . . . 157
12.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 158
12.6 Vận dụng và mở rộng mô hình . . . 158
12.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 158
12.8 Phòng thực hành sau khi đã hoàn tất mô hình . . . 159
12.9 Tóm tắt cuối chương . . . 159

A Xưởng 24 bài toán - Đi từ Toán học và đời sống đến Python 161
A.1 Điểm trung bình có trọng số . . . 161
A.2 Chuyển động thẳng đều . . . 162
A.3 Phương trình bậc hai . . . 163
A.4 Tam giác hợp lệ và diện tích . . . 164
A.5 Tiền điện theo bậc . . . 165
A.6 Chỉ số BMI . . . 166
A.7 Lãi kép . . . 167
A.8 Tiết kiệm đến mục tiêu . . . 168
A.9 Kiểm tra số nguyên tố . . . 169
A.10 Thuật toán Euclid . . . 170
A.11 Đổi số thập phân sang nhị phân . . . 171
A.12 Dãy Collatz và điều kiện dừng . . . 172
A.13 Máy tính bốn phép . . . 173
A.14 Kiểm tra mật khẩu theo quy tắc . . . 174
A.15 Trò chơi đoán số . . . 175
A.16 Độ cao vật ném theo thời gian . . . 176
A.17 Điểm hòa vốn . . . 177
A.18 Trung bình và phương sai . . . 178
A.19 Năm nhuận . . . 179
A.20 Khoản trả góp cố định . . . 180
A.21 Mô phỏng bể nước . . . 181
A.22 Logic đèn giao thông . . . 182

A.23 Lịch học theo quỹ thời gian 183
A.24 Báo cáo tiến độ tuần 184

Tài liệu tham khảo **187**

Phần I

Khởi hành cùng Python

Chương 1

Làm quen với Python - Tại sao lại là Python?

Giới thiệu ngắn

Mỗi ngôn ngữ lập trình đều được tạo ra với những mục đích và thế mạnh riêng. Có ngôn ngữ được thiết kế để xử lý hệ thống ở mức rất thấp, có ngôn ngữ phù hợp cho phát triển web, có ngôn ngữ mạnh về tính toán khoa học, và cũng có những ngôn ngữ được tạo ra để giúp người học tiếp cận lập trình một cách dễ dàng hơn. Trong số đó, Python nổi bật như một lựa chọn thân thiện, linh hoạt và có tính ứng dụng rất cao.

Trước khi bắt đầu viết những dòng mã đầu tiên, chúng ta cần hiểu Python là gì, vì sao nó trở thành một trong những ngôn ngữ lập trình phổ biến nhất thế giới, và điều gì khiến nó được yêu thích trong nhiều lĩnh vực khác nhau. Khi nắm được bức tranh tổng quan này, người học sẽ không còn cảm thấy Python là một công cụ xa lạ, mà ngược lại, sẽ thấy rõ vì sao đây là một ngôn ngữ rất phù hợp để bắt đầu hành trình lập trình của mình.

Bên cạnh đó, việc chuẩn bị môi trường làm việc cũng là một bước quan trọng không thể bỏ qua. Một môi trường được cài đặt đúng cách sẽ giúp quá trình học diễn ra thuận lợi hơn, hạn chế lỗi phát sinh và giúp bạn tập trung vào điều quan trọng nhất: hiểu cách tư duy và giải quyết vấn đề bằng lập trình. Vì vậy, ở phần tiếp theo, chúng ta sẽ lần lượt tìm hiểu Python là gì, lợi ích của việc học Python, và cách cài đặt, thiết lập môi trường làm việc trên máy tính của bạn.

1.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một học sinh muốn tính tổng thời gian tự học trong tuần. Bạn ấy có thể cộng bằng tay, dùng bảng tính hoặc viết chương trình. Cùng một nhu cầu, nhưng mỗi công cụ đòi hỏi cách mô tả khác nhau. Câu hỏi của chương không chỉ là cài Python, mà là làm thế nào biến một mong muốn bằng ngôn ngữ tự nhiên thành chuỗi chỉ dẫn đủ chính

xác để máy tính thực hiện và để con người kiểm tra.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **ngôn ngữ lập trình, trình thông dịch và môi trường chạy**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

1.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: phiên bản Python, đường dẫn tệp, thư mục làm việc, lệnh chạy, dữ liệu vào và kết quả quan sát.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

1.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{vấn đề} \xrightarrow{\text{mô hình}} \text{thuật toán} \xrightarrow{\text{mã nguồn}} \text{trình thông dịch} \xrightarrow{\text{hành động}} \text{kết quả}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: tệp nguồn đang dùng, thư mục hiện tại, phiên bản trình thông dịch và các gói đã cài?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

1.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Cùng một tệp nguồn và cùng dữ liệu đầu vào phải được chạy bằng đúng môi trường đã mô tả; mọi kết quả bàn giao phải có cách tái tạo.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

1.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **ngôn ngữ lập trình, trình thông dịch và môi trường chạy** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

1.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Ta cần bàn giao cho một bạn khác chương trình tính tổng số phút học của bảy ngày. Bạn ấy phải biết lấy tệp nào, dùng phiên bản Python nào, chạy lệnh gì và kiểm tra kết quả bằng phép cộng tay. Nếu chương trình chỉ chạy trên máy người viết, bài toán chưa hoàn thành.

1.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

1.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

1.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

1.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

1.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

1.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

1.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Cài nhiều Python rồi không biết lệnh `python` đang trỏ đến bản nào.
- Nhìn thấy chương trình in kết quả một lần và kết luận môi trường đã ổn định.
- Chép nguyên lệnh trên mạng mà không hiểu thư mục hiện tại và đường dẫn tệp.

- Trộn lỗi cài đặt, lỗi cú pháp và lỗi mô hình thành một vấn đề duy nhất.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

1.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

1.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ sơ đồ từ tệp `.py` đến kết quả trên terminal và giải thích vai trò của từng khối.
2. Tạo một chương trình in tên, mục tiêu học và tổng số phút học; dự đoán đầu ra trước khi chạy.
3. Cố ý chạy ở sai thư mục, ghi lại thông báo lỗi, rồi mô tả cách xác định nguyên nhân.
4. Viết hướng dẫn năm bước để một người chưa cài Python có thể tái tạo đúng kết quả.
5. So sánh chạy tương tác và chạy tệp theo mục đích, dữ liệu tồn tại và khả năng lặp lại.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

1.8 Python là gì? Tại sao nên học Python?

Rất nhiều bạn đã nói với tôi rằng việc học muốn học Python, nhưng nếu đặt ngược về câu hỏi về việc: *Học Python để làm gì?*. Thì câu trả lời sẽ luôn như là trống rỗng - Thật sự, trong chúng ta không biết về mục tiêu học, kết quả cũng sẽ rất dễ để quay về con số 0 tròn trĩnh. Hay đơn giản hơn đó là sự quanh quẩn của một vòng xoáy, nơi mà việc học không thể được

ứng dụng từ các bạn. Để dẹp khuất đi những góc nhìn đó. Chúng ta phải hoàn toàn hiểu được việc: *Ngôn ngữ lập trình là gì?* và *Tại sao nên học Python?*

Cách bạn giao tiếp và truyền đạt ý tưởng, phong cách hằng ngày phải sử dụng "Ngôn ngữ", đó là phương tiện giao tiếp duy nhất và gần như là trong mọi tình huống. Sự khác biệt về cách giao tiếp cũng có thể được thể hiện qua ngôn ngữ giao tiếp: Giữa hai người đến từ hai nền văn hóa, vùng đất khác nhau. Nhưng, giữa máy tính và con người - Chúng ta không phải khác biệt về văn hóa, cũng không khác biệt về vùng đất; chúng ta khác biệt về cách hình thành, cách hoạt động, cách vận hành. Chính vì thế, để điều khiển giữa hai vật chứng này, chúng ta phải sử dụng một ngôn ngữ giao tiếp đứng giữa.

Trong máy tính, hay các thiết bị thông minh; chúng sử dụng *Hệ nhị phân* hay *Binary number* làm ngôn ngữ chính. Trong khi đó, cách giao tiếp của con người là sử dụng *ASCII* để giao tiếp, hay còn gọi là *Character*. Vì thế, việc các ngôn ngữ lập trình đứng giữa như là một "Thông dịch viên" đứng giữa để có thể giúp Con người và Vi tính có thể giao tiếp được với nhau. Đã từ lâu, việc xuất hiện các ngôn ngữ lập trình từ bậc thấp đã là cứu tinh cho con người trong việc này. Cho đến hiện đại, việc các ngôn ngữ lập trình bậc cao - C, CPP, C#, Python, ... đã phát triển đến một mức phù hợp và tạo ra một môi trường tốt để mọi người có thể sử dụng.

Là một ngôn ngữ lập trình bậc cao - Python chính là minh chứng sống cho điều đó. Khác với các dạng ngôn ngữ lập trình khác, Python giống "gần như hoàn toàn" với việc bạn đang giao tiếp với máy tính bằng Tiếng Anh trên hệ văn đầy đủ. Ngoài ra, Python là một trong những ngôn ngữ lập trình lý tưởng nhất cho người mới bắt đầu nhờ cú pháp đơn giản, sáng sủa và rất dễ đọc. Thay vì phải đối mặt với những cấu trúc phức tạp ngay từ đầu, người học có thể nhanh chóng làm quen với cách tư duy lập trình thông qua những câu lệnh gần gũi với ngôn ngữ tự nhiên. Bên cạnh đó, Python còn là một ngôn ngữ đa năng, được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau như phát triển web với Django và Flask, trí tuệ nhân tạo và học máy với TensorFlow và PyTorch, tự động hóa, phân tích dữ liệu, cũng như nhiều bài toán thực tiễn khác. Một ưu điểm quan trọng nữa của Python là cộng đồng người dùng rất lớn và năng động trên toàn thế giới. Điều này giúp người học dễ dàng tìm thấy tài liệu, ví dụ minh họa, hướng dẫn chi tiết và lời giải cho hầu hết các lỗi thường gặp. Chính sự thân thiện, linh hoạt và giàu tài nguyên hỗ trợ ấy đã khiến Python trở thành lựa chọn phù hợp để bắt đầu hành trình khám phá lập trình.

1.9 Cài đặt Python, IDE và cách chạy chương trình

1.9.1 Cài đặt Python

1. Truy cập trang chủ chính thức của Python theo website sau - Việc sử dụng trang chính thức của Python sẽ giúp các bạn dễ lựa chọn phù hợp các môi trường, cấu hình máy phù hợp và dễ dàng thực hiện khi gặp những tình huống đặc biệt. `https://www.python.org/downloads/`

2. Ở trang chính thức của Python sẽ có tùy chọn các version để thuận tiện cho việc ứng dụng. Ở đay, sử dụng version Python 3.x sẽ thuận tiện hơn rất nhiều và đã có thể thực

thi tốt trên các hệ điều hành và môi trường phức tạp khác nhau.

3. Khi cài đặt Python trên Windows, **bắt buộc phải đánh dấu** vào ô *"Add Python to PATH"* trước khi nhấn nút *Install Now*. Thao tác này sẽ tự động thêm Python vào biến môi trường *PATH*, cho phép bạn chạy các lệnh như `python` hoặc `pip` trực tiếp trong *Command Prompt*, *PowerShell* hoặc *Windows Terminal* mà không cần chỉ định đường dẫn đến thư mục cài đặt. Nếu bỏ qua bước này, bạn có thể gặp lỗi `'python' is not recognized as an internal or external command`, và sẽ phải cấu hình biến môi trường thủ công sau khi cài đặt.

1.9.2 Chọn IDE (Môi trường phát triển)

IDE (Integrated Development Environment) là môi trường để bạn viết, chạy và quản lý mã nguồn. Nói một cách đơn giản, đây là "nơi làm việc" chính của lập trình viên. Mặc dù Python đi kèm với IDLE, nhưng trong thực tế, nhiều người thường sử dụng các IDE hiện đại hơn vì chúng hỗ trợ viết code nhanh hơn, dễ quản lý dự án hơn và có nhiều công cụ tiện ích hơn.

Trong số các IDE phổ biến hiện nay, hai lựa chọn rất đáng chú ý là **VS Code** và **PyCharm**.

- **VS Code**: Miễn phí, nhẹ, khởi động nhanh và cực kỳ phổ biến. VS Code không phải là IDE chuyên Python hoàn toàn, nhưng nhờ hệ sinh thái tiện ích mở rộng phong phú, nó có thể trở thành một môi trường lập trình Python rất mạnh mẽ.
- **PyCharm**: Là IDE chuyên dụng cho Python, được phát triển để hỗ trợ tối đa cho việc viết code, debug, chạy dự án và quản lý thư viện. Phiên bản Community của PyCharm hoàn toàn miễn phí và đủ dùng cho người mới học.

Cách cài đặt VS Code

Để cài đặt VS Code, bạn có thể thực hiện theo các bước sau:

1. Truy cập trang web chính thức của VS Code và tải về phiên bản phù hợp với hệ điều hành Windows.
2. Mở file cài đặt vừa tải xuống.
3. Nhấn *Next* để bắt đầu quá trình cài đặt.
4. Khi xuất hiện các tùy chọn, nên đánh dấu các mục hỗ trợ mở file nhanh hơn, chẳng hạn như:
    - *Add to PATH*
    - *Create a desktop icon* (nếu muốn tạo biểu tượng ngoài màn hình)
    - *Register Code as an editor for supported file types*
5. Nhấn *Install* và chờ quá trình hoàn tất.
6. Sau khi cài xong, mở VS Code lên.

7. Vào phần *Extensions* và tìm tiện ích **Python** do Microsoft phát triển, sau đó cài đặt.

Sau khi cài đặt xong, bạn nên kiểm tra lại xem Python đã được nhận diện chưa. Trong VS Code, khi mở một file `.py`, chương trình thường sẽ gợi ý chọn interpreter phù hợp. Nếu chưa thấy, bạn có thể mở Command Palette và chọn lại môi trường Python đã cài.

Cách cài đặt PyCharm

PyCharm là một lựa chọn rất tốt nếu bạn muốn có một môi trường Python đầy đủ tính năng ngay từ đầu. Các bước cài đặt như sau:

1. Truy cập trang web chính thức của JetBrains và tải về **PyCharm Community Edition**.
2. Mở file cài đặt.
3. Nhấn *Next* để tiếp tục.
4. Chọn thư mục cài đặt nếu cần, sau đó tiếp tục nhấn *Next*.
5. Ở phần tùy chọn cài đặt, bạn nên chú ý:
    - Tạo shortcut trên desktop nếu muốn truy cập nhanh.
    - Chọn mở file `.py` bằng PyCharm nếu bạn muốn dùng PyCharm làm công cụ mặc định.
6. Nhấn *Install* và chờ hoàn tất.
7. Sau khi cài xong, khởi động PyCharm.
8. Khi mở lần đầu, PyCharm có thể hỏi bạn muốn import cài đặt cũ hay không; nếu là lần đầu sử dụng, bạn có thể chọn bỏ qua.

Tiếp theo, bạn cần tạo một dự án mới:

1. Chọn *New Project*.
2. Đặt tên cho dự án.
3. Chọn vị trí lưu dự án.
4. Chọn Python Interpreter.
5. Nếu PyCharm chưa tự nhận Python, bạn hãy trỏ đến file cài đặt Python đã có trên máy.
6. Nhấn *Create* để tạo dự án.

Nên chọn VS Code hay PyCharm?

Việc chọn IDE phụ thuộc vào mục tiêu học tập của bạn:

- Nếu bạn muốn một công cụ nhẹ, linh hoạt, có thể dùng cho nhiều ngôn ngữ lập trình

khác nhau, **VS Code** là lựa chọn rất tốt.

- Nếu bạn muốn một môi trường chuyên sâu cho Python, có sẵn nhiều tính năng hỗ trợ debug và quản lý dự án, **PyCharm** là lựa chọn phù hợp hơn.

Với người mới học, cả hai đều có thể dùng tốt. Tuy nhiên, nếu bạn muốn bắt đầu đơn giản và dễ mở rộng sang các ngôn ngữ khác trong tương lai, VS Code thường là lựa chọn được ưu tiên. Còn nếu bạn xác định học Python nghiêm túc và muốn một môi trường mạnh mẽ ngay từ đầu, PyCharm là công cụ rất đáng dùng.

1.9.3 Chạy chương trình đầu tiên

Sau khi cài đặt xong, hãy mở *Terminal* (trên macOS hoặc Linux) hoặc *Command Prompt* trên Windows, rồi gõ lệnh sau:

```
python --version
```

Lệnh này dùng để kiểm tra xem Python đã được cài đặt đúng trên máy tính hay chưa, đồng thời cho biết phiên bản Python hiện tại đang được sử dụng. Nếu hệ thống hiển thị một dòng thông báo chứa số phiên bản, ví dụ như `Python 3.10.5` hoặc `Python 3.12.1`, điều đó có nghĩa là quá trình cài đặt đã thành công và máy tính của bạn đã sẵn sàng để chạy chương trình Python.

Ngoài lệnh `python --version`, Python còn cung cấp lệnh trợ giúp để bạn xem các tùy chọn có thể sử dụng khi chạy chương trình từ dòng lệnh. Trong *Terminal* hoặc *Command Prompt*, bạn có thể nhập:

```
python --help
```

hoặc viết ngắn gọn hơn:

```
python -h
```

Hai lệnh này có cùng mục đích là hiển thị trang hướng dẫn sử dụng cơ bản của Python trên dòng lệnh. Khi thực hiện, màn hình sẽ xuất hiện danh sách các tham số hỗ trợ, kèm theo mô tả ngắn gọn cho từng tùy chọn. Nhờ đó, bạn có thể biết Python có thể được khởi động theo những cách nào, chẳng hạn như chạy một tệp chương trình, thực thi lệnh trực tiếp, vào chế độ tương tác, hoặc xem các thiết lập liên quan đến trình thông dịch.

Đối với người mới học, bạn chưa cần phải ghi nhớ hết tất cả các tùy chọn này. Tuy nhiên, lệnh `python --help` rất hữu ích khi bạn muốn tra cứu nhanh cách sử dụng Python trên Terminal mà không cần tìm tài liệu ở nơi khác. Đây là một thói quen tốt, vì trong quá trình học lập trình, bạn sẽ thường xuyên cần đọc hướng dẫn của công cụ để biết cách dùng đúng và hiệu quả.

Nếu sau khi gõ lệnh `python --help` mà máy không nhận diện được Python, thì có thể Python chưa được cài đặt đúng hoặc biến môi trường *PATH* chưa được cấu hình đầy đủ. Trong trường hợp đó, bạn nên kiểm tra lại bước cài đặt và đảm bảo đã chọn **Add Python to PATH** trên Windows.

Chương trình đầu tiên

Trong IDE hoặc ngay trong Terminal, hãy nhập đoạn mã sau:

```
1 print("Hello, World!")
```

Khi chạy chương trình, màn hình sẽ hiển thị:
`Hello, World!`

Lỗi không nhận lệnh `python`

Trên Windows, nếu bạn gõ `python` nhưng hệ thống báo lỗi *"'python' is not recognized..."*, nguyên nhân thường là do bạn đã quên tích vào ô **Add Python to PATH** trong quá trình cài đặt. Để khắc phục, bạn có thể gỡ cài đặt Python rồi cài lại và nhớ chọn đúng tùy chọn này, hoặc tự thêm đường dẫn cài đặt Python vào biến môi trường *PATH* của hệ thống.

1.10 Bài tập luyện tập

Sau khi đã cài đặt Python, chạy được chương trình đầu tiên và làm quen với lệnh `print()`, đây là lúc bạn chuyển từ *đọc* sang *làm*. Ở giai đoạn này, điều quan trọng không phải là viết được những chương trình lớn, mà là tập cho mình thói quen quan sát, thử nghiệm và hiểu vì sao một câu lệnh lại cho ra kết quả như vậy.

Nhiều người mới học thường nghĩ lập trình là ghi nhớ thật nhiều lệnh. Thực ra, ở chương đầu tiên, điều cần xây dựng trước tiên là cảm giác quen tay với môi trường Python, khả năng đọc một chương trình đơn giản và nhận ra mối liên hệ giữa *câu lệnh* và *kết quả*. Những bài tập dưới đây được sắp xếp từ dễ đến khó để bạn vừa làm vừa hiểu, thay vì chỉ sao chép máy móc. Nếu nói đơn giản thì Python cơ bản có bản chất rất giống so với ngôn ngữ Tiếng Anh thực tế, nên điều đó có thể cho rằng việc sử dụng Tiếng Anh thành thạo cũng là một điểm cộng rất lớn cho các bạn khi thực hiện các câu lệnh bằng ngôn ngữ Python.

1.10.1 Bài tập cơ bản

1. **Kiểm tra môi trường Python** Trước khi viết bất kỳ chương trình nào, bạn cần biết máy tính của mình đã sẵn sàng hay chưa. Điều này sẽ giúp cho việc các môi trường hoạt động trơn tru, không bị lỗi gây ảnh hưởng đến nhiều thao tác trong quá trình viết lệnh. Mở *Terminal* hoặc *Command Prompt* và thực hiện các yêu cầu sau:

   (a) Kiểm tra phiên bản Python đang được cài đặt trên máy tính. Nếu đã thực hiện theo các bước trên thì việc xuất hiện version 3.x là điều đúng đắn. Vì hoàn toàn các bước ở trên đã hỗ trợ hoàn toàn cho việc xuất hiện của version mới nhất.

   (b) Hiển thị thông tin trợ giúp của Python bằng lệnh thích hợp.

   (c) Kiểm tra Python có thể chạy được chương trình từ Terminal hay không. Chạy Python từ Terminal thay cho việc sử dụng IDE cũng là một lợi ích tốt, vì chúng

giúp hoàn toàn thay thế được việc chỉ lao đầu vào sử dụng extension có sẵn.

2. **Chương trình Python đầu tiên**

   Đây là bài tập rất quan trọng vì nó giúp bạn hiểu một nguyên tắc cốt lõi: máy tính chỉ làm đúng những gì bạn yêu cầu. Sử dụng hàm `print()` để viết một chương trình hiển thị các thông tin sau:

   - Họ và tên của bạn.
   - Trường học hoặc lớp hiện tại.
   - Mục tiêu học lập trình trong tương lai.

   Yêu cầu:

   - Mỗi thông tin được hiển thị trên một dòng riêng biệt.
   - Sử dụng ít nhất 3 câu lệnh `print()` khác nhau.

3. **Làm quen với ký tự đặc biệt**

   Lập trình không chỉ là chữ và số; đôi khi những ký hiệu đơn giản cũng tạo nên cách trình bày rõ ràng và đẹp mắt. Viết chương trình sử dụng `print()` để hiển thị đoạn văn bản sau:

```
  HOC PYTHON MOI NGAY
==================================
Bat dau tu nhung dong lenh dau tien!
==================================
```

   Yêu cầu:

   - Sử dụng ký tự `=` để tạo đường phân cách.
   - Giữ đúng định dạng hiển thị như đề bài.

1.10.2 Bài tập về tư duy lập trình

4. **Thiết kế trang giới thiệu cá nhân**

   Lập trình thường bắt đầu từ những điều gần gũi nhất với chính bạn. Hãy tạo một chương trình Python mô phỏng một trang giới thiệu cá nhân đơn giản.

   Chương trình cần hiển thị:

   - Tên người dùng.
   - Sở thích.
   - Lĩnh vực muốn tìm hiểu.
   - Một câu nói hoặc mục tiêu cá nhân.

   Ví dụ:

```
Xin chao!
```

```
Toi ten la Nguyen Van A.
Toi yeu thich lap trinh va khoa hoc.
Muc tieu cua toi la tro thanh ky su AI.
```

Bài này không chỉ luyện `print()`, mà còn giúp bạn tập diễn đạt một ý tưởng bằng nhiều dòng lệnh liên tiếp. Đây là bước đầu của tư duy cấu trúc: chia nội dung lớn thành các phần nhỏ, rõ ràng và có thứ tự. Các cấu trúc về Database, dữ liệu lớn đều cần có phân hóa cấu trúc phù hợp; chính vì thế, việc điều khiển các vùng dữ liệu hiển thị và được đặt trong các phạm vi phù hợp sẽ giúp người dùng tiếp cận và thay thế, sử dụng các nguồn dữ liệu được hoàn toàn hiệu quả.

5. **Chương trình giới thiệu khóa học**

   Hãy tưởng tượng bạn đang giới thiệu một khóa học Python cho người mới bắt đầu. Viết chương trình hiển thị:

   - Tên khóa học.
   - Thời lượng học.
   - Nội dung chính.
   - Lời mời tham gia.

   Yêu cầu:

   - Trình bày đẹp bằng các dòng phân cách.
   - Sử dụng nhiều câu lệnh `print()`.

   Ở bài này, điều quan trọng là bạn học cách nghĩ như một người thiết kế nội dung: câu nào nên đặt trước, câu nào cần nhấn mạnh, và làm thế nào để màn hình trông dễ đọc hơn.

6. **Sửa lỗi chương trình**

   Khi học lập trình, việc gặp lỗi là bình thường. Xuất hiện các lỗi là điều cần thiết để biết cách khắc phục, thông thường đối với các lỗi về cú pháp sẽ cần nhiều yếu tố tỉ mỉ hơn. Khi lập trình một chương trình, chúng thường sẽ xuất hiện khi người dùng đã quá nhanh trong việc xây dựng ý tưởng. Vì vậy, điều quan trọng là nhận ra lỗi và biết cách sửa. Cho chương trình sau:

   ```
   print("Xin chao Python)
   print("Toi dang hoc lap trinh")
   ```

   Hãy:

   (a) Xác định lỗi trong chương trình.

   (b) Sửa lại để chương trình chạy đúng.

   (c) Giải thích nguyên nhân gây ra lỗi.

   Đây là bài tập rất đáng giá vì nó dạy bạn rằng lập trình không chỉ là viết, mà còn là *đọc lỗi*, *hiểu lỗi*, và *khắc phục lỗi*.

1.10.3 Bài tập mở rộng

7. **Tạo bảng thông tin cá nhân**

   Hãy viết chương trình tạo một bảng thông tin cá nhân bằng Python. Đây là bài tập giúp bạn làm quen với việc căn lề và trình bày văn bản bằng chuỗi.

   Ví dụ:

```
+----------------------------+
|      THONG TIN CA NHAN     |
+----------------------------+
| Ho ten: Nguyen Van A       |
| Tuoi: 18                   |
| So thich: Lap trinh        |
+----------------------------+
```

   Yêu cầu:

   - Sử dụng duy nhất các câu lệnh `print()`.
   - Căn chỉnh nội dung để bảng hiển thị đẹp mắt.

8. **Khám phá Python**

   Hãy tạm rời khỏi việc gõ lệnh và thử tìm hiểu Python như một công cụ thật sự. Trả lời các câu hỏi sau:

   (a) Python được phát triển bởi ai?

   (b) Python xuất hiện lần đầu tiên vào năm nào?

   (c) Python hiện được sử dụng trong những lĩnh vực nào?

   (d) Vì sao Python được xem là một ngôn ngữ phù hợp cho người mới học?

9. **Thử nghiệm với Python Interactive Mode**

   Mở Python Interactive Shell và thực hiện:

```
>>> print("Hello Python")
>>> print(2026)
>>> print(3 + 5)
>>>
```

   Trả lời:

   - Python xử lý các câu lệnh trên như thế nào?
   - Sự khác nhau giữa việc chạy từng dòng lệnh và chạy một file Python là gì?

1.11 Tóm tắt cuối chương

Nếu nhìn lại toàn bộ chương này, bạn sẽ thấy mình đã đi qua một hành trình rất ngắn nhưng rất quan trọng. Từ một người chưa quen môi trường lập trình, bạn đã biết cách kiểm tra Python, chạy chương trình đầu tiên, in nội dung ra màn hình và quan sát cách máy tính phản hồi lại câu lệnh của mình.

- **Python là gì?**

  Python là một ngôn ngữ lập trình bậc cao, được thiết kế với mục tiêu giúp con người viết chương trình một cách đơn giản, rõ ràng và dễ hiểu. Python hỗ trợ nhiều mô hình lập trình khác nhau như lập trình hướng đối tượng, lập trình hàm và lập trình thủ tục. Điểm mạnh lớn nhất của Python ở giai đoạn đầu là cú pháp dễ đọc, vì vậy người mới học có thể tập trung vào *tư duy* thay vì bị rối bởi quá nhiều ký hiệu.

- **Ưu điểm của Python** Python được sử dụng rộng rãi nhờ:

  - Cú pháp đơn giản, gần với ngôn ngữ tự nhiên.
  - Không yêu cầu khai báo kiểu dữ liệu phức tạp khi bắt đầu.
  - Có cộng đồng người dùng lớn trên toàn thế giới.
  - Có hệ sinh thái thư viện phong phú.

  Với người mới học, đây là một lợi thế lớn vì bạn có thể nhanh chóng tạo ra kết quả nhìn thấy được, từ đó tạo động lực học tiếp.

- **Cài đặt môi trường lập trình**

  Để bắt đầu học Python, cần chuẩn bị:

  - Bộ thông dịch Python để chạy chương trình.
  - Một trình soạn thảo mã nguồn hoặc IDE như VS Code, PyCharm.
  - Terminal hoặc Command Prompt để kiểm tra và thực thi lệnh.

  Môi trường lập trình giống như bàn làm việc của bạn. Nếu bàn làm việc gọn gàng và đúng công cụ, việc học sẽ trôi chảy hơn rất nhiều.

- **Kiểm tra Python**

  Người học cần biết cách sử dụng các lệnh cơ bản:

  ```
  python --version
  python --help
  ```

  để kiểm tra phiên bản và xem hướng dẫn sử dụng Python. Đây là bước nhỏ nhưng rất thực tế: trước khi xây dựng bất kỳ thứ gì, hãy chắc chắn rằng công cụ của bạn đang hoạt động đúng.

- **Chương trình Python đầu tiên**

  Một chương trình Python có thể bắt đầu từ những câu lệnh đơn giản. Hàm:

`print()`

được sử dụng để xuất dữ liệu ra màn hình, giúp người lập trình giao tiếp với người sử dụng.

Ở mức độ nền tảng nhất, lập trình là quá trình viết ra một chuỗi chỉ dẫn để máy tính làm đúng điều bạn muốn. Lệnh `print()` là lời chào đầu tiên giữa bạn và Python.

- **Tư duy lập trình ban đầu**

  Khi học Python, điều quan trọng không chỉ là ghi nhớ câu lệnh mà còn là rèn luyện khả năng:

  - Phân tích vấn đề.
  - Chia nhỏ nhiệm vụ thành các bước.
  - Viết thuật toán trước khi lập trình.
  - Kiểm tra và sửa lỗi trong chương trình.

  Đây là phần đáng giá nhất của chương đầu tiên. Cú pháp có thể học nhanh, nhưng tư duy lập trình cần được hình thành qua thực hành đều đặn.

- **Kiến thức cần ghi nhớ**

  Sau chương này, bạn cần nắm được:

  1. Python là gì và tại sao Python phổ biến.
  2. Cách cài đặt và kiểm tra Python.
  3. Cách chạy một chương trình Python đơn giản.
  4. Cách sử dụng hàm `print()`.
  5. Vai trò của việc thực hành thường xuyên trong quá trình học lập trình.

1.12 Thử thách cuối chương

Hãy viết một chương trình Python có tên:

`introduction.py`

Chương trình mô phỏng một lời chào dành cho người dùng mới học Python.

Yêu cầu:

1. Hiển thị tiêu đề chương trình.
2. Giới thiệu bản thân.
3. Nêu lý do bạn học Python.
4. Viết một mục tiêu muốn đạt được sau 6 tháng học lập trình.
5. Trình bày kết quả theo dạng một trang giới thiệu nhỏ.

Ví dụ:

```
  MY PYTHON JOURNEY
==============================

Xin chao!
Toi ten la ...

Toi bat dau hoc Python vi ...

Muc tieu 6 thang toi:

* Viet duoc nhieu chuong trinh hon.
* Hieu ve thuat toan.
* Xay dung duoc du an dau tien.

==============================
```

Góc nhìn cá nhân

Nếu nhìn chương này như một người học thật sự, thì đây chưa phải là phần "ngầu" nhất của lập trình. Nó không có thuật toán phức tạp, không có AI, không có trò chơi hay ứng dụng hoàn chỉnh. Nhưng chính nó lại là phần quan trọng nhất, vì mọi con đường dài đều bắt đầu từ bước đầu tiên.

Tôi cho rằng học lập trình giống như học cách nói chuyện với máy tính. Ban đầu, bạn chỉ cần học cách chào hỏi: in ra một dòng chữ, kiểm tra Python có chạy không, hiểu lỗi cú pháp là gì. Sau đó, bạn sẽ dần biết cách đặt câu lệnh có trật tự hơn, rồi tiến tới việc giải quyết vấn đề thật.

Vì vậy, nếu bạn làm xong chương này và cảm thấy những dòng code đầu tiên của mình "nhỏ bé", điều đó hoàn toàn bình thường. Điều quan trọng là bạn đã bắt đầu. Và trong lập trình, biết bắt đầu đúng cách luôn là một lợi thế rất lớn.

Chương 2

Nền tảng đầu tiên - Biến, Kiểu dữ liệu và Toán tử

Giới thiệu ngắn

Sau khi đã làm quen với Python và biết cách chạy chương trình đầu tiên, chúng ta bước sang một giai đoạn quan trọng hơn: học cách lưu trữ, xử lý và thao tác với dữ liệu. Nếu ở chương trước bạn đã học cách "nói chuyện" với máy tính bằng câu lệnh `print()`, thì ở chương này bạn sẽ học cách đưa dữ liệu vào bên trong chương trình, biến đổi dữ liệu đó và tạo ra kết quả có ý nghĩa.

Trong lập trình, gần như mọi bài toán đều xoay quanh dữ liệu. Một chương trình tính điểm trung bình, quản lý danh sách học sinh, tính tiền mua hàng, hay kiểm tra một điều kiện đúng/sai đều cần đến biến, kiểu dữ liệu và toán tử. Vì vậy, nắm vững chương này là một bước rất quan trọng để bạn không chỉ biết viết câu lệnh, mà còn bắt đầu hiểu cách máy tính xử lý thông tin.

Có thể nói, đây là chương học nền móng. Nếu ví lập trình như xây một ngôi nhà, thì biến là nơi cất giữ vật liệu, kiểu dữ liệu là cách phân loại vật liệu, còn toán tử là những công cụ giúp bạn lắp ghép và biến đổi chúng thành một cấu trúc hoàn chỉnh.

Trong toán học, đã từ lâu chúng ta đã làm quen với các phép tính (Cộng, trừ, nhân, chia, ...) Đơn giản, các biến và các phép toán tử được định nghĩa giống hoàn toàn với điều đó. Nhưng, điều chúng trở nên khác biệt là chúng ta không phải dùng bút và viết để diễn đạt ý tưởng một cách bình thường nữa mà chúng ta sử dụng hoàn toàn bằng code. Phải chăng việc lập trình sử dụng toán? Bản chất ngay từ đầu, mọi chương trình được tạo ra cũng dùng hoàn toàn bằng các phương trình toàn để phục vụ cho điều này.

Mục tiêu học tập

- Hiểu rõ khái niệm biến và biết cách khai báo, gán giá trị cho biến trong Python.
- Phân biệt được các kiểu dữ liệu cơ bản: số nguyên, số thực, chuỗi và logic.
- Sử dụng các toán tử số học, so sánh và logic để xử lý dữ liệu.

- Biết cách ép kiểu khi cần chuyển đổi dữ liệu từ dạng này sang dạng khác.
- Làm quen với tư duy làm việc theo từng bước, thay vì chỉ viết code một cách cảm tính.

2.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Trong Vật lý, công thức chuyển động thẳng biến đổi đều dùng quãng đường, vận tốc, gia tốc và thời gian. Trên giấy, ta có thể ngầm hiểu đơn vị từ đề bài. Trong chương trình, nếu trộn giây với giờ hoặc chuỗi `'9.5'` với số `9.5`, phép toán có thể lỗi hoặc tạo kết quả vô nghĩa. Biến và kiểu dữ liệu là cách gắn ý nghĩa có kiểm soát cho các đại lượng ấy.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **biến, kiểu dữ liệu và phép toán như một mô hình của đại lượng**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

2.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tên đại lượng, giá trị, kiểu biểu diễn, đơn vị, miền hợp lệ và độ chính xác cần thiết.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

2.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$s(t) = s_0 + v_0 t + \frac{1}{2} a t^2, \qquad [s] = \text{m},\ [t] = \text{s}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: giá trị hiện tại của mỗi đại lượng cùng giả định về kiểu và đơn vị của nó?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

2.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi phép toán chỉ được thực hiện trên những giá trị có ý nghĩa tương thích; sau phép gán, tên biến phải tiếp tục đại diện đúng cho đại lượng đã công bố.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

2.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **biến, kiểu dữ liệu và phép toán như một mô hình của đại lượng** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

2.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Xây dựng bộ tính quãng đường sau một khoảng thời gian. Trước khi dùng Python, ta lập bảng đơn vị, đổi vận tốc từ km/h sang m/s, thử trường hợp gia tốc bằng 0 và tính tay một bộ số. Sau đó mới quyết định dùng `int`, `float` hay `str` ở từng vị trí.

2.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

2.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

2.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

2.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

2.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

2.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?

5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

2.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Đặt tên biến theo hình dạng dữ liệu mà quên ý nghĩa vật lý của nó.
- Dùng `float` rồi cho rằng mọi chữ số thập phân đều được biểu diễn chính xác.
- Ghép chuỗi với số khi mục tiêu thực sự là cộng đại lượng.
- Đổi đơn vị sau khi tính thay vì chuẩn hóa ngay tại biên đầu vào.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

2.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

2.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Lập bảng kiểu và đơn vị cho bài tính chỉ số BMI, sau đó mới viết các biểu thức.
2. Giải thích vì sao `0.1 + 0.2` cần được so sánh với một sai số thay vì đòi bằng tuyệt đối.
3. Mô hình hóa tiền điện theo số kWh và đơn giá; kiểm tra các giá trị âm hoặc nhập dạng chuỗi.
4. Dùng công thức động năng để chỉ ra lỗi thứ nguyên trong một biểu thức được cho sẵn.
5. Thiết kế ba ca thử biên cho phép ép kiểu dữ liệu người dùng nhập.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và

ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

2.8 Biến (Variables) - Những chiếc hộp chứa dữ liệu

Trong toán học, chúng ta đã từng rất quen với các biến: x, y, z và luôn thường được gán với các giá trị nhất định để phục vụ cho việc tính toán. Trong học thuật, những biến số đó đã hoạt động như một ẩn độc lập; hoàn toàn có thể sử dụng để xử lý những bài toán phức tạp nếu thật sự chỉ rõ đến mô hình toán của nó.

Trong lập trình, **biến** là nơi dùng để lưu trữ dữ liệu. Bạn có thể hình dung biến giống như một chiếc hộp có dán nhãn. Phần *nhãn* chính là tên biến, còn phần *bên trong hộp* là giá trị mà biến đang giữ.

Ví dụ, nếu bạn có một chiếc hộp dán nhãn `ten_hoc_sinh`, thì bên trong nó có thể chứa chuỗi `"Nguyen Van A"`. Một chiếc hộp khác dán nhãn `diem_toan` có thể chứa số `9.5`. Mỗi hộp giữ một giá trị, và giá trị đó có thể thay đổi trong quá trình chương trình chạy.

Điểm đặc biệt của Python là bạn không cần khai báo trước kiểu của biến như trong C, C++ hay Java. Chỉ cần gán giá trị, Python sẽ tự hiểu kiểu dữ liệu của biến là gì. Đây là một ưu điểm rất lớn đối với người mới học vì nó giúp bạn tập trung vào ý tưởng và cách giải quyết bài toán thay vì phải ghi nhớ quá nhiều quy tắc phức tạp ngay từ đầu.

Khai báo biến

```
# Dat ten bien va gan gia tri
ten_hoc_sinh = "Nguyen Van A"
diem_toan = 9.5
print(ten_hoc_sinh)
print(diem_toan)
```

Trong ví dụ trên, chúng ta đã tạo ra ba biến khác nhau. Biến `ten_hoc_sinh` lưu một chuỗi ký tự, `diem_toan` lưu một số thực, còn `tuoi` lưu một số nguyên. Mỗi biến có một mục đích riêng, và điều đó giúp chương trình trở nên rõ ràng hơn.

2.8.1 Quy tắc đặt tên biến

Đặt tên biến không chỉ là chuyện "đặt sao cho nhớ", mà còn là một kỹ năng rất quan trọng trong lập trình. Một cái tên tốt sẽ giúp chương trình dễ đọc, dễ hiểu và dễ bảo trì hơn.

Một số quy tắc cơ bản cần nhớ:

- Tên biến chỉ nên chứa chữ cái, chữ số và dấu gạch dưới (_).
- Tên biến không được bắt đầu bằng số.
- Không dùng khoảng trắng trong tên biến.

- Python phân biệt chữ hoa và chữ thường, nên `tuoi` khác `TUOI`.
- Nên đặt tên rõ nghĩa, phản ánh đúng nội dung mà biến lưu trữ.

Khi mới học, nhiều bạn thường đặt tên biến rất ngắn như `a`, `b`, `x`, `y`. Cách này có thể dùng trong những ví dụ toán học đơn giản, nhưng trong chương trình thực tế, bạn nên ưu tiên những tên biến có ý nghĩa như `chieu_dai`, `can_nang`, `diem_trung_binh`, `so_hoc_sinh`.

Tên biến tốt sẽ giúp bạn đọc lại code sau vài ngày, vài tuần hoặc vài tháng mà vẫn hiểu mình đã làm gì.

Tên biến trùng từ khóa

Không được đặt tên biến trùng với các từ dành riêng (keywords) của Python như: `if`, `for`, `while`, `print`, `class`... Nếu làm vậy, chương trình sẽ báo lỗi cú pháp.

2.8.2 Gán lại giá trị cho biến

Một biến trong Python không bị "đóng khung" mãi mãi với một giá trị duy nhất. Bạn hoàn toàn có thể gán lại cho nó một giá trị mới.

Gán lại giá trị

```
diem = 8
print(diem)

diem = 9
print(diem)
```

Khi chạy đoạn chương trình trên, kết quả in ra sẽ thay đổi theo từng lần gán. Điều này cho thấy biến không chỉ dùng để lưu trữ, mà còn là công cụ để chương trình thay đổi trạng thái trong quá trình hoạt động.

2.9 Kiểu dữ liệu (Data Types)

Mỗi giá trị trong Python đều thuộc về một kiểu dữ liệu nhất định. Kiểu dữ liệu cho biết giá trị đó là loại gì, được lưu như thế nào và có thể thực hiện những thao tác gì.

Nếu biến là chiếc hộp chứa dữ liệu, thì kiểu dữ liệu chính là "loại đồ vật" nằm trong chiếc hộp đó. Một chiếc hộp có thể chứa số, chứa chữ, hoặc chứa giá trị đúng/sai. Và vì mỗi kiểu dữ liệu có đặc điểm khác nhau, nên cách xử lý chúng cũng khác nhau.

Bốn kiểu dữ liệu cơ bản nhất trong giai đoạn đầu là:

2.9.1 Số nguyên (`int`)

Số nguyên là những số không có phần thập phân, chẳng hạn như `0`, `5`, `-12`, `2026`. Trong lập trình, kiểu số nguyên thường được dùng cho tuổi, số lượng, điểm danh sách, số lần lặp, hoặc

Kiểu dữ liệu	Tên tiếng Anh	Ví dụ
Số nguyên	Integer (`int`)	`5, -10, 1000`
Số thực	Float (`float`)	`3.14, -0.5, 2.0`
Chuỗi ký tự	String (`str`)	`"Xin chao", "123"`
Luận lý	Boolean (`bool`)	`True, False`

Bảng 2.1: Các kiểu dữ liệu cơ bản

các đại lượng đếm được.

Ví dụ về số nguyên

```
so_hoc_sinh = 35
so_quyen_vo = 12
nam_hien_tai = 2026
```

2.9.2 Số thực (`float`)

Số thực là những số có phần thập phân, ví dụ `3.14`, `9.5`, `-0.75`. Kiểu dữ liệu này thường dùng khi cần độ chính xác cao hơn số nguyên, chẳng hạn như đo chiều cao, cân nặng, điểm trung bình, diện tích, vận tốc hoặc các giá trị khoa học.

Ví dụ về số thực

```
chieu_cao = 1.72
can_nang = 56.5
diem_trung_binh = 8.75
```

2.9.3 Chuỗi ký tự (`str`)

Chuỗi ký tự là tập hợp các ký tự nằm trong dấu nháy đơn `'...'` hoặc nháy kép `"..."`. Chuỗi không dùng để tính toán trực tiếp như số, mà dùng để lưu văn bản, tên, câu nói, địa chỉ, mã số hoặc bất kỳ dữ liệu nào được xem như văn bản.

Ví dụ về chuỗi

```
ho_ten = "Nguyen Van A"
lop_hoc = "12A1"
thong_bao = "Chao mung ban den voi Python"
```

Một điểm rất quan trọng là: nếu viết `123`, đó là số. Nhưng nếu viết `"123"`, đó là chuỗi. Hai giá trị này nhìn giống nhau, nhưng máy tính hiểu chúng hoàn toàn khác nhau.

Vì sao phải phân biệt số và chuỗi?

Nếu bạn muốn cộng, trừ, nhân, chia, thì giá trị phải là số. Nếu bạn muốn hiển thị văn bản, ghép tên, hoặc trình bày thông tin, thì thường dùng chuỗi.
Ví dụ, `123 + 5` là phép toán số học, còn `"123" + "5"` là phép nối chuỗi.

2.9.4 Giá trị logic (`bool`)

Kiểu logic chỉ có hai giá trị là `True` và `False`. Đây là kiểu dữ liệu rất quan trọng vì nó là nền tảng cho các câu lệnh điều kiện và vòng lặp ở những chương sau.

Ví dụ về kiểu logic

```
dang_hoc = True
da_lam_bai_tap = False
```

Kiểu logic thường dùng để trả lời các câu hỏi đúng/sai, chẳng hạn như:

- Học sinh đã hoàn thành bài tập chưa?
- Số này có lớn hơn 10 không?
- Mật khẩu nhập vào có đúng hay không?

Kiểu chuỗi (String)

Chuỗi luôn phải được bọc trong cặp dấu nháy đơn `'...'` hoặc nháy kép `"..."`. Nếu bạn viết `123`, đó là số (có thể tính toán được), còn `"123"` là chuỗi (chỉ là văn bản).

2.10 Toán tử (Operators)

Nếu ở trường phổ thông bạn đã quen với những biểu thức như

$$2x + 3, \quad \frac{a}{b}, \quad x^2 - 4x + 4,$$

thì trong Python, toán tử chính là phần giúp máy tính hiểu và thực hiện những phép biến đổi tương tự. Nói cách khác, toán tử là cầu nối giữa *ý tưởng toán học* và *hành động của chương trình*.

Trong Toán THPT, khi giải một bài toán, ta thường bắt đầu từ một biểu thức rồi biến đổi nó theo các quy tắc quen thuộc: cộng, trừ, nhân, chia, lũy thừa, so sánh lớn nhỏ, kiểm tra điều kiện đúng hay sai. Python cũng làm điều tương tự, nhưng thay vì viết trên giấy, ta viết bằng cú pháp lập trình. Vì vậy, hiểu toán tử không chỉ là học một phần của Python, mà còn là học cách chuyển tư duy toán học sang tư duy máy tính.

Toán tử trong Python được chia thành nhiều nhóm khác nhau. Ở giai đoạn đầu, bạn cần tập trung vào ba nhóm quan trọng nhất: toán tử số học, toán tử so sánh và toán tử logic.

Ba nhóm này xuất hiện rất nhiều trong các bài toán cơ bản, từ tính diện tích hình học, kiểm tra số chẵn lẻ, cho đến xét điều kiện một giá trị có thỏa mãn yêu cầu hay không.

2.10.1 Toán tử số học

Toán tử số học là nhóm toán tử quen thuộc nhất, vì chúng gần như trùng với các phép tính cơ bản trong Toán học. Nếu trong toán phổ thông bạn đã học cách thực hiện các phép cộng, trừ, nhân, chia, chia lấy dư và lũy thừa, thì trong Python các phép đó được biểu diễn bằng các ký hiệu tương ứng.

- **Cộng** `+`: dùng để cộng hai giá trị số.
- **Trừ** `-`: dùng để lấy hiệu giữa hai giá trị.
- **Nhân** `*`: dùng để nhân hai giá trị.
- **Chia** `/`: dùng để thực hiện phép chia thông thường.
- **Chia lấy phần nguyên** `//`: chỉ lấy phần nguyên của kết quả chia.
- **Chia lấy dư** `%`: lấy phần dư sau khi chia.
- **Lũy thừa** `**`: nâng một số lên lũy thừa.

Với người học Toán, nhóm toán tử này rất dễ tiếp cận vì bản chất của nó không khác nhiều so với ký hiệu toán học quen thuộc. Tuy nhiên, điều cần lưu ý là máy tính không chỉ nhìn biểu thức như con người nhìn trên giấy, mà còn phân biệt rất rõ kiểu dữ liệu của các giá trị tham gia phép toán. Vì vậy, một biểu thức chỉ chạy đúng khi các thành phần của nó có kiểu phù hợp.

Ví dụ về các phép toán số học

```
a = 10
b = 3

print("Tong:", a + b)
print("Hieu:", a - b)
print("Tich:", a * b)
print("Thuong:", a / b)
print("Chia lay phan nguyen:", a // b)
print("Chia lay du:", a % b)
print("Luy thua:", a ** b)
```

Trong các phép toán trên, phép chia lấy phần nguyên và chia lấy dư là hai phép rất đáng chú ý. Trong Toán THPT, bạn có thể đã gặp các bài toán liên quan đến tính chia hết, tìm số dư, xét tính chẵn lẻ, hoặc phân tích một số thành dạng:

$$a = bq + r, \quad 0 \le r < b.$$

Đó chính là cơ sở của phép chia lấy dư trong Python. Khi viết `a % b`, ta đang hỏi: sau khi chia `a` cho `b`, còn lại bao nhiêu?

Chính vì vậy, toán tử `%` xuất hiện rất thường xuyên trong các bài toán thực tế. Nó được dùng để kiểm tra một số có chia hết cho một số khác hay không, xác định số chẵn lẻ, tạo vòng lặp theo chu kỳ, hoặc xây dựng các điều kiện logic trong chương trình.

2.10.2 Thứ tự ưu tiên của toán tử

Trong Toán học, bạn đã quen với quy tắc:

$$\text{nhân, chia trước; cộng, trừ sau.}$$

Python cũng tuân theo nguyên tắc ưu tiên toán tử, và điều này rất quan trọng vì nó quyết định kết quả của biểu thức.

Ví dụ:

$$2 + 3 \times 4 = 14,$$

chứ không phải 20, vì phép nhân được thực hiện trước phép cộng. Nếu muốn thay đổi thứ tự tính toán, bạn phải dùng ngoặc.

Ưu tiên của toán tử

```
ket_qua_1 = 2 + 3 * 4
ket_qua_2 = (2 + 3) * 4

print(ket_qua_1)  # 14
print(ket_qua_2)  # 20
```

Điều này rất giống với cách bạn xử lý biểu thức trong đại số. Khi nhìn một công thức, không nên đọc máy móc từ trái sang phải, mà phải quan sát cấu trúc và thứ tự thực hiện. Kỹ năng đó trong toán học cũng chính là kỹ năng giúp bạn đọc code chính xác hơn.

2.10.3 Toán tử so sánh

Nếu toán học giúp ta tính toán giá trị, thì toán tử so sánh giúp ta trả lời câu hỏi: một điều gì đó có đúng hay không, có lớn hơn hay nhỏ hơn, có bằng nhau hay không. Trong các bài toán phổ thông, đây chính là phần liên quan đến dấu:

$$>, \quad <, \quad =, \quad \geq, \quad \leq, \quad \neq .$$

Trong Python, các toán tử này được dùng để so sánh hai giá trị và cho ra kết quả kiểu logic, tức là chỉ có `True` hoặc `False`. Đây là nền tảng của rất nhiều quyết định trong chương trình.

- `==`: bằng nhau.

- `!=`: khác nhau.
- `>`: lớn hơn.
- `<`: nhỏ hơn.
- `>=`: lớn hơn hoặc bằng.
- `<=`: nhỏ hơn hoặc bằng.

Ví dụ về so sánh

```
x = 7
y = 10

print(x == y)
print(x != y)
print(x < y)
print(x >= 5)
```

Trong Toán THPT, so sánh thường xuất hiện khi ta xét điều kiện của một bài toán: nghiệm có thỏa mãn hay không, một điểm có thuộc miền xác định hay không, một số có nằm trong khoảng cho trước hay không. Trong lập trình, các biểu thức so sánh được dùng theo đúng tinh thần đó. Mỗi biểu thức so sánh là một câu hỏi, và kết quả của nó là một câu trả lời đúng hoặc sai.

Điều này rất quan trọng vì từ những giá trị đúng/sai đó, chương trình có thể quyết định tiếp tục làm gì. Nói cách khác, so sánh là cửa ngõ dẫn đến tư duy điều kiện và rẽ nhánh trong lập trình.

2.10.4 Toán tử logic

Nếu toán tử so sánh tạo ra các mệnh đề đúng/sai đơn lẻ, thì toán tử logic giúp ta kết hợp nhiều mệnh đề lại với nhau. Trong Toán học, bạn có thể đã gặp những biểu thức như:

$$x > 0 \text{ và } x < 10,$$

hoặc:

$$x \leq 0 \text{ hoặc } x \geq 100.$$

Python cũng xử lý tương tự thông qua ba toán tử logic cơ bản:

- `and`: đúng khi cả hai điều kiện đều đúng.
- `or`: đúng khi ít nhất một điều kiện đúng.
- `not`: phủ định một giá trị logic.

Ví dụ về toán tử logic

```
diem = 8
co_chuyen_can = True

print(diem >= 5 and co_chuyen_can)
print(diem < 5 or co_chuyen_can)
print(not co_chuyen_can)
```

Trong toán phổ thông, khi giải bất phương trình hay xét miền nghiệm, ta thường phải kết hợp nhiều điều kiện cùng lúc. Toán tử logic chính là công cụ lập trình cho nhiệm vụ đó. Nó cho phép chương trình kiểm tra xem một đối tượng có thỏa mãn đồng thời nhiều tiêu chí hay không.

Một ví dụ rất gần gũi là kiểm tra học sinh được công nhận đạt hay không. Có thể điều kiện không chỉ là điểm số, mà còn là chuyên cần, hạnh kiểm, hoặc các tiêu chí khác. Khi đó, `and` và `or` giúp chương trình mô tả đúng tư duy của con người trong việc ra quyết định.

2.10.5 Ý nghĩa thực hành của toán tử

Với người mới học, toán tử không chỉ là ký hiệu kỹ thuật. Nó là cách để bạn chuyển một bài toán toán học thành một chuỗi thao tác cụ thể cho máy tính. Mỗi phép cộng, mỗi dấu so sánh, mỗi điều kiện logic đều là một phần của quá trình mô hình hóa bài toán.

Bạn nên hình thành thói quen đọc toán tử theo hai lớp:

- Ở lớp thứ nhất, đọc theo nghĩa toán học: cộng, trừ, lớn hơn, nhỏ hơn, và, hoặc...
- Ở lớp thứ hai, đọc theo nghĩa lập trình: thao tác với dữ liệu, tạo điều kiện, quyết định hành vi của chương trình.

Khi làm được điều đó, bạn sẽ không còn nhìn code như những dòng ký hiệu rời rạc nữa, mà sẽ thấy đó là một lời mô tả có cấu trúc của một bài toán quen thuộc.

2.11 Ép kiểu dữ liệu (Type Casting)

Trong toán học, ta thường làm việc với nhiều dạng biểu diễn khác nhau của cùng một đại lượng. Ví dụ, một số có thể được viết dưới dạng phân số, số thập phân hoặc biểu thức đại số. Trong Python, dữ liệu cũng có thể tồn tại ở nhiều kiểu khác nhau như số nguyên, số thực và chuỗi ký tự. Để chuyển từ kiểu này sang kiểu khác, ta dùng **ép kiểu dữ liệu**.

Ép kiểu là một thao tác rất quan trọng vì không phải lúc nào dữ liệu cũng ở đúng dạng mà bạn cần. Người dùng có thể nhập vào một con số nhưng dưới dạng chuỗi; một giá trị ban đầu là số nguyên có thể cần chuyển sang số thực để tính toán; hoặc một số cần được chuyển thành chuỗi để trình bày ra màn hình.

Ba hàm ép kiểu cơ bản nhất là:

- `int()`: chuyển dữ liệu sang số nguyên.
- `float()`: chuyển dữ liệu sang số thực.
- `str()`: chuyển dữ liệu sang chuỗi ký tự.

2.11.1 Ép kiểu từ chuỗi sang số

Đây là tình huống rất thường gặp. Ví dụ, khi người dùng nhập một con số qua bàn phím, Python xem dữ liệu đó là chuỗi. Nếu bạn muốn cộng, trừ, nhân, chia với nó, bạn phải chuyển nó sang kiểu số trước.

Ép kiểu từ chuỗi sang số

```
tuoi = "20"
tuoi_so = int(tuoi)
print(tuoi_so + 5)
```

Nếu không ép kiểu, biểu thức trên sẽ không hoạt động như mong muốn. Điều này cho thấy một điểm rất quan trọng: trong lập trình, **hình thức biểu diễn không quyết định bản chất của dữ liệu**. Cùng một chuỗi ký tự `"20"` có thể trông giống số, nhưng về mặt xử lý, nó vẫn chỉ là văn bản cho đến khi bạn chuyển nó thành số thật sự.

2.11.2 Ép kiểu giữa số nguyên và số thực

Trong nhiều bài toán toán học, ta không chỉ dùng số nguyên mà còn cần số thực. Chẳng hạn:

- Khi tính diện tích, chu vi, vận tốc, tỉ lệ phần trăm.
- Khi cần độ chính xác cao hơn số nguyên.
- Khi kết quả của phép chia không còn là số nguyên.

Lúc này, việc chuyển đổi giữa `int` và `float` là rất tự nhiên.

Chuyển đổi giữa số nguyên và số thực

```
a = 5
b = float(a)

c = 3.14
d = int(c)

print(b)
print(d)
```

Khi chuyển từ số thực sang số nguyên bằng `int()`, phần thập phân sẽ bị bỏ đi, không phải làm tròn. Đây là điểm người học cần chú ý. Ví dụ:

$$\texttt{int(3.99)} = 3$$

chứ không phải 4. Điều này khác với thói quen làm tròn trong một số ngữ cảnh toán học thông thường.

2.11.3 Ép kiểu sang chuỗi

Khi bạn muốn ghép một giá trị số vào câu văn, bạn thường cần đổi số thành chuỗi. Đây là lý do hàm `str()` rất hữu ích.

Ép kiểu sang chuỗi

```
diem = 9
thong_bao = "Diem cua ban la: " + str(diem)
print(thong_bao)
```

Trong trường hợp này, `str()` giúp máy tính hiểu rằng giá trị số `9` cần được xem như văn bản để nối vào câu chữ. Khi chưa học `f-string`, đây là một cách rất phổ biến để trình bày kết quả.

2.11.4 Vì sao ép kiểu quan trọng?

Ép kiểu không phải là một thủ thuật phụ, mà là một kỹ năng nền tảng. Nó xuất hiện ở rất nhiều nơi:

- Khi nhập dữ liệu từ bàn phím bằng `input()`.
- Khi tính toán với các giá trị số.
- Khi xuất dữ liệu ra màn hình dưới dạng chuỗi.
- Khi kết nối các phần của chương trình lại với nhau.

Nếu không hiểu ép kiểu, người học rất dễ gặp lỗi mà không biết nguyên nhân. Nhiều lỗi tưởng như do phép tính sai, nhưng thực chất chỉ là do dữ liệu chưa ở đúng kiểu. Vì vậy, ép kiểu là một trong những kiến thức nhỏ nhưng có ảnh hưởng rất lớn đến toàn bộ quá trình học lập trình.

2.11.5 Mối liên hệ giữa ép kiểu và tư duy toán học

Trong Toán THPT, ta thường học cách chuyển đổi biểu thức về dạng thuận tiện hơn để xử lý. Ví dụ:

$$\frac{a}{2} \text{ có thể cần viết thành } 0.5a,$$

hoặc một biểu thức phân số có thể được quy đồng trước khi tính. Trong lập trình, ép kiểu cũng mang tinh thần tương tự: đưa dữ liệu về dạng phù hợp để chương trình có thể xử lý đúng.

Có thể nói, ép kiểu là một dạng "quy đổi biểu diễn dữ liệu". Khi nắm được điều này, bạn sẽ thấy đây không phải là kiến thức rời rạc, mà là sự tiếp nối rất tự nhiên của tư duy toán học sang tư duy lập trình.

2.12 Bài tập luyện tập

Hãy thực hành các bài tập dưới đây để củng cố kiến thức về toán tử và ép kiểu dữ liệu. Mỗi bài đều được thiết kế theo hướng từ dễ đến khó, giúp bạn vừa ôn lại kiến thức toán học phổ thông, vừa làm quen với cách diễn đạt bài toán bằng Python.

1. Khai báo hai biến `a` và `b` có giá trị số nguyên. Hãy in ra kết quả của các phép toán: cộng, trừ, nhân, chia, chia lấy phần nguyên và chia lấy dư.
2. Khai báo một biến kiểu chuỗi chứa giá trị số, chẳng hạn `"15"`. Hãy ép kiểu biến đó sang số nguyên rồi cộng thêm 10.
3. Viết chương trình kiểm tra xem một số có phải số chẵn hay không bằng cách sử dụng toán tử `%`.
4. Cho ba biến `x`, `y`, `z`. Hãy kiểm tra xem `x` có vừa lớn hơn `y` vừa nhỏ hơn `z` hay không.
5. Viết chương trình nhập vào tuổi của một người dưới dạng chuỗi, sau đó ép kiểu sang số nguyên và tính tuổi của người đó sau 5 năm.
6. Viết chương trình nhập vào bán kính của một hình tròn, ép kiểu sang số thực, rồi tính diện tích hình tròn với công thức:
$$S = \pi r^2$$
với `pi` lấy gần đúng bằng `3.14`.
7. Giải thích vì sao lệnh sau bị lỗi và sửa lại cho đúng:
```
print("Diem cua ban la: " + 9)
```
8. Cho một số thực `3.99`. Hãy ép kiểu sang số nguyên và quan sát kết quả. Từ đó, hãy giải thích vì sao kết quả không phải là 4.
9. Viết một biểu thức logic để kiểm tra một học sinh đạt yêu cầu khi đồng thời thỏa mãn hai điều kiện: điểm lớn hơn hoặc bằng 5 và có chuyên cần.
10. Tạo một biểu thức phức hợp kết hợp cả toán tử số học, so sánh và logic, rồi dự đoán kết quả trước khi chạy chương trình.

2.13 Tóm tắt cuối chương

Nếu nhìn lại toàn bộ nội dung của chương này, ta có thể rút ra một số ý chính rất quan trọng:

- Toán tử là công cụ giúp chương trình thao tác với dữ liệu theo cách giống với tư duy toán học.
- Toán tử số học gồm cộng, trừ, nhân, chia, chia lấy phần nguyên, chia lấy dư và lũy thừa.
- Toán tử so sánh trả về kết quả đúng hoặc sai, và là nền tảng cho tư duy điều kiện.
- Toán tử logic cho phép kết hợp nhiều điều kiện lại với nhau.
- Trong Python, thứ tự ưu tiên của toán tử cần được tôn trọng, giống như quy tắc ưu tiên trong toán học.
- Ép kiểu dữ liệu là thao tác chuyển một giá trị từ kiểu này sang kiểu khác để chương trình có thể xử lý đúng.
- `int()`, `float()` và `str()` là ba hàm ép kiểu cơ bản cần ghi nhớ từ sớm.
- Không phải dữ liệu nào nhìn giống số cũng là số; kiểu dữ liệu bên trong mới là yếu tố quyết định cách máy tính xử lý.
- Nhiều lỗi lập trình thực ra bắt nguồn từ việc dùng sai kiểu dữ liệu chứ không phải do phép toán sai.

Khi nắm vững phần này, bạn đã có trong tay một lớp nền rất chắc để bước sang các chương tiếp theo. Từ đây, bạn không chỉ biết viết những câu lệnh rời rạc, mà đã bắt đầu hiểu cách một chương trình thực sự suy nghĩ, so sánh, tính toán và đưa ra phản hồi. Đó là bước chuyển rất quan trọng từ việc học cú pháp sang việc học tư duy lập trình.

Chương 3

Tương tác với chương trình - Nhập và Xuất dữ liệu

Giới thiệu ngắn

Trong Toán học ở bậc THPT, bạn thường không chỉ dừng lại ở việc tính toán trên giấy. Nhiều bài toán còn yêu cầu đọc dữ kiện từ đề bài, chọn đúng công thức, thực hiện phép biến đổi, rồi trình bày kết quả một cách rõ ràng và chính xác. Lập trình cũng hoạt động theo tinh thần đó. Một chương trình có ích không chỉ biết tính toán bên trong, mà còn phải biết nhận dữ liệu từ người dùng và trả lại kết quả theo cách dễ hiểu.

Có thể xem chương này như bước chuyển từ những phép tính "đóng" sang những bài toán "mở". Ở các chương trước, giá trị được gán sẵn trong chương trình. Sang chương này, chương trình bắt đầu làm việc với dữ liệu do người dùng cung cấp. Chính sự thay đổi đó khiến chương trình trở nên gần gũi hơn với thực tế, vì nó bắt đầu tương tác với con người chứ không chỉ vận hành một chiều.

Trong lập trình, **nhập dữ liệu** là quá trình đưa thông tin từ bên ngoài vào chương trình, còn **xuất dữ liệu** là quá trình trình bày kết quả ra màn hình. Hai thao tác này là nền tảng của hầu hết các chương trình cơ bản: máy tính điểm, tính tiền, khảo sát đơn giản, trò chơi nhỏ, hay các công cụ học tập. Vì vậy, hiểu rõ cách nhập và xuất dữ liệu là một bước rất quan trọng để bạn tiến từ việc viết câu lệnh rời rạc sang việc xây dựng một chương trình hoàn chỉnh.

Mục tiêu học tập

Sau khi học xong chương này, bạn cần:

- Hiểu được vai trò của nhập và xuất dữ liệu trong một chương trình Python.
- Sử dụng được hàm `input()` để nhận dữ liệu từ người dùng.
- Hiểu rằng `input()` luôn trả về chuỗi, dù dữ liệu người dùng nhập vào có thể là số.
- Biết ép kiểu sang `int()` hoặc `float()` khi cần xử lý số.
- Sử dụng được `print()` để trình bày kết quả ra màn hình một cách rõ ràng.

- Làm quen với `sep`, `end` và `f-string` để định dạng đầu ra.

3.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một biểu mẫu tính điểm trung bình nhận ba điểm số và trả về kết quả có hai chữ số thập phân. Người dùng có thể gõ khoảng trắng, dùng dấu phẩy thập phân hoặc nhập một điểm ngoài thang 10. Nếu chỉ gọi `input()` rồi ép kiểu, chương trình đã bỏ qua phần khó nhất: định nghĩa hợp đồng giao tiếp và đường đi khi dữ liệu không hợp lệ.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **nhập–xử lý–xuất như một hợp đồng giao tiếp**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

3.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: chuỗi nhập thô, quy tắc chuẩn hóa, miền điểm hợp lệ, công thức tổng hợp và định dạng đầu ra.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

3.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$y = G(P(V(N(x)))), \quad N : \text{nhập thô},\ V : \text{kiểm tra},\ P : \text{xử lý},\ G : \text{trình bày}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng

thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: dữ liệu thô, dữ liệu đã chuẩn hóa, trạng thái hợp lệ và thông báo cần phản hồi?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

3.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Chỉ dữ liệu vượt qua toàn bộ kiểm tra mới được chuyển sang bước tính; đầu ra phải nói rõ giá trị, đơn vị và cách làm tròn.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

3.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **nhập–xử lý–xuất như một hợp đồng giao tiếp** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình,

một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

3.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế bộ tính điểm trung bình có trọng số cho ba thành phần. Ta ghi rõ trọng số phải có tổng bằng 1, điểm nằm trong đoạn từ 0 đến 10, kết quả được làm tròn để hiển thị nhưng giá trị nội bộ vẫn giữ độ chính xác cho bước sau.

3.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

3.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

3.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

3.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

3.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

3.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

3.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Cho rằng `input()` trả về số chỉ vì người dùng gõ chữ số.
- Ép kiểu trước khi chuẩn hóa khiến thông báo lỗi không giúp người dùng sửa dữ liệu.
- Dùng `print()` ở khắp nơi làm lẫn bước tính với bước trình bày.
- Làm tròn quá sớm rồi dùng giá trị đã làm tròn cho các phép tính tiếp theo.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

3.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

3.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết đặc tả đầu vào cho một máy tính diện tích hình tròn, gồm đơn vị và miền hợp lệ.
2. Thiết kế thông báo khác nhau cho chuỗi rỗng, sai kiểu và số ngoài miền.
3. So sánh ba cách trình bày tiền tệ và giải thích cách tách giá trị khỏi chuỗi hiển thị.
4. Mô phỏng bằng tay đường đi của năm dữ liệu nhập, trong đó có ba dữ liệu sai.
5. Xây dựng giao diện terminal cho phép người dùng hiểu phải sửa gì sau mỗi lỗi.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

3.8 Hàm nhập liệu input()

Trong các bài toán Toán THPT, đề bài thường cho sẵn dữ kiện. Tuy nhiên, trong thực tế, chương trình không thể lúc nào cũng biết trước người dùng sẽ nhập gì. Vì vậy, Python cung cấp hàm `input()` để chương trình có thể nhận dữ liệu trực tiếp từ bàn phím.

Về mặt hoạt động, `input()` làm cho chương trình tạm dừng tại thời điểm được gọi, chờ người dùng gõ thông tin rồi nhấn Enter. Sau khi người dùng hoàn tất việc nhập, dữ liệu đó sẽ được trả về cho chương trình để tiếp tục xử lý.

Điều rất quan trọng cần nhớ là: **`input()` luôn trả về một chuỗi (`str`)**, bất kể người dùng nhập vào chữ, số hay ký hiệu. Đây là điểm dễ gây nhầm lẫn nhất đối với người mới học. Nhiều bạn nhìn thấy người dùng nhập `18` thì nghĩ đó là số nguyên, nhưng với Python, đó vẫn là chuỗi `"18"` cho đến khi bạn ép kiểu.

Sử dụng `input()`

```
ten = input("Nhap ten cua ban: ")
print("Xin chao,", ten)
```

Ở ví dụ trên, chương trình yêu cầu người dùng nhập tên rồi in ra lời chào tương ứng. Đây là dạng tương tác đơn giản nhất, nhưng lại rất quan trọng vì nó giúp bạn hiểu bản chất của việc chương trình nhận dữ liệu từ bên ngoài.

3.8.1 Cách hiểu `input()` theo tư duy bài toán

Nếu nhìn dưới góc độ Toán học, `input()` giống như việc bạn đọc dữ kiện từ đề bài. Trước khi giải, bạn phải biết đề cho gì. Trong lập trình, `input()` đóng vai trò tương tự như cửa nhận dữ kiện. Chương trình không tự sinh ra mọi thông tin, mà phải chờ người dùng cung cấp.

Điều này dẫn đến một tư duy rất quan trọng: một chương trình tốt không chỉ biết làm phép toán, mà còn biết hỏi đúng thông tin cần thiết. Nếu bài toán yêu cầu tuổi, tên, chiều cao hay bán kính, thì chương trình phải chủ động yêu cầu người dùng nhập các giá trị đó. Nhờ vậy, một chương trình có thể được dùng nhiều lần với nhiều dữ liệu khác nhau, thay vì chỉ chạy với một trường hợp cố định.

3.8.2 Tại sao `input()` luôn trả về chuỗi?

Python chọn cách này vì chuỗi là kiểu dữ liệu linh hoạt nhất để tiếp nhận thông tin từ bàn phím. Khi người dùng gõ phím, hệ thống ghi nhận đầu vào dưới dạng văn bản trước. Sau đó, tùy mục đích của chương trình, ta mới chuyển đổi nó sang kiểu phù hợp.

Cách thiết kế này giúp Python đơn giản hơn trong phần nhập liệu, nhưng đồng thời cũng đặt trách nhiệm lên người lập trình: bạn phải biết khi nào cần giữ nguyên chuỗi, khi nào cần chuyển sang số.

Ví dụ, nếu bạn hỏi tên học sinh thì kết quả nên giữ ở dạng chuỗi. Nhưng nếu bạn hỏi

tuổi hoặc điểm số, bạn thường cần ép kiểu sang số để có thể tính toán. Đó là lý do `input()` và ép kiểu luôn đi cùng nhau trong các chương trình thực hành.

Lỗi thường gặp với `input()`

Vì `input()` trả về chuỗi, nếu bạn lấy kết quả đó đem cộng trực tiếp với số thì chương trình sẽ báo lỗi.

```
# SAI
tuoi = input("Nhap tuoi: ")
print(tuoi + 1)

# DUNG
tuoi = int(input("Nhap tuoi: "))
print(tuoi + 1)
```

Trong trường hợp đầu tiên, `tuoi` là chuỗi nên không thể cộng trực tiếp với số nguyên. Trong trường hợp thứ hai, dữ liệu đã được chuyển sang `int`, nên phép cộng hoạt động bình thường.

3.9 Hàm xuất liệu `print()` nâng cao

Nếu `input()` là cánh cửa để đưa dữ liệu vào chương trình, thì `print()` là công cụ để đưa kết quả từ chương trình ra bên ngoài. Ở chương đầu tiên, bạn đã làm quen với `print()` như một cách đơn giản để hiển thị văn bản. Tuy nhiên, trên thực tế, `print()` còn linh hoạt hơn nhiều.

Trong các bài toán hay bảng biểu ở Toán THPT, cách trình bày kết quả rất quan trọng. Một lời giải đúng nhưng trình bày rối rắm sẽ khó theo dõi. Cũng như vậy, một chương trình có kết quả đúng nhưng in ra lộn xộn sẽ khó đọc và khó kiểm tra. Vì vậy, việc biết cách điều chỉnh đầu ra là một phần thiết yếu trong lập trình.

3.9.1 `sep` - ký tự phân cách

Khi `print()` nhận nhiều đối số, Python sẽ chèn khoảng trắng mặc định giữa chúng. Nếu muốn thay đổi ký tự phân cách, bạn dùng tham số `sep`.

Tùy chỉnh `print()`

```
print("Python", "la", "de", sep="-")
# Ket qua: Python-la-de

print("Dang tai", end="...")
print("Xong!")
# Ket qua tren 1 dong: Dang tai...Xong!
```

Trong thực hành, sep hữu ích khi bạn muốn in ngày tháng, mã số, danh sách hoặc các cụm thông tin cần nối liền nhau. Đây là một chi tiết nhỏ nhưng giúp đầu ra của chương trình trở nên rõ ràng và có tổ chức hơn.

3.9.2 end - ký tự kết thúc dòng

Mặc định, sau mỗi lần print(), Python sẽ xuống dòng. Tuy nhiên, đôi khi bạn muốn nhiều lệnh print() cùng nằm trên một dòng. Khi đó, end là tham số cần dùng.

Ví dụ, nếu bạn muốn in ra thông báo đang xử lý và sau đó tiếp tục in trạng thái hoàn tất trên cùng một dòng, end sẽ giúp bạn làm điều đó. Kiểu trình bày này thường được dùng để tạo hiệu ứng hoặc thể hiện tiến trình.

Về mặt tư duy, sep và end cho bạn quyền kiểm soát cách chương trình "nói chuyện" với người dùng. Điều này quan trọng không kém việc chương trình tính đúng, vì trình bày tốt giúp người dùng hiểu kết quả nhanh hơn.

3.10 Định dạng chuỗi với f-string

Khi chương trình có nhiều biến, việc ghép chuỗi bằng dấu cộng sẽ nhanh chóng trở nên dài dòng. Ví dụ, nếu muốn ghép tên, tuổi và điểm số vào một câu, bạn phải chuyển kiểu rất cẩn thận. Để giải quyết vấn đề này, Python cung cấp f-string.

f-string là một cách viết chuỗi hiện đại, ngắn gọn và dễ đọc. Bạn chỉ cần đặt chữ f trước chuỗi, rồi đặt biến hoặc biểu thức trong dấu ngoặc nhọn {}. Python sẽ tự thay giá trị tương ứng vào vị trí đó.

Sử dụng f-string

```
mon_hoc = "Python"
diem = 9.5
print(f"Mon {mon_hoc} dat diem {diem}")

# Co the thuc hien phep tinh ben trong {}
x = 10
y = 3
print(f"Tich cua x va y la: {x * y}")
```

3.10.1 Vì sao f-string quan trọng?

Trong các bài toán trình bày kết quả, bạn thường cần kết hợp văn bản với giá trị biến. Nếu làm bằng cách nối chuỗi thủ công, chương trình sẽ dễ bị rối, đặc biệt khi có nhiều biến cùng xuất hiện. f-string giúp code gọn hơn, rõ hơn và gần với cách con người diễn đạt tự nhiên hơn.

Ngoài ra, f-string còn cho phép bạn viết biểu thức trực tiếp bên trong dấu ngoặc nhọn. Điều đó có nghĩa là bạn không chỉ chèn biến, mà còn có thể chèn cả kết quả của một phép

tính. Đây là điểm khiến `f-string` trở thành một công cụ rất hữu ích trong thực tế.

Tại sao nên dùng `f-string`?

Trước đây, người ta thường dùng `%` hoặc `.format()`, nhưng cú pháp thường dài dòng và khó đọc hơn. `f-string` làm cho việc trình bày chuỗi trở nên trực quan, dễ theo dõi và phù hợp hơn với người mới học. Vì vậy, đây là cách nên tạo thói quen sử dụng từ sớm.

3.11 Nhập dữ liệu số và ép kiểu

Trong Toán học, nếu bạn muốn tính diện tích hình tròn, bạn cần bán kính là một giá trị số. Tương tự, nếu bạn muốn tính tuổi sau một năm, bạn cần tuổi là số nguyên. Nhưng như đã nói, `input()` luôn trả về chuỗi. Vì vậy, nếu chương trình cần xử lý số, bạn phải ép kiểu ngay sau khi nhập.

Hai kiểu ép kiểu hay dùng nhất là `int()` và `float()`:

- `int()` dùng khi dữ liệu cần là số nguyên.
- `float()` dùng khi dữ liệu cần là số thực.

Nhập số nguyên

```
tuoi = int(input("Nhap tuoi cua ban: "))
print(f"Nam sau ban se {tuoi + 1} tuoi")
```

Nhập số thực

```
ban_kinh = float(input("Nhap ban kinh hinh tron: "))
dien_tich = 3.14 * ban_kinh * ban_kinh
print(f"Dien tich hinh tron la: {dien_tich}")
```

Khi ép kiểu đúng, bạn mới có thể thực hiện các phép toán phù hợp với ý nghĩa bài toán. Đây là một thói quen rất quan trọng: trước khi tính toán, hãy hỏi xem dữ liệu cần ở kiểu gì.

3.12 Kết hợp nhập, xử lý và xuất dữ liệu

Một chương trình cơ bản thường có ba phần: nhận đầu vào, xử lý và xuất kết quả. Đây chính là mô hình hoạt động đơn giản nhất của hầu hết các phần mềm.

Nếu nhìn theo cách này, một chương trình giống như quá trình giải một bài toán Toán THPT:

- Đầu vào là dữ kiện của đề bài.

- Xử lý là các phép biến đổi, tính toán, so sánh.
- Đầu ra là kết quả cuối cùng hoặc lời giải.

Chương trình hoàn chỉnh

```
ten = input("Nhap ten: ")
tuoi = int(input("Nhap tuoi: "))
diem_toan = float(input("Nhap diem Toan: "))

print(f"Xin chao {ten}!")
print(f"Nam sau ban se {tuoi + 1} tuoi.")
print(f"Diem Toan cua ban la {diem_toan}")
```

Chỉ với vài dòng lệnh, bạn đã tạo ra một chương trình có thể nhận thông tin, tính toán và trình bày kết quả theo cách rõ ràng. Đó là một bước tiến rất quan trọng so với việc chỉ in một dòng chữ cố định.

3.13 Bài tập luyện tập

Hãy thực hành các bài tập dưới đây để củng cố kiến thức. Mỗi bài đều được xây dựng theo hướng vừa luyện cú pháp, vừa rèn tư duy mô hình hóa bài toán.

1. Viết chương trình yêu cầu người dùng nhập tên và tuổi, sau đó in ra câu: *Chào [tên], năm sau bạn sẽ [tuổi + 1] tuổi.*
2. Nhập vào bán kính của một đường tròn, tính và in ra diện tích với công thức $S = \pi r^2$, dùng `3.14` cho π.
3. Nhập vào một số nguyên và in ra bình phương của số đó.
4. Nhập vào chiều dài và chiều rộng của hình chữ nhật, sau đó tính chu vi và diện tích.
5. Viết chương trình yêu cầu người dùng nhập ba thông tin: họ tên, lớp học và điểm trung bình, rồi in ra thành một đoạn giới thiệu ngắn.
6. Nhập vào hai số, sau đó in ra tổng, hiệu, tích và thương của chúng.

3.14 Tóm tắt cuối chương

Nếu nhìn lại toàn bộ chương này, ta có thể rút ra những ý chính sau:

- `input()` dùng để nhận dữ liệu từ người dùng và luôn trả về chuỗi.
- Khi cần xử lý số, bạn phải ép kiểu bằng `int()` hoặc `float()`.
- `print()` có thể được tùy chỉnh bằng `sep` và `end`.
- `f-string` là cách hiện đại, ngắn gọn và dễ đọc để chèn giá trị biến vào chuỗi.

- Một chương trình cơ bản thường gồm ba bước: nhập, xử lý và xuất.
- Hiểu đúng kiểu dữ liệu là điều kiện cần để chương trình chạy đúng.

Khi nắm được chương này, bạn đã bắt đầu làm quen với cách một chương trình tương tác với con người. Từ đây, lập trình không còn là những dòng lệnh khép kín nữa, mà trở thành một công cụ biết hỏi, biết xử lý và biết trả lời. Đó chính là bước đầu của việc xây dựng những chương trình có ích trong thực tế.

Chương 4

Rẽ nhánh chương trình - Câu lệnh điều kiện

Giới thiệu ngắn

Trong đời sống lẫn trong Toán học, không phải lúc nào mọi việc cũng đi theo một con đường cố định. Có những tình huống phụ thuộc vào điều kiện cụ thể: nếu đủ điểm thì qua môn, nếu không đủ thì học lại; nếu một số chia hết cho 2 thì là số chẵn, còn nếu không thì là số lẻ; nếu một biểu thức thỏa mãn một miền xác định nào đó thì ta mới tiếp tục xét, còn nếu không thì phải loại bỏ. Tư duy điều kiện chính là tư duy *nếu – thì – ngược lại*, và đó cũng là nền tảng của mọi quyết định trong lập trình.

Ở trường phổ thông, khi giải các bài toán đại số hoặc bất phương trình, bạn đã quen với việc chia một tình huống thành nhiều trường hợp. Ví dụ, khi xét dấu của một biểu thức, ta phải tách theo các khoảng; khi giải phương trình có điều kiện, ta phải kiểm tra nghiệm có thỏa mãn hay không; khi xét tam thức bậc hai, ta lại dựa vào dấu của hệ số và biệt thức để kết luận. Lập trình điều khiển chương trình theo đúng tinh thần đó: chương trình không chạy một cách mù quáng, mà kiểm tra dữ kiện trước rồi mới quyết định hành động.

Chính vì vậy, câu lệnh điều kiện là một trong những công cụ quan trọng nhất trong Python. Nhờ nó, chương trình có thể phản hồi linh hoạt, xử lý nhiều trường hợp khác nhau, và trở nên gần với thực tế hơn rất nhiều. Từ đây, bạn sẽ không còn chỉ viết những dòng lệnh để in ra kết quả cố định, mà bắt đầu biết cách làm cho chương trình "suy nghĩ" trước khi hành động.

Mục tiêu học tập

Sau khi học xong chương này, bạn cần:

- Hiểu ý nghĩa của cấu trúc điều kiện trong lập trình và vai trò của nó trong việc ra quyết định.
- Sử dụng thành thạo các khối lệnh `if`, `if-else` và `if-elif-else`.
- Hiểu quy tắc thụt lề trong Python và biết vì sao nó quyết định cấu trúc của chương

trình.

- Biết kết hợp các toán tử so sánh và toán tử logic trong điều kiện.
- Biết phân tích một bài toán thành nhiều nhánh xử lý khác nhau.
- Làm quen với tư duy mô hình hóa bài toán bằng các điều kiện đúng hoặc sai.

4.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một chương trình xét học bổng dựa trên điểm trung bình, số buổi vắng và điều kiện kỷ luật. Cùng một học sinh có thể thỏa một tiêu chí nhưng không thỏa tiêu chí khác. Nếu các nhánh được viết theo cảm giác, thứ tự `if--elif--else` có thể loại sai người. Ta cần biến quy định bằng lời thành tập điều kiện không chồng chéo và có thể giải thích.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **câu lệnh điều kiện như hàm từng đoạn và cây quyết định**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

4.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: điểm trung bình (g), số buổi vắng (a), trạng thái kỷ luật (d), thứ tự ưu tiên và lý do quyết định.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

4.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$f(x) = \begin{cases} \text{đủ điều kiện}, & g \geq 8.0 \land a \leq 2 \land d = 0, \\ \text{cần xem xét}, & g \geq 7.5 \land a \leq 3, \\ \text{chưa đủ}, & \text{còn lại.} \end{cases}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: các mệnh đề đã kiểm tra, nhánh đang xét và lý do được chọn?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

4.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi đầu vào hợp lệ phải rơi vào đúng một nhánh; hai hồ sơ giống nhau phải nhận cùng quyết định và cùng lời giải thích.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

4.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **câu lệnh điều kiện như hàm từng đoạn và cây quyết định** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi

những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

4.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Chuyển một quy chế học bổng thành bảng quyết định. Trước hết liệt kê từng tiêu chí, sau đó tìm vùng giao nhau, sắp xếp nhánh đặc biệt trước nhánh tổng quát, rồi dùng các điểm sát ngưỡng 7.49, 7.50, 7.99 và 8.00 để kiểm tra.

4.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

4.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

4.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

4.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

4.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

4.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?

5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

4.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Viết điều kiện tổng quát trước làm nhánh đặc biệt không bao giờ được chạy.
- Nhầm `and` với `or` vì không dịch mệnh đề sang ngôn ngữ tự nhiên.
- Dùng nhiều `if` độc lập khi yêu cầu chỉ cho phép một kết quả.
- Chỉ thử giá trị giữa miền mà không thử đúng tại biên.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

4.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

4.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ trục số và chương trình phân loại nghiệm của một bất phương trình bậc nhất.
2. Dựng bảng chân trị cho ba điều kiện trong bài xét học bổng.
3. Thiết kế cây quyết định phân loại tam giác từ ba cạnh, gồm cả dữ liệu không tạo thành tam giác.
4. Tìm phản ví dụ cho một đoạn `if--elif` xếp loại học lực sai thứ tự.
5. Viết ca kiểm thử sát mọi ngưỡng của biểu giá theo bậc.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn

phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

4.8 Ý nghĩa của câu lệnh điều kiện

Trong Toán THPT, khi giải một bài toán, ta không phải lúc nào cũng đi theo một công thức duy nhất. Rất nhiều bài yêu cầu xét trường hợp, chia miền, hoặc kiểm tra điều kiện để kết luận. Ví dụ, khi giải phương trình bậc hai, bạn phải xét biệt thức để biết phương trình có bao nhiêu nghiệm; khi khảo sát hàm số, bạn phải xét dấu đạo hàm để kết luận sự đồng biến hay nghịch biến; khi tìm giá trị lớn nhất, nhỏ nhất, ta thường so sánh nhiều khả năng khác nhau trước khi chọn ra kết quả cuối cùng.

Nếu nhìn rộng hơn, đây chính là tư duy ra quyết định. Một người học giỏi toán không chỉ biết biến đổi công thức, mà còn biết nhìn vào dữ kiện để xác định nên đi theo hướng nào. Lập trình cũng vậy. Câu lệnh điều kiện cho phép chương trình nhìn vào một điều kiện và đưa ra hành động tương ứng. Nếu điều kiện đúng, chương trình thực hiện một khối lệnh; nếu điều kiện sai, chương trình có thể bỏ qua hoặc chuyển sang một nhánh khác. Đây là điểm rất quan trọng vì nó biến chương trình từ dạng "thực hiện một chuỗi câu lệnh cố định" thành dạng "tự lựa chọn hành vi tùy theo dữ kiện".

Có thể nói, điều kiện là chiếc cầu nối giữa dữ liệu và quyết định. Dữ liệu cho ta biết tình huống hiện tại, còn điều kiện giúp ta xác định nên làm gì tiếp theo. Trong các chương trình thực tế, đặc biệt là những chương trình có giao tiếp với người dùng, câu lệnh điều kiện xuất hiện liên tục. Từ kiểm tra tuổi, điểm số, mật khẩu, cho đến xét trạng thái đơn hàng hay hợp lệ của dữ liệu nhập vào, tất cả đều cần điều kiện.

4.9 Câu lệnh `if`

Cấu trúc `if` là dạng điều kiện cơ bản nhất trong Python. Nó có ý nghĩa như sau: nếu điều kiện đúng thì thực hiện khối lệnh bên trong, nếu điều kiện sai thì bỏ qua khối lệnh đó. Đây là một khái niệm rất tự nhiên, gần như tương ứng với cách con người suy nghĩ trong đời sống hằng ngày.

Ví dụ, ta có thể nói: nếu trời mưa thì mang áo mưa; nếu điểm lớn hơn hoặc bằng 5 thì đạt; nếu số chia hết cho 2 thì là số chẵn. Tất cả đều là những quyết định dựa trên điều kiện.

Câu lệnh `if` đơn giản

```
1 diem = 8
2 if diem >= 5:
3     print("Ban da qua mon!")
```

Trong ví dụ trên, chương trình sẽ kiểm tra xem biến `diem` có lớn hơn hoặc bằng 5 hay không. Nếu đúng, dòng thông báo sẽ được in ra. Nếu sai, chương trình sẽ không làm gì thêm ở khối lệnh này.

4.9.1 Cấu trúc tổng quát của `if`

Một câu lệnh `if` thường có cấu trúc:

```
if dieu_kien:
    khoi_lenh
```

Ở đây, `dieu_kien` phải là một biểu thức cho ra giá trị đúng hoặc sai, tức là kiểu logic (`True` hoặc `False`). Điều này rất quan trọng vì Python chỉ thực thi khối lệnh bên trong khi điều kiện có giá trị đúng.

Ví dụ, trong bài toán kiểm tra số chẵn, ta có thể viết:

```
so = 10
if so % 2 == 0:
    print("So nay la so chan")
```

Đây là một cách biểu đạt rất gần với toán học: điều kiện `so % 2 == 0` chính là cách nói rằng số đó chia hết cho 2. Nếu điều kiện đúng, ta kết luận số chẵn.

4.9.2 Vai trò của điều kiện trong tư duy giải toán

Ở mức độ tư duy sâu hơn, câu lệnh `if` không chỉ là một cú pháp của ngôn ngữ lập trình mà còn là công cụ mô phỏng chính xác cách con người suy luận khi giải quyết vấn đề. Hầu hết các bài toán trong Toán học, Vật lý hay trong cuộc sống đều không thể được giải bằng một chuỗi thao tác cố định áp dụng cho mọi trường hợp. Thay vào đó, người giải luôn phải quan sát dữ kiện, xác định các điều kiện cần thiết, rồi từ những điều kiện đó lựa chọn hướng giải phù hợp. Chính quá trình "nếu... thì..." này là nền tảng của tư duy logic, và `if` là cách để máy tính thực hiện chính kiểu tư duy đó.

Trong chương trình Toán học phổ thông, chúng ta thường gặp rất nhiều bài toán được chia thành nhiều trường hợp khác nhau. Chẳng hạn, khi giải một phương trình chứa dấu giá trị tuyệt đối, trước tiên ta phải xét biểu thức bên trong dấu giá trị tuyệt đối là dương hay âm để bỏ dấu một cách chính xác. Khi khảo sát hàm số, ta phải xác định miền xác định trước khi thực hiện các phép biến đổi tiếp theo. Khi giải phương trình bậc hai, việc xét giá trị của biệt thức Δ quyết định phương trình có hai nghiệm phân biệt, một nghiệm kép hay vô nghiệm trong tập số thực. Trong hình học, nhiều định lý chỉ được áp dụng khi các giả thiết đã được thỏa mãn; nếu điều kiện không đúng thì kết luận cũng không còn giá trị. Mỗi bước như vậy đều là một quá trình kiểm tra điều kiện trước khi đưa ra quyết định tiếp theo.

Lập trình cũng vận hành theo đúng nguyên tắc đó. Máy tính không có khả năng tự suy đoán hay hiểu ngữ cảnh như con người mà chỉ thực hiện chính xác những gì được lập trình viên mô tả. Nếu chương trình không được hướng dẫn cách xử lý các trường hợp khác nhau, nó sẽ luôn thực hiện cùng một chuỗi lệnh, bất kể dữ liệu đầu vào là gì. Điều này khiến chương trình trở nên cứng nhắc và không thể phản ứng trước những tình huống đa dạng của thực tế. Vì vậy, lập trình viên phải chỉ rõ rằng trong trường hợp nào chương trình cần thực hiện hành động nào, và câu lệnh `if` chính là công cụ để biểu diễn những quyết định đó.

Có thể hình dung rằng mỗi câu lệnh `if` giống như một nút rẽ trên một con đường. Khi chương trình đến nút rẽ này, nó sẽ đánh giá một điều kiện. Nếu điều kiện đúng, chương trình đi theo một hướng; nếu điều kiện sai, chương trình sẽ đi theo hướng khác hoặc tiếp tục kiểm tra những điều kiện tiếp theo. Bằng cách kết hợp nhiều nút rẽ như vậy, một chương trình có thể xử lý được những bài toán rất phức tạp, mặc dù mỗi quyết định riêng lẻ đều khá đơn giản. Đây cũng chính là tư duy phân chia bài toán thành các trường hợp nhỏ hơn, giúp việc giải quyết vấn đề trở nên rõ ràng và có hệ thống.

Trong thực tế, hầu như mọi chương trình đều phải đưa ra các quyết định dựa trên dữ liệu đầu vào. Một học sinh có thể đạt hoặc không đạt môn học tùy thuộc vào điểm số. Một số nguyên có thể là số chẵn hoặc số lẻ. Một người có thể đủ hoặc chưa đủ tuổi để đăng ký dịch vụ. Một tài khoản có thể nhập đúng hoặc sai mật khẩu. Một đơn hàng có thể hợp lệ hoặc không hợp lệ. Một cảm biến nhiệt độ có thể báo mức an toàn hoặc cảnh báo quá nhiệt. Trong tất cả những tình huống này, chương trình đều phải kiểm tra điều kiện trước khi lựa chọn hành động tiếp theo. Không có điều kiện, chương trình sẽ không thể phân biệt được các trường hợp khác nhau và do đó cũng không thể phản ứng một cách chính xác.

Có thể nói rằng, việc học câu lệnh `if` không đơn thuần là học thêm một cú pháp mới của Python mà còn là bước chuyển quan trọng trong tư duy lập trình. Từ đây, chương trình không còn chỉ thực hiện tuần tự các lệnh từ trên xuống dưới mà đã bắt đầu biết đưa ra quyết định dựa trên thông tin nhận được. Đây là nền tảng để xây dựng các thuật toán thông minh hơn, xử lý những bài toán đa dạng hơn và phát triển các ứng dụng gần gũi với đời sống. Sau khi thành thạo `if`, người học sẽ có cơ sở để tiếp cận những cấu trúc điều khiển phức tạp hơn như `if-else`, `if-elif-else`, các biểu thức logic, cũng như các thuật toán có nhiều nhánh xử lý trong những chương tiếp theo.

4.10 Quy tắc thụt lề trong Python

Python không dùng ngoặc nhọn `{}` để nhóm các khối lệnh như C, C++ hay Java. Thay vào đó, Python sử dụng **thụt lề** để nhận biết đâu là phần nội dung thuộc về câu lệnh điều kiện.

Điều này có nghĩa là tất cả các dòng nằm trong cùng một khối phải được thụt vào cùng mức. Thông thường, người ta dùng 4 khoảng trắng cho một mức thụt lề. Nếu bạn viết sai thụt lề, Python sẽ không hiểu đâu là khối lệnh đi kèm điều kiện và sẽ báo lỗi.

Quy tắc thụt lề

Python rất nghiêm ngặt với khoảng trắng. Nếu một khối lệnh bị thụt lề sai, chương trình sẽ báo lỗi `IndentationError`. Vì vậy, hãy luôn giữ cho phần nội dung bên trong `if`, `else`, `elif`, vòng lặp hay hàm có cùng mức thụt lề.

Cách viết đúng

```
1 diem = 8
2 if diem >= 5:
3     print("Ban da qua mon!")
```

```
4     print("Chuc mung ban!")
```

Trong ví dụ này, hai dòng `print()` đều thuộc khối của `if`. Điều đó có nghĩa là nếu điều kiện đúng thì cả hai dòng đều được thực hiện. Nếu điều kiện sai thì cả hai đều bị bỏ qua.

4.10.1 Ý nghĩa của thụt lề

Đối với nhiều ngôn ngữ lập trình, việc thụt lề (indentation) chủ yếu nhằm mục đích trình bày mã nguồn cho đẹp và dễ đọc; ngay cả khi người lập trình không thụt lề đúng quy ước, chương trình vẫn có thể hoạt động nếu các dấu ngoặc hoặc ký hiệu phân tách được viết đầy đủ. Tuy nhiên, Python lựa chọn một triết lý hoàn toàn khác. Trong Python, thụt lề không chỉ là vấn đề về hình thức mà còn là một phần của cú pháp ngôn ngữ. Những khoảng trắng ở đầu dòng mang ý nghĩa xác định phạm vi của một khối lệnh, từ đó quyết định chương trình sẽ thực thi những câu lệnh nào trong từng trường hợp cụ thể.

Khi viết một câu lệnh điều kiện bằng `if`, dòng chứa điều kiện luôn kết thúc bằng dấu hai chấm (`:`). Ngay sau đó, tất cả các câu lệnh thuộc nhánh xử lý phải được thụt vào cùng một mức. Python sẽ hiểu rằng những câu lệnh có cùng mức thụt lề này thuộc về cùng một khối lệnh và chỉ được thực hiện khi điều kiện của `if` là đúng. Khi mức thụt lề kết thúc, Python hiểu rằng khối lệnh cũng kết thúc và chương trình tiếp tục thực hiện các câu lệnh nằm bên ngoài điều kiện.

Điều này khiến khoảng trắng trong Python mang một ý nghĩa hoàn toàn khác so với nhiều ngôn ngữ lập trình khác. Chỉ cần thay đổi vị trí thụt lề của một dòng lệnh, ý nghĩa của cả chương trình có thể thay đổi theo. Nếu thụt lề sai, Python sẽ không cố gắng đoán ý định của người lập trình mà sẽ báo lỗi ngay lập tức. Mặc dù điều này đôi khi khiến người mới học cảm thấy bất tiện, nhưng chính sự nghiêm ngặt ấy giúp hạn chế rất nhiều lỗi khó phát hiện và buộc người lập trình phải viết mã nguồn một cách rõ ràng, nhất quán.

Ở góc độ rộng hơn, thụt lề còn phản ánh cách tổ chức tư duy của người lập trình. Một chương trình không phải là tập hợp ngẫu nhiên của các câu lệnh, mà được chia thành nhiều khối logic, mỗi khối đảm nhận một nhiệm vụ riêng. Những khoảng trắng ở đầu dòng giúp biểu diễn trực quan mối quan hệ giữa các khối đó: câu lệnh nào phụ thuộc vào điều kiện, câu lệnh nào được thực hiện trong một vòng lặp, câu lệnh nào nằm ngoài các cấu trúc điều khiển. Chỉ cần nhìn vào cách thụt lề, người đọc có thể nhanh chóng hình dung cấu trúc tổng thể của chương trình mà không cần đọc từng dòng lệnh một cách chi tiết.

Đối với người mới học lập trình, đây là một lợi thế rất lớn. Khi chương trình còn ngắn, việc theo dõi luồng thực hiện tương đối đơn giản. Tuy nhiên, khi chương trình có hàng chục hoặc hàng trăm dòng lệnh, việc nhận biết ranh giới giữa các khối xử lý trở nên quan trọng hơn bao giờ hết. Thụt lề đều đặn và nhất quán giúp người học dễ dàng nhận ra đâu là điều kiện cần kiểm tra, đâu là phần xử lý của từng nhánh, đâu là các câu lệnh sẽ luôn được thực hiện sau khi điều kiện kết thúc. Nhờ đó, việc đọc, sửa lỗi và mở rộng chương trình trở nên thuận tiện hơn rất nhiều.

Nếu nhìn sâu hơn nữa, có thể xem thụt lề như cách Python ”vẽ” cấu trúc tư duy của

người lập trình lên màn hình. Mỗi mức thụt lề tương ứng với một cấp độ của quá trình suy luận. Các câu lệnh nằm cùng một mức thể hiện rằng chúng có cùng vai trò trong thuật toán; những câu lệnh được thụt sâu hơn biểu thị rằng chúng chỉ được thực hiện trong một ngữ cảnh cụ thể. Nhờ biểu diễn trực tiếp cấu trúc logic bằng khoảng trắng, mã nguồn Python thường có tính trực quan rất cao. Người đọc không chỉ thấy chương trình đang làm gì mà còn hiểu được vì sao từng câu lệnh lại nằm ở vị trí đó.

Chính triết lý này là một trong những lý do khiến Python được sử dụng rộng rãi trong giáo dục và được xem là ngôn ngữ lập trình phù hợp cho người mới bắt đầu. Việc buộc người học viết mã nguồn theo một cấu trúc rõ ràng ngay từ đầu không chỉ giúp hạn chế lỗi cú pháp mà còn rèn luyện thói quen tổ chức chương trình một cách khoa học. Khi đã hình thành được thói quen này, người học sẽ dễ dàng tiếp cận các dự án lớn hơn, nơi cấu trúc và tính dễ đọc của mã nguồn có vai trò quan trọng không kém việc chương trình chạy đúng.

4.11 Câu lệnh `if-else`

Trong phần trước, chúng ta đã làm quen với câu lệnh `if`. Cấu trúc này cho phép chương trình thực hiện một khối lệnh khi điều kiện đúng. Tuy nhiên, nếu điều kiện không đúng thì chương trình chỉ đơn giản bỏ qua khối lệnh đó và tiếp tục thực hiện các câu lệnh phía sau. Điều này phù hợp với những tình huống mà ta chỉ quan tâm đến một trường hợp, nhưng trong thực tế, rất nhiều bài toán yêu cầu chương trình phải biết xử lý cả hai khả năng: điều kiện đúng và điều kiện sai.

Để giải quyết vấn đề đó, Python cung cấp câu lệnh `if-else`. Khác với `if`, cấu trúc này luôn có hai nhánh xử lý. Khi điều kiện đúng, chương trình thực hiện khối lệnh nằm dưới `if`; ngược lại, nếu điều kiện sai, chương trình sẽ bỏ qua khối lệnh đó và thực hiện khối lệnh nằm dưới `else`. Nhờ vậy, chương trình luôn có một hành động tương ứng với mọi kết quả của phép kiểm tra.

Về bản chất, `if-else` phản ánh một trong những kiểu tư duy đơn giản nhưng quan trọng nhất trong lập trình: tư duy hai nhánh. Mỗi quyết định đều dẫn đến đúng một trong hai kết quả. Trong Toán học, một mệnh đề hoặc đúng hoặc sai. Trong logic, một điều kiện hoặc được thỏa mãn hoặc không được thỏa mãn. Trong cuộc sống, một học sinh hoặc đủ điểm qua môn hoặc chưa đủ điểm; một tài khoản hoặc đăng nhập thành công hoặc bị từ chối; một giao dịch hoặc hợp lệ hoặc không hợp lệ. Chính vì thế, `if-else` là công cụ tự nhiên để mô tả những quyết định có hai khả năng loại trừ lẫn nhau.

Phân loại số chẵn lẻ

```python
so = int(input("Nhap mot so nguyen: "))
if so % 2 == 0:
    print(f"{so} la so chan")
else:
    print(f"{so} la so le")
```

Trong ví dụ trên, chương trình yêu cầu người dùng nhập một số nguyên, sau đó sử dụng

toán tử chia lấy dư (`%`) để kiểm tra xem số đó có chia hết cho 2 hay không. Nếu phần dư bằng 0, điều kiện `so % 2 == 0` là đúng và chương trình sẽ thực hiện khối lệnh trong `if`, kết luận đây là số chẵn. Ngược lại, nếu điều kiện sai thì số đó chắc chắn là số lẻ và chương trình thực hiện khối lệnh trong `else`.

Hai khối lệnh này bao quát toàn bộ các khả năng có thể xảy ra, vì một số nguyên không thể vừa là số chẵn vừa là số lẻ, đồng thời cũng không thể nằm ngoài hai trường hợp này.

Đây là một ví dụ rất điển hình về cách sử dụng `if-else`. Chương trình không chỉ dừng lại ở việc kiểm tra một điều kiện mà còn phải đưa ra hành động phù hợp trong cả hai tình huống đối lập. Khi mới học lập trình, nhiều người thường chỉ nghĩ đến việc kiểm tra điều kiện đúng, nhưng trong thực tế, việc xác định chương trình phải làm gì khi điều kiện sai cũng quan trọng không kém. Một chương trình chỉ xử lý trường hợp đúng mà bỏ qua trường hợp sai thường sẽ thiếu hoàn chỉnh và có thể gây ra những kết quả không mong muốn.

Nếu quan sát kỹ hơn, ta sẽ thấy rất nhiều bài toán cơ bản đều có thể được mô hình hóa bằng tư duy hai nhánh. Chẳng hạn, nếu điểm trung bình lớn hơn hoặc bằng 5 thì học sinh được lên lớp, ngược lại phải học lại. Nếu mật khẩu nhập vào trùng với mật khẩu đã lưu thì cho phép đăng nhập, ngược lại hiển thị thông báo sai mật khẩu. Nếu số dư tài khoản đủ để thanh toán thì giao dịch được thực hiện, ngược lại giao dịch bị từ chối. Nếu nhiệt độ vượt quá ngưỡng an toàn thì kích hoạt hệ thống cảnh báo, nếu không thì tiếp tục hoạt động bình thường. Tất cả những tình huống này đều có cùng một cấu trúc logic: kiểm tra một điều kiện và lựa chọn một trong hai hướng xử lý.

Không chỉ xuất hiện trong các ví dụ đơn giản, `if-else` còn có vai trò rất quan trọng trong các chương trình thực tế. Dữ liệu do người dùng nhập vào luôn tiềm ẩn khả năng sai sót hoặc không hợp lệ. Người dùng có thể nhập tuổi âm, nhập điểm vượt quá giới hạn cho phép, nhập mật khẩu sai hoặc để trống thông tin bắt buộc. Nếu chương trình không kiểm tra những trường hợp này, các phép tính phía sau có thể cho kết quả sai hoặc thậm chí làm chương trình bị lỗi. Vì vậy, một trong những nhiệm vụ đầu tiên của lập trình viên là kiểm tra tính hợp lệ của dữ liệu đầu vào và xử lý thích hợp cho cả hai trường hợp: dữ liệu hợp lệ và dữ liệu không hợp lệ. Đây chính là một trong những ứng dụng phổ biến nhất của `if-else` trong phát triển phần mềm.

Để hình dung trực quan hơn, hãy tưởng tượng bạn đang đứng trước cổng vào của một rạp chiếu phim hoặc một khu vui chơi. Nhân viên kiểm soát sẽ kiểm tra vé của từng khách. Nếu vé hợp lệ, cổng sẽ được mở và khách được phép đi vào. Ngược lại, nếu vé không hợp lệ hoặc đã hết hạn, khách sẽ không được vào và có thể nhận được thông báo tương ứng. Quy trình này chỉ có hai hướng xử lý và mỗi người đều phải đi qua đúng một trong hai nhánh. Câu lệnh `if-else` trong lập trình hoạt động theo nguyên lý hoàn toàn tương tự: chương trình đánh giá điều kiện, sau đó lựa chọn duy nhất một nhánh để thực hiện.

4.12 Câu lệnh `if-elif-else`

Trong nhiều bài toán thực tế, một điều kiện không chỉ dẫn đến hai khả năng đơn giản là *đúng* hoặc *sai*, mà còn có thể dẫn đến nhiều mức xử lý khác nhau. Khi đó, câu lệnh `if` đơn lẻ hoặc cặp `if-else` sẽ trở nên chưa đủ linh hoạt. Python cung cấp cấu trúc `if-elif-else`

để giải quyết những tình huống có nhiều nhánh lựa chọn. Đây là cách giúp chương trình không chỉ biết phân biệt hai trường hợp đối lập, mà còn có thể chia một bài toán thành nhiều mức nhỏ hơn, tương ứng với nhiều kết quả khác nhau.

Từ khóa `elif` là viết tắt của cụm 'else if", có thể hiểu là 'nếu điều kiện trước đó không đúng thì xét tiếp điều kiện này". Nói cách khác, chương trình sẽ kiểm tra các điều kiện theo thứ tự từ trên xuống dưới. Mỗi khi gặp một điều kiện đúng, chương trình sẽ thực hiện ngay nhánh lệnh tương ứng và bỏ qua toàn bộ các nhánh phía sau. Chính cơ chế này làm cho `if-elif-else` trở thành một cấu trúc rất phù hợp với các bài toán phân loại, xếp hạng, hoặc chia dữ liệu thành nhiều khoảng giá trị khác nhau.

Trong toán học phổ thông, kiểu tư duy nhiều nhánh xuất hiện rất thường xuyên. Chẳng hạn, khi xếp loại học lực, người ta thường chia điểm số thành nhiều mức: xuất sắc, giỏi, khá, trung bình, yếu. Mỗi mức tương ứng với một khoảng điểm riêng. Khi giải toán, ta cũng thường phải xét nhiều trường hợp theo khoảng giá trị của một biến số, chẳng hạn xét dấu của biểu thức, xét các khoảng đồng biến nghịch biến của hàm số, hay phân loại nghiệm theo giá trị của tham số. Trong những tình huống như vậy, điều quan trọng không chỉ là kiểm tra một điều kiện duy nhất, mà là xác định xem dữ liệu đang thuộc về nhánh nào trong nhiều nhánh có thể xảy ra.

Xếp loại học lực

```
diem = float(input("Nhap diem: "))

if diem >= 9:
    print("Xuat sac")
elif diem >= 8:
    print("Gioi")
elif diem >= 6.5:
    print("Kha")
elif diem >= 5:
    print("Trung binh")
else:
    print("Yeu")
```

Ở ví dụ trên, chương trình yêu cầu người dùng nhập vào một điểm số rồi so sánh giá trị đó với nhiều ngưỡng khác nhau. Nếu điểm lớn hơn hoặc bằng 9, chương trình kết luận là `Xuat sac`. Nếu không đạt mức đó, nó tiếp tục kiểm tra xem điểm có lớn hơn hoặc bằng 8 hay không. Quá trình này được lặp lại cho đến khi tìm thấy nhánh phù hợp nhất. Nếu không có điều kiện nào phía trên đúng, chương trình sẽ rơi xuống nhánh `else` và in ra `Yeu`. Như vậy, toàn bộ các khả năng đã được bao phủ một cách đầy đủ và có trật tự.

Điểm đặc biệt cần chú ý là Python không kiểm tra tất cả các điều kiện cùng một lúc theo kiểu độc lập, mà kiểm tra tuần tự từng nhánh. Ngay khi một điều kiện đúng, chương trình sẽ dừng việc xét các nhánh còn lại. Điều này có nghĩa là thứ tự viết các điều kiện là vô cùng quan trọng. Trong ví dụ xếp loại học lực, điều kiện `diem >= 9` phải được đặt trước `diem >=`

`8`, rồi mới đến `diem >= 6.5` và `diem >= 5`. Nếu đảo ngược thứ tự, chương trình sẽ không còn phân loại đúng nữa.

4.12.1 Vì sao thứ tự các điều kiện lại quan trọng?

Giả sử bạn viết `elif diem >= 5` trước, rồi mới đến `elif diem >= 9`. Khi đó, một học sinh đạt điểm 9 vẫn thỏa mãn điều kiện `diem >= 5`. Vì chương trình luôn dừng lại ở nhánh đúng đầu tiên, nó sẽ kết luận ngay rằng học sinh đó chỉ đạt mức `Trung binh`, và nhánh `Xuat sac` phía sau sẽ không bao giờ được chạy. Điều này cho thấy trong cấu trúc nhiều nhánh, không chỉ bản thân điều kiện quan trọng mà thứ tự giữa các điều kiện cũng ảnh hưởng trực tiếp đến kết quả.

Tư duy này rất giống với cách làm toán khi phân chia một tập hợp thành các khoảng hoặc các trường hợp riêng biệt. Nếu các khoảng bị chồng lấn lên nhau, một phần tử có thể bị đếm hai lần. Ngược lại, nếu bạn chia không đủ chặt, sẽ có những phần tử bị bỏ sót. Trong lập trình cũng vậy, một chuỗi điều kiện được sắp xếp sai có thể khiến chương trình chọn nhầm nhánh, hoặc bỏ qua nhánh cần thiết. Vì thế, trước khi viết code, người lập trình cần xác định rõ: các điều kiện có loại trừ nhau hay không, điều kiện nào nên kiểm tra trước, và điều kiện nào chỉ nên xét khi những điều kiện trước đó không đúng.

4.12.2 Ý nghĩa của `elif`

So với việc viết nhiều câu lệnh `if` rời rạc, `if-elif-else` thể hiện đúng hơn bản chất của một bài toán phân loại. Khi dùng nhiều `if` độc lập, chương trình sẽ kiểm tra từng điều kiện như những mệnh đề riêng biệt, và có thể thực hiện nhiều nhánh cùng lúc nếu nhiều điều kiện đều đúng. Điều đó phù hợp với một số bài toán đặc thù, nhưng không thích hợp khi ta chỉ muốn chọn đúng một kết quả trong nhiều khả năng loại trừ lẫn nhau. Ngược lại, `if-elif-else` đảm bảo rằng tại mỗi thời điểm, chương trình chỉ đi theo một con đường duy nhất.

Chính vì vậy, `elif` giúp mã nguồn trở nên ngắn gọn, sáng sủa và phản ánh đúng cấu trúc logic của bài toán. Thay vì viết nhiều khối `if` riêng lẻ và phải tự kiểm soát sự chồng chéo giữa chúng, người lập trình chỉ cần mô tả các mức điều kiện theo đúng thứ tự từ trên xuống dưới. Cách viết này đặc biệt hữu ích trong các bài toán xếp loại, tính mức thuế, phân hạng sản phẩm, kiểm tra cấp độ, hoặc xử lý đầu vào theo nhiều ngưỡng khác nhau.

4.12.3 Nhận diện khi nào cần nhiều nhánh

Ngay khi một bài toán có dấu hiệu *phân loại*, *xếp hạng*, *chia khoảng* hoặc *chọn một trong nhiều mức xử lý*, bạn nên nghĩ đến cấu trúc `if-elif-else`. Những từ khóa như *nếu lớn hơn*, *nếu nằm trong khoảng*, *nếu thuộc loại A, B, C*, *nếu đạt mức này thì*, *nếu không thì*, thường là tín hiệu cho thấy bài toán không còn là bài toán hai nhánh đơn giản nữa. Khi đó, thay vì cố gắng ép bài toán vào một cấu trúc quá đơn giản, ta nên dùng nhiều nhánh điều kiện để mô tả chính xác hơn thực tế của vấn đề.

Đây cũng là một bước phát triển quan trọng trong tư duy lập trình. Nếu `if` giúp chương trình biết chọn giữa hai khả năng, thì `if-elif-else` giúp chương trình biết cách suy nghĩ

theo nhiều mức phân loại. Từ đây, người học bắt đầu làm quen với việc thiết kế thuật toán không chỉ dựa trên đúng/sai, mà còn dựa trên thứ tự ưu tiên của các điều kiện, trên miền giá trị của dữ liệu, và trên việc tổ chức logic của bài toán thành nhiều tầng xử lý khác nhau. Đó là một bước chuyển rất quan trọng để đi từ tư duy lập trình cơ bản sang tư duy giải quyết vấn đề một cách hệ thống.

4.13 Kết hợp điều kiện với toán tử logic

Ở chương trước, chúng ta đã làm quen với các toán tử so sánh (`==`, `!=`, `>`, `<`, `>=`, `<=`) và các toán tử logic (`and`, `or`, `not`). Khi đó, chúng được giới thiệu như những công cụ để tạo ra giá trị logic `True` hoặc `False`. Đến chương này, những kiến thức đó mới thực sự phát huy vai trò của mình. Mỗi điều kiện trong câu lệnh `if` đều được xây dựng từ các phép so sánh và các phép logic. Có thể nói rằng, toán tử so sánh cung cấp những "viên gạch" cơ bản, còn toán tử logic là chất kết dính giúp ghép nhiều điều kiện đơn giản thành một điều kiện phức tạp hơn.

Trong thực tế, rất hiếm khi một quyết định chỉ phụ thuộc vào một điều kiện duy nhất. Để đánh giá một học sinh có đủ điều kiện nhận học bổng hay không, nhà trường không chỉ xét điểm trung bình mà còn xem xét điểm rèn luyện, số buổi nghỉ học hoặc kết quả của các môn học bắt buộc. Khi đăng nhập vào một hệ thống, phần mềm không chỉ kiểm tra tên đăng nhập mà còn phải xác minh mật khẩu, đôi khi còn kiểm tra thêm mã xác thực hai lớp. Trong thương mại điện tử, một đơn hàng chỉ được xử lý khi sản phẩm còn hàng, người mua thanh toán thành công và địa chỉ giao hàng hợp lệ. Những ví dụ này cho thấy một quyết định trong thực tế thường phụ thuộc vào nhiều điều kiện đồng thời.

Ví dụ:

- Một học sinh được công nhận đạt yêu cầu khi điểm trung bình lớn hơn hoặc bằng 5 **và** có ý thức chuyên cần tốt.
- Một số được coi là hợp lệ khi nó **đồng thời** lớn hơn 0 và nhỏ hơn 100.
- Một tài khoản chỉ đăng nhập thành công khi tên đăng nhập đúng **và** mật khẩu chính xác.
- Một cảnh báo sẽ được kích hoạt khi nhiệt độ quá cao **hoặc** áp suất vượt quá giới hạn cho phép.

Những yêu cầu như vậy không thể được mô tả chỉ bằng một phép so sánh đơn lẻ. Ta cần kết hợp nhiều điều kiện lại với nhau bằng các toán tử logic.

Kết hợp điều kiện

```
diem = 8
chuyen_can = True

if diem >= 5 and chuyen_can:
```

```
    print("Ban dat yeu cau")
else:
    print("Ban chua dat yeu cau")
```

Trong ví dụ trên, điều kiện của câu lệnh `if` không còn là một phép so sánh đơn giản mà là sự kết hợp của hai điều kiện nhỏ. Điều kiện thứ nhất kiểm tra xem điểm có đạt yêu cầu hay không. Điều kiện thứ hai kiểm tra trạng thái chuyên cần của học sinh. Chỉ khi cả hai điều kiện đều đúng, toàn bộ biểu thức logic mới nhận giá trị `True` và chương trình mới thông báo học sinh đạt yêu cầu. Nếu chỉ một trong hai điều kiện không được thỏa mãn, kết quả của toàn bộ biểu thức sẽ là `False` và chương trình chuyển sang nhánh `else`.

4.13.1 Ba phép logic cơ bản

Trong Python, ba toán tử logic được sử dụng thường xuyên nhất là `and`, `or` và `not`. Mặc dù chỉ gồm ba từ khóa rất ngắn gọn, chúng có khả năng biểu diễn phần lớn các điều kiện gặp trong lập trình.

Toán tử `and` được sử dụng khi tất cả các điều kiện đều phải đúng. Đây là phép kết hợp mang ý nghĩa "đồng thời". Chỉ cần một điều kiện sai thì toàn bộ biểu thức sẽ sai.

Toán tử `or` được sử dụng khi chỉ cần một trong các điều kiện đúng. Nó thường xuất hiện trong các bài toán có nhiều khả năng dẫn đến cùng một kết quả. Ví dụ, một người được giảm giá nếu là học sinh **hoặc** là sinh viên.

Toán tử `not` có nhiệm vụ đảo ngược giá trị logic của một biểu thức. Nếu điều kiện ban đầu là đúng thì sau khi áp dụng `not`, kết quả trở thành sai; ngược lại, nếu điều kiện ban đầu sai thì kết quả trở thành đúng. Toán tử này thường được dùng khi ta muốn kiểm tra điều ngược lại của một điều kiện đã có.

Có thể xem ba toán tử này giống như những từ nối trong ngôn ngữ tự nhiên. Chúng giúp người lập trình diễn đạt những yêu cầu phức tạp bằng những biểu thức logic rõ ràng và chính xác. Khi bài toán ngày càng lớn, khả năng kết hợp các điều kiện một cách hợp lý sẽ trở thành một kỹ năng rất quan trọng.

4.14 Ứng dụng của câu lệnh điều kiện

Sau khi học các cấu trúc `if`, `if-else` và `if-elif-else`, có thể bạn sẽ nhận ra rằng hầu hết các ví dụ đều khá đơn giản. Tuy nhiên, trong các phần mềm thực tế, câu lệnh điều kiện xuất hiện gần như ở mọi nơi. Bất kỳ khi nào máy tính phải đưa ra một quyết định dựa trên dữ liệu đầu vào, chương trình đều cần sử dụng điều kiện.

Một trong những ứng dụng phổ biến nhất là kiểm tra dữ liệu do người dùng nhập vào. Chương trình phải xác định xem dữ liệu có hợp lệ hay không trước khi tiếp tục xử lý. Nếu người dùng nhập tuổi âm, nhập điểm vượt quá giới hạn cho phép hoặc bỏ trống thông tin bắt buộc, chương trình cần phát hiện và thông báo lỗi thay vì tiếp tục thực hiện các phép tính sai.

Điều kiện cũng được sử dụng để phân loại và xếp hạng dữ liệu. Các hệ thống giáo dục phân loại học sinh theo điểm số, các cửa hàng trực tuyến phân loại khách hàng theo hạng thành viên, các ngân hàng đánh giá mức độ tín nhiệm của khách hàng, còn các trò chơi điện tử phân chia cấp độ của người chơi dựa trên số điểm hoặc kinh nghiệm tích lũy. Tất cả những hệ thống này đều hoạt động dựa trên việc kiểm tra nhiều điều kiện khác nhau.

Trong các thuật toán, điều kiện còn quyết định hướng đi của chương trình. Một thuật toán tìm kiếm cần kiểm tra xem phần tử cần tìm đã xuất hiện hay chưa. Thuật toán sắp xếp liên tục so sánh hai phần tử để quyết định có cần đổi chỗ hay không. Các thuật toán xử lý văn bản phải xác định xem một ký tự là chữ cái, chữ số hay ký tự đặc biệt trước khi xử lý tiếp. Có thể nói rằng, điều kiện chính là cơ chế giúp thuật toán biết phải lựa chọn hành động nào trong từng tình huống.

Nếu không có câu lệnh điều kiện, một chương trình chỉ là một chuỗi các lệnh được thực hiện tuần tự từ đầu đến cuối. Nó không thể phản ứng trước dữ liệu đầu vào, không thể xử lý lỗi, không thể tương tác với người dùng và cũng không thể thích nghi với những tình huống khác nhau. Chính vì vậy, điều kiện thường được xem là ”trái tim” của logic chương trình. Chúng giúp phần mềm trở nên linh hoạt, thông minh và phù hợp với những yêu cầu đa dạng của thực tế.

Trong thế giới phần mềm hiện đại, rất khó tìm thấy một ứng dụng không sử dụng câu lệnh điều kiện. Mỗi nút đăng nhập, mỗi trang thanh toán, mỗi bộ lọc dữ liệu, mỗi hệ thống gợi ý, mỗi trò chơi điện tử hay mỗi ứng dụng trí tuệ nhân tạo đều liên tục kiểm tra các điều kiện để quyết định bước xử lý tiếp theo. Dù người dùng không nhìn thấy, hàng triệu phép kiểm tra điều kiện vẫn đang diễn ra mỗi giây bên trong các chương trình mà chúng ta sử dụng hằng ngày.

4.15 Bài tập luyện tập

Sau khi hoàn thành chương này, bạn nên dành thời gian tự xây dựng chương trình thay vì chỉ đọc các ví dụ. Lập trình là một kỹ năng thực hành, và khả năng sử dụng câu lệnh điều kiện chỉ được hình thành khi bạn tự suy nghĩ, tự viết chương trình và tự kiểm tra kết quả của mình. Các bài tập dưới đây được sắp xếp từ cơ bản đến nâng cao, giúp bạn từng bước rèn luyện tư duy phân tích điều kiện và xây dựng các nhánh xử lý phù hợp.

1. Nhập vào ba số thực `a`, `b`, `c`. Hãy xác định và in ra số lớn nhất mà không sử dụng hàm `max()`.

2. Viết chương trình kiểm tra một số nguyên là số chẵn hay số lẻ.

3. Nhập điểm trung bình của một học sinh và xếp loại theo các mức: xuất sắc, giỏi, khá, trung bình và yếu.

4. Nhập một số nguyên và kiểm tra xem số đó có thuộc đoạn từ 1 đến 100 hay không.

5. Viết chương trình kiểm tra mật khẩu. Nếu người dùng nhập đúng chuỗi `python123` thì thông báo đăng nhập thành công, ngược lại thông báo đăng nhập thất bại.

6. Tính tiền điện theo quy tắc: 50 kWh đầu tiên có giá 1000 đồng/kWh, phần điện vượt

quá 50 kWh được tính với giá 1500 đồng/kWh.

7. Nhập một số bất kỳ và xác định xem số đó là số dương, số âm hay bằng 0.

4.16 Tóm tắt cuối chương

Qua chương này, bạn đã làm quen với một trong những cấu trúc điều khiển quan trọng nhất của lập trình: câu lệnh điều kiện. Nếu chương trước giúp bạn hiểu cách tạo ra các biểu thức logic, thì chương này đã cho thấy cách sử dụng những biểu thức đó để giúp chương trình đưa ra quyết định.

Bạn cần ghi nhớ những nội dung cốt lõi sau:

- Câu lệnh điều kiện cho phép chương trình lựa chọn hành động dựa trên kết quả của một biểu thức logic.
- `if` được sử dụng khi chỉ cần xử lý trường hợp điều kiện đúng.
- `if-else` phù hợp với các bài toán có hai khả năng loại trừ nhau.
- `if-elif-else` được sử dụng khi bài toán có nhiều mức phân loại hoặc nhiều nhánh xử lý.
- Python sử dụng thụt lề để xác định phạm vi của từng khối lệnh, vì vậy việc thụt lề đúng là yêu cầu bắt buộc.
- Các toán tử so sánh và toán tử logic là nền tảng để xây dựng những điều kiện đơn giản cũng như phức tạp.
- Khi sử dụng nhiều nhánh `elif`, cần đặc biệt chú ý đến thứ tự của các điều kiện để chương trình luôn đưa ra kết quả chính xác.
- Quan trọng hơn cả, hãy luôn bắt đầu bằng việc phân tích bài toán và xác định các trường hợp cần xét trước khi viết mã nguồn. Khi tư duy logic đã rõ ràng, việc chuyển thành chương trình Python sẽ trở nên đơn giản hơn rất nhiều.

Chương tiếp theo, chúng ta sẽ học về ”vòng lặp”. Nếu câu lệnh điều kiện giúp chương trình biết ”lựa chọn” giữa nhiều hướng xử lý, thì vòng lặp sẽ giúp chương trình ”lặp lại” một công việc nhiều lần một cách tự động. Hai cấu trúc này kết hợp với nhau tạo thành nền tảng của hầu hết các thuật toán trong lập trình.

Góc nhìn cá nhân

Nếu nhìn từ góc độ người học, chương này là bước đầu tiên để chương trình bắt đầu giống như một hệ thống có khả năng “suy nghĩ đơn giản”. Nó không suy nghĩ như con người, nhưng nó có thể kiểm tra dữ kiện, so sánh, lựa chọn và thực hiện nhánh lệnh phù hợp. Đó là sự thay đổi rất lớn so với những chương trước, nơi chương trình chỉ làm đúng một chuỗi lệnh cố định.

Với người học Toán, phần này rất gần gũi vì bản chất của nó là phân tích trường hợp.

Bạn đã quen với việc xét điều kiện, chia miền, kiểm tra dấu và chọn nhánh xử lý từ lâu. Lập trình chỉ biến những thao tác đó thành một ngôn ngữ mà máy tính có thể hiểu. Khi bạn hiểu điều này, câu lệnh điều kiện sẽ không còn là một cú pháp mới, mà là một cách diễn đạt khác của tư duy toán học quen thuộc.

Chương 5

Lặp lại công việc - Vòng lặp

Giới thiệu ngắn

Trong toán học và trong cuộc sống, có rất nhiều công việc cần được lặp lại nhiều lần. Một giáo viên chấm 45 bài kiểm tra, một nhà toán học thử nhiều giá trị khác nhau để tìm quy luật, một học sinh làm cùng một dạng bài nhiều lần để luyện phản xạ, hay một chương trình cần in ra hàng trăm con số liên tiếp. Nếu phải viết thủ công từng câu lệnh cho mỗi lần lặp, chương trình sẽ dài, rườm rà và dễ sai.

Vòng lặp sinh ra để giải quyết đúng vấn đề đó. Nó cho phép máy tính thực hiện một công việc lặp đi lặp lại một cách tự động, đúng số lần hoặc đúng cho đến khi điều kiện dừng được thỏa mãn. Đây là một trong những cấu trúc quan trọng nhất trong lập trình, vì rất nhiều thuật toán thực tế đều dựa trên ý tưởng lặp.

Ở trường phổ thông, bạn đã gặp các dãy số, quy luật lặp, bảng cửu chương, cấp số cộng, cấp số nhân, các bước biến đổi lặp lại trong đại số, hoặc những thao tác kiểm tra nhiều giá trị liên tiếp. Vòng lặp trong Python chính là cách biến những tư duy đó thành hành động cụ thể của chương trình. Từ đây, bạn sẽ thấy rằng máy tính đặc biệt mạnh ở chỗ nó không biết mệt khi làm đi làm lại một công việc hàng ngàn lần.

Mục tiêu học tập

Sau khi học xong chương này, bạn cần đạt được những mục tiêu sau:

- Hiểu được khái niệm vòng lặp và giải thích được vì sao vòng lặp là một trong ba cấu trúc điều khiển cơ bản của lập trình.
- Phân biệt được hai loại vòng lặp chính trong Python là `for` và `while`, đồng thời biết lựa chọn cấu trúc phù hợp với từng bài toán.
- Sử dụng thành thạo vòng lặp `for` kết hợp với hàm `range()` để thực hiện các công việc có số lần lặp xác định hoặc để duyệt qua một dãy giá trị.
- Sử dụng được vòng lặp `while` để giải quyết các bài toán mà số lần lặp chưa biết trước và phụ thuộc vào một điều kiện thay đổi trong quá trình thực hiện.
- Hiểu vai trò của các câu lệnh `break` và `continue` trong việc điều khiển luồng thực thi

của vòng lặp.

- Vận dụng vòng lặp để giải quyết các bài toán quen thuộc như tính tổng, tính tích, lập bảng giá trị, kiểm tra điều kiện, tìm kiếm và xử lý dữ liệu.
- Hình thành tư duy mô tả một quá trình lặp bằng thuật toán, biết nhận diện quy luật lặp và chuyển quy luật đó thành chương trình Python.

5.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Khi gửi tiết kiệm đều mỗi tháng, số dư mới phụ thuộc số dư cũ, khoản gửi thêm và lãi của kỳ hiện tại. Viết mười hai phép tính riêng lẻ sẽ che mất quy luật. Vòng lặp cho phép mô tả quy luật một lần, nhưng ta phải biết trạng thái nào thay đổi, đại lượng nào tích lũy và điều kiện nào bảo đảm quá trình kết thúc.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **vòng lặp như dãy truy hồi và quá trình tiến dần đến điều kiện dừng**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

5.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: chỉ số bước k, số dư B_k, lãi suất r, khoản gửi p, tổng số bước n và điều kiện dừng.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

5.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$B_{k+1} = B_k(1 + r) + p, \qquad k = 0, 1, \ldots, n - 1$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: biến đếm, giá trị tích lũy, phần dữ liệu đã xử lý và phần còn lại?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

5.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Sau (k) vòng, trạng thái phải đúng bằng kết quả của (k) bước đầu trong mô hình; mỗi vòng phải làm đại lượng tiến gần điều kiện dừng.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

5.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **vòng lặp như dãy truy hồi và quá trình tiến dần đến điều kiện dừng** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

5.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Tính số dư sau 12 tháng và tìm tháng đầu tiên số dư vượt một mốc. Bài toán thứ nhất có số bước biết trước nên phù hợp với `for`; bài toán thứ hai dừng theo trạng thái nên phù hợp với `while`. Cả hai phải được truy vết ít nhất ba vòng bằng bảng.

5.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

5.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

5.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

5.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

5.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

5.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?

5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

5.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Cập nhật biến đếm nhưng quên cập nhật đại lượng quyết định điều kiện dừng.
- Cộng phần tử hiện tại hai lần do đặt phép cập nhật ở sai vị trí.
- Dùng `break` để che một điều kiện lặp chưa được thiết kế rõ.
- Lặp trên dữ liệu đang bị thay đổi cấu trúc mà không xét ảnh hưởng đến phần chưa duyệt.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

5.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

5.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Lập bảng truy vết tổng $1 + 2 + \cdots + n$ và phát biểu bất biến sau mỗi vòng.
2. Mô phỏng lãi kép rồi so sánh kết quả vòng lặp với công thức đóng trong trường hợp không gửi thêm.
3. Thiết kế vòng lặp Euclid tìm ước chung lớn nhất và giải thích vì sao nó dừng.
4. Tạo bộ kiểm thử cho một vòng `while` nhập đến khi dữ liệu hợp lệ.
5. So sánh chi phí của lặp (n) lần và hai vòng lặp lồng nhau bằng số phép toán.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và

ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

5.8 Ý nghĩa của vòng lặp

Trong chương trước, chúng ta đã học cách giúp chương trình đưa ra quyết định thông qua các câu lệnh điều kiện. Tuy nhiên, trong thực tế, rất nhiều bài toán không chỉ yêu cầu chương trình lựa chọn giữa các nhánh xử lý mà còn đòi hỏi một hành động phải được thực hiện nhiều lần. Nếu mỗi lần lặp lại đều phải viết một câu lệnh riêng, chương trình sẽ nhanh chóng trở nên dài dòng, khó đọc và rất khó bảo trì. Chính vì vậy, lập trình cần đến một cơ chế cho phép mô tả một hành động chỉ một lần nhưng có thể thực hiện lặp đi lặp lại nhiều lần theo một quy luật xác định. Cơ chế đó chính là vòng lặp.

Có thể nói rằng vòng lặp là một trong những ý tưởng quan trọng nhất của lập trình. Thay vì yêu cầu máy tính thực hiện từng bước riêng lẻ, người lập trình chỉ cần mô tả quy tắc lặp, còn máy tính sẽ kiên trì thực hiện quy tắc đó cho đến khi hoàn thành nhiệm vụ. Đây cũng là một trong những điểm khác biệt lớn nhất giữa con người và máy tính. Con người rất giỏi trong việc phát hiện quy luật nhưng thường mất kiên nhẫn khi phải lặp lại cùng một thao tác hàng trăm hoặc hàng nghìn lần. Ngược lại, máy tính không biết tự tìm ra quy luật, nhưng một khi quy luật đã được mô tả rõ ràng, nó có thể thực hiện chính xác với tốc độ rất cao mà không hề mệt mỏi.

Trong Toán học ở bậc trung học phổ thông, tư duy lặp xuất hiện ở rất nhiều nội dung khác nhau, mặc dù chúng ta không gọi tên nó là "vòng lặp". Khi tính tổng của nhiều số hạng trong một dãy số, ta liên tục thực hiện phép cộng theo cùng một quy tắc. Khi lập bảng giá trị của một hàm số, ta lần lượt thay nhiều giá trị của biến độc lập vào công thức để tính giá trị tương ứng của hàm số. Khi khảo sát một quy luật hoặc kiểm tra một giả thuyết, ta thường thử nhiều trường hợp khác nhau trước khi rút ra kết luận. Trong nhiều bài toán đại số, ta còn phải lặp đi lặp lại cùng một phép biến đổi cho đến khi biểu thức đạt được dạng mong muốn. Mặc dù hình thức biểu hiện rất đa dạng, tất cả những hoạt động này đều có chung một đặc điểm: một thao tác được thực hiện nhiều lần theo cùng một quy luật.

- Tính tổng của nhiều số hạng trong một dãy số.
- Lập bảng giá trị của một hàm số trên một khoảng xác định.
- Thử nhiều giá trị khác nhau để tìm quy luật hoặc kiểm chứng một giả thuyết.
- Lặp lại cùng một phép biến đổi đại số cho đến khi thu được kết quả mong muốn.
- Kiểm tra lần lượt từng phần tử trong một tập dữ liệu hoặc một danh sách.

Nếu thực hiện những công việc trên bằng cách viết từng câu lệnh riêng lẻ, chương trình sẽ rất dài và gần như không thể mở rộng. Hãy tưởng tượng bạn muốn in các số từ 1 đến 1000. Nếu không có vòng lặp, bạn sẽ phải viết một nghìn câu lệnh `print()`, vừa mất thời gian vừa rất dễ mắc lỗi. Trong khi đó, với vòng lặp, bạn chỉ cần mô tả một lần rằng "hãy in giá trị hiện tại", còn việc lặp lại bao nhiêu lần sẽ do chương trình đảm nhiệm. Nhờ vậy,

cùng một đoạn mã ngắn có thể thay thế hàng nghìn câu lệnh giống hệt nhau.

Điều quan trọng hơn là vòng lặp không chỉ giúp giảm số lượng câu lệnh. Nó còn giúp người lập trình chuyển từ tư duy thực hiện từng thao tác sang tư duy mô tả một quy trình. Thay vì suy nghĩ 'làm bước thứ nhất, rồi bước thứ hai, rồi bước thứ ba", ta bắt đầu đặt câu hỏi: 'Có quy luật nào đang được lặp lại không?" Nếu tìm được quy luật đó, ta có thể viết chương trình ngắn gọn hơn, rõ ràng hơn và linh hoạt hơn rất nhiều. Đây chính là một trong những kỹ năng quan trọng nhất của tư duy thuật toán.

Một đặc điểm thú vị của vòng lặp là nó giúp chương trình dễ dàng mở rộng quy mô. Nếu hôm nay bài toán yêu cầu tính tổng 10 số thì ngày mai có thể yêu cầu tính tổng 10, 000 số. Nếu thuật toán được xây dựng bằng vòng lặp, bạn hầu như không cần thay đổi cấu trúc chương trình. Chỉ cần thay đổi dữ liệu đầu vào hoặc số lần lặp, chương trình vẫn hoạt động đúng. Chính khả năng tổng quát hóa này làm cho vòng lặp trở thành công cụ không thể thiếu trong hầu hết các chương trình máy tính.

5.8.1 Một hình dung đơn giản

Hãy tưởng tượng bạn đang làm một bài tập Toán yêu cầu tính giá trị của biểu thức

$$1^2, 2^2, 3^2, \ldots, 100^2.$$

Nếu làm thủ công, bạn sẽ phải lặp đi lặp lại cùng một thao tác: lấy một số, bình phương nó rồi ghi kết quả. Công việc này không khó, nhưng lại rất tốn thời gian vì phải thực hiện cùng một quy trình tới một trăm lần. Điều thay đổi duy nhất ở mỗi bước chỉ là giá trị của con số đang được xét, còn cách tính hoàn toàn giống nhau.

Máy tính cũng nhìn bài toán theo cách tương tự. Thay vì viết một trăm câu lệnh khác nhau, người lập trình chỉ cần mô tả quy trình một lần: lấy giá trị hiện tại, thực hiện phép tính, hiển thị kết quả, sau đó chuyển sang giá trị tiếp theo. Vòng lặp sẽ tự động lặp lại quy trình đó cho đến khi đã xử lý hết tất cả các giá trị cần thiết. Nhờ vậy, chương trình vừa ngắn gọn, vừa dễ đọc, lại có thể mở rộng cho những bài toán có quy mô lớn hơn rất nhiều.

Có thể xem vòng lặp như một chiếc máy tự động. Người lập trình không điều khiển từng thao tác của chiếc máy mà chỉ thiết lập quy luật hoạt động và điều kiện dừng. Sau đó, chiếc máy sẽ kiên trì thực hiện công việc theo đúng quy luật đã được mô tả. Đây cũng chính là bản chất của vòng lặp trong lập trình: mô tả một lần, thực hiện nhiều lần.

Trong các phần tiếp theo của chương, chúng ta sẽ tìm hiểu hai loại vòng lặp quan trọng nhất trong Python là `for` và `while`. Mặc dù đều có mục đích lặp lại công việc, mỗi loại vòng lặp lại phù hợp với một kiểu bài toán khác nhau. Việc hiểu rõ sự khác biệt giữa chúng sẽ giúp bạn lựa chọn công cụ thích hợp để xây dựng các thuật toán hiệu quả và dễ hiểu.

5.9 Vòng lặp `for` và hàm `range()`

Trong Python, có hai loại vòng lặp được sử dụng phổ biến nhất là `for` và `while`. Mặc dù đều có mục đích lặp lại một công việc nhiều lần, chúng được thiết kế để giải quyết những kiểu bài toán khác nhau. Trong phần này, chúng ta sẽ tìm hiểu vòng lặp `for` — cấu trúc

phù hợp khi biết trước số lần cần lặp hoặc khi cần xử lý lần lượt từng phần tử trong một dãy dữ liệu.

Có thể hiểu một cách đơn giản rằng, vòng lặp `for` hoạt động theo nguyên tắc "đi qua từng giá trị". Thay vì yêu cầu chương trình thực hiện một công việc đúng một lần, ta yêu cầu nó thực hiện cùng một công việc với từng giá trị trong một dãy. Mỗi lần lấy được một giá trị mới, chương trình sẽ thực hiện lại khối lệnh nằm bên trong vòng lặp. Khi đã đi hết toàn bộ các giá trị, vòng lặp tự động kết thúc và chương trình tiếp tục thực hiện các câu lệnh phía sau.

Đây là một kiểu tư duy rất quen thuộc trong Toán học. Khi lập bảng giá trị của một hàm số, chúng ta thường chọn một loạt giá trị của biến x, sau đó lần lượt tính giá trị tương ứng của y. Khi tính tổng các số từ 1 đến 100, ta cũng thực hiện cùng một phép cộng với từng số trong khoảng đó. Khi lập bảng cửu chương, ta lần lượt nhân một số cố định với các số từ 1 đến 10. Trong tất cả những ví dụ này, thao tác thực hiện luôn giống nhau, chỉ có dữ liệu đầu vào thay đổi sau mỗi bước. Vòng lặp `for` được xây dựng chính để mô tả loại công việc như vậy.

Điểm mạnh của `for` nằm ở khả năng diễn đạt rất tự nhiên. Người lập trình không cần quan tâm đến việc tăng biến đếm hay kiểm tra điều kiện dừng ở mỗi lần lặp. Thay vào đó, chỉ cần chỉ rõ dãy giá trị cần duyệt và công việc cần thực hiện với mỗi giá trị. Python sẽ tự động quản lý quá trình lặp từ đầu đến cuối.

In các số từ 1 đến 5

```
# range(1, 6) se chay tu 1 den 5 (khong bao gom 6)
for i in range(1, 6):
    print(i)
```

Trong ví dụ trên, từ khóa `for` mở đầu vòng lặp. Biến `i` được gọi là **biến lặp** (*loop variable*). Mỗi lần thực hiện vòng lặp, Python sẽ gán cho biến này một giá trị mới lấy từ dãy do hàm `range()` tạo ra. Lần lượt, `i` nhận các giá trị 1, 2, 3, 4 và 5. Mỗi khi nhận một giá trị, câu lệnh `print(i)` được thực hiện một lần. Sau khi giá trị cuối cùng đã được xử lý, vòng lặp tự động kết thúc.

Có thể mô tả quá trình hoạt động của đoạn chương trình trên như sau:

- Ban đầu, `i` nhận giá trị 1, chương trình in ra số 1.
- Tiếp theo, `i` nhận giá trị 2, chương trình in ra số 2.
- Quá trình này tiếp tục với các giá trị 3, 4 và 5.
- Khi không còn giá trị nào trong dãy, vòng lặp kết thúc.

Qua ví dụ đầu tiên này, chúng ta có thể rút ra bản chất của vòng lặp `for`: chương trình không tự tạo ra các giá trị để lặp, mà chỉ lần lượt lấy từng giá trị từ một đối tượng có thể duyệt được. Trong phần lớn các ví dụ cơ bản, đối tượng đó chính là dãy số được tạo bởi hàm `range()`.

5.9.1 Hàm `range()`

Trong hầu hết các chương trình sử dụng vòng lặp `for`, bạn sẽ bắt gặp hàm `range()`. Đây là một trong những hàm được sử dụng nhiều nhất trong Python, bởi nó có nhiệm vụ tạo ra một dãy số nguyên theo quy luật nhất định để vòng lặp có thể duyệt qua.

Có thể hình dung `range()` như một "máy tạo dãy số". Thay vì tự viết ra từng giá trị một cách thủ công, người lập trình chỉ cần mô tả quy luật của dãy, còn Python sẽ tự động tạo ra các giá trị tương ứng. Điều này giúp chương trình ngắn gọn hơn, đồng thời giảm đáng kể khả năng xảy ra sai sót.

Các dạng của hàm `range()`

- `range(n)` tạo ra các số nguyên từ `0` đến `n-1`.
- `range(a, b)` tạo ra các số nguyên từ `a` đến `b-1`.
- `range(a, b, c)` tạo ra các số nguyên từ `a` đến trước `b`, với mỗi lần tăng hoặc giảm `c` đơn vị.

Điểm khiến nhiều người mới học cảm thấy bối rối là `range()` luôn **không bao gồm giá trị cuối**. Chẳng hạn, `range(1, 6)` chỉ sinh ra các số 1, 2, 3, 4 và 5, chứ không sinh ra số 6.

Thoạt nhìn, quy ước này có vẻ bất tiện. Tuy nhiên, đây là một thiết kế rất hợp lý. Nhờ không bao gồm điểm cuối, việc chia các khoảng liên tiếp trở nên đơn giản và không bị chồng lấn. Ví dụ, hai khoảng `range(0, 10)` và `range(10, 20)` ghép lại sẽ tạo thành dãy từ 0 đến 19 mà không có giá trị nào bị lặp lại hoặc bị bỏ sót. Đây cũng là lý do mà nhiều ngôn ngữ lập trình hiện đại sử dụng cùng một quy ước.

5.9.2 Tư duy đằng sau `range()`

Nếu nhìn dưới góc độ Toán học, `range()` có thể được xem như một cách mô tả một dãy số hữu hạn. Mỗi phần tử của dãy đều được xác định theo một quy luật rõ ràng: có điểm bắt đầu, có điểm kết thúc và có bước nhảy giữa hai giá trị liên tiếp.

Trong chương trình, những dãy số như vậy xuất hiện rất thường xuyên. Chúng được sử dụng để đếm số lần lặp, đánh số thứ tự, duyệt qua các vị trí của dữ liệu hoặc tạo ra những quy luật có tính tuần hoàn. Chính vì vậy, `range()` gần như luôn đi cùng vòng lặp `for` trong các bài toán cơ bản.

Một ưu điểm quan trọng khác là `range()` không chỉ tạo ra các số liên tiếp mà còn cho phép thay đổi khoảng cách giữa hai giá trị kế tiếp thông qua tham số bước nhảy.

Sử dụng bước nhảy

```
1 for i in range(2, 11, 2):
2     print(i)
```

Trong ví dụ trên, vòng lặp không đi qua tất cả các số từ 2 đến 10 mà chỉ lấy các giá trị

cách nhau 2 đơn vị. Vì vậy, chương trình lần lượt in ra các số

$$2; 4; 6; 8; 10.$$

Có thể thấy rằng bước nhảy quyết định khoảng cách giữa hai lần lặp liên tiếp. Nếu bước nhảy bằng 1, vòng lặp đi qua mọi số nguyên trong khoảng. Nếu bước nhảy lớn hơn 1, một số giá trị sẽ được bỏ qua. Cơ chế này rất hữu ích trong nhiều bài toán như in các số chẵn, các số lẻ, các bội số của một số nguyên hoặc duyệt dữ liệu theo từng nhóm.

5.9.3 Ứng dụng của vòng lặp `for`

Nhờ khả năng duyệt qua từng giá trị một cách tự động, vòng lặp `for` được sử dụng trong rất nhiều bài toán khác nhau. Đặc biệt, những bài toán mà số lần thực hiện đã được xác định trước gần như luôn có thể giải quyết bằng `for`.

- Lặp một công việc với số lần xác định trước.
- Duyệt lần lượt các phần tử trong danh sách, chuỗi hoặc các cấu trúc dữ liệu khác.
- Lập bảng cửu chương hoặc bảng giá trị của một hàm số.
- Tính tổng, tích hoặc các đại lượng được xác định từ nhiều giá trị.
- In ra các mẫu hình hoặc các quy luật lặp trên màn hình.
- Thực hiện cùng một phép xử lý đối với nhiều dữ liệu đầu vào.

Ví dụ, để in bảng cửu chương của số 7, ta không cần viết mười phép nhân khác nhau. Chỉ cần sử dụng một vòng lặp `for` để lần lượt lấy các số từ 1 đến 10, sau đó thực hiện cùng một phép nhân với mỗi giá trị. Cách làm này phản ánh đúng bản chất của bài toán: một thao tác được lặp lại nhiều lần với dữ liệu thay đổi.

Không chỉ trong các ví dụ học tập, `for` còn xuất hiện ở hầu hết các chương trình thực tế. Khi một phần mềm cần hiển thị danh sách sản phẩm, xử lý từng dòng trong một tệp văn bản, gửi thông báo cho nhiều người dùng hoặc tính toán trên một tập dữ liệu lớn, vòng lặp `for` thường là công cụ được lựa chọn đầu tiên.

5.9.4 Cảm giác quen thuộc với người học Toán

Đối với người học Toán, vòng lặp `for` có lẽ là cấu trúc điều khiển dễ hình dung nhất. Mỗi lần lặp có thể xem như một bước trong quá trình lập bảng giá trị, một lần thử của một bài toán hoặc một số hạng trong một tổng. Người lập trình không cần nghĩ đến việc "lặp như thế nào", mà chỉ cần xác định hai yếu tố: dãy giá trị cần xét và công việc cần thực hiện với từng giá trị trong dãy đó.

Đây cũng chính là cách suy nghĩ của rất nhiều thuật toán. Thay vì giải quyết toàn bộ bài toán trong một bước duy nhất, ta chia bài toán thành nhiều bước nhỏ giống hệt nhau, mỗi bước xử lý một giá trị, một phần tử hoặc một trường hợp cụ thể. Vòng lặp `for` giúp máy tính thực hiện chính xác quá trình đó một cách tự động, nhanh chóng và nhất quán.

Trong phần tiếp theo, chúng ta sẽ tìm hiểu vòng lặp `while`. Nếu `for` phù hợp khi đã biết trước cần lặp bao nhiêu lần hoặc cần duyệt qua dãy giá trị nào, thì `while` sẽ giải quyết những bài toán mà số lần lặp không thể xác định ngay từ đầu và chỉ kết thúc khi một điều kiện nhất định được thỏa mãn.

5.10 Vòng lặp `while`

Trong phần trước, chúng ta đã tìm hiểu vòng lặp `for`, một công cụ rất phù hợp khi đã biết trước số lần cần lặp hoặc khi cần duyệt lần lượt qua một dãy giá trị. Tuy nhiên, không phải mọi bài toán đều cho chúng ta biết trước chương trình sẽ phải thực hiện bao nhiêu lần. Có những tình huống mà việc lặp chỉ kết thúc khi một điều kiện nào đó được thỏa mãn. Đối với những bài toán như vậy, Python cung cấp vòng lặp `while`.

Nếu `for` trả lời câu hỏi 'lặp với những giá trị nào?", thì `while` lại trả lời câu hỏi 'lặp cho đến khi nào?". Thay vì dựa vào một dãy giá trị có sẵn, `while` dựa vào một biểu thức điều kiện. Chừng nào điều kiện đó còn đúng, chương trình sẽ tiếp tục thực hiện khối lệnh bên trong vòng lặp. Khi điều kiện trở thành sai, vòng lặp tự động kết thúc và chương trình chuyển sang các câu lệnh tiếp theo.

Đây là một kiểu tư duy rất quen thuộc trong Toán học cũng như trong đời sống. Khi giải một phương trình bằng phương pháp gần đúng, ta tiếp tục tính toán cho đến khi sai số đủ nhỏ. Khi thử nhiều giá trị để tìm nghiệm của một bài toán, ta tiếp tục thử cho đến khi tìm được kết quả phù hợp. Khi học một dạng bài mới, ta luyện tập nhiều lần cho đến khi thành thạo. Điểm chung của tất cả các quá trình này là không thể xác định chính xác ngay từ đầu cần bao nhiêu lần lặp; số lần thực hiện phụ thuộc vào việc điều kiện kết thúc được thỏa mãn ở thời điểm nào.

Trong lập trình cũng vậy, rất nhiều bài toán không thể biết trước số lần lặp. Chẳng hạn, chương trình có thể yêu cầu người dùng nhập lại mật khẩu cho đến khi đúng, tiếp tục đọc dữ liệu cho đến khi gặp ký hiệu kết thúc, hoặc lặp việc tìm kiếm cho đến khi phát hiện đối tượng cần tìm. Trong những trường hợp như vậy, sử dụng `while` sẽ tự nhiên và hợp lý hơn nhiều so với `for`.

- Nhập mật khẩu cho đến khi người dùng nhập đúng.
- Tăng dần một giá trị cho đến khi đạt điều kiện mong muốn.
- Lặp kiểm tra nhiều trường hợp cho đến khi tìm thấy kết quả.
- Đọc dữ liệu liên tục cho đến khi hết dữ liệu.
- Thực hiện một quá trình tính toán cho đến khi sai số đạt yêu cầu.

Điểm quan trọng nhất của vòng lặp `while` là điều kiện được kiểm tra trước mỗi lần lặp. Nếu ngay từ đầu điều kiện đã sai thì khối lệnh bên trong sẽ không được thực hiện lần nào. Ngược lại, nếu điều kiện đúng thì chương trình sẽ thực hiện khối lệnh, sau đó quay trở lại kiểm tra điều kiện một lần nữa. Quá trình này cứ tiếp tục cho đến khi điều kiện trở thành sai.

Tìm số chia hết cho 7

```
so = 1

while so % 7 != 0:
    so = so + 1

print(f"So chia het cho 7 dau tien la: {so}")
```

Trong ví dụ trên, biến `so` được khởi tạo bằng giá trị 1. Sau đó, chương trình kiểm tra xem `so` đã chia hết cho 7 hay chưa. Nếu chưa chia hết, điều kiện của vòng lặp vẫn đúng và chương trình tăng giá trị của `so` thêm một đơn vị. Sau khi tăng xong, chương trình lại quay về kiểm tra điều kiện. Quá trình này tiếp tục cho đến khi `so` nhận giá trị 7. Lúc đó, điều kiện `so % 7 != 0` trở thành sai, vòng lặp kết thúc và kết quả được in ra màn hình.

Qua ví dụ này, chúng ta có thể thấy rằng chương trình không hề biết trước cần thực hiện bao nhiêu lần lặp. Nó chỉ liên tục tự đặt câu hỏi: 'Điều kiện còn đúng hay không?" Nếu câu trả lời là 'còn", chương trình tiếp tục làm việc; nếu câu trả lời là "không", chương trình dừng lại. Đây chính là bản chất của vòng lặp `while`.

5.10.1 Bản chất của vòng lặp `while`

Có thể xem vòng lặp `while` như một cơ chế "kiểm tra rồi mới hành động". Trước mỗi lần thực hiện khối lệnh, Python luôn đánh giá biểu thức điều kiện. Chỉ khi kết quả là `True`, các câu lệnh bên trong mới được thực hiện. Sau khi hoàn thành một lần lặp, chương trình không chuyển ngay sang phần tiếp theo mà quay trở lại kiểm tra điều kiện một lần nữa. Chu trình này lặp đi lặp lại cho đến khi điều kiện trở thành `False`.

Điểm khác biệt lớn nhất giữa `while` và `for` nằm ở cách điều khiển vòng lặp. Với `for`, số lần lặp được quyết định bởi số lượng phần tử trong dãy cần duyệt. Với `while`, số lần lặp lại phụ thuộc hoàn toàn vào sự thay đổi của điều kiện trong quá trình thực hiện chương trình. Chính vì vậy, khi sử dụng `while`, người lập trình phải đặc biệt chú ý đến việc điều kiện có thực sự thay đổi hay không.

Dưới góc nhìn của tư duy thuật toán, `while` mô tả rất nhiều quá trình có tính thích nghi. Chương trình không biết trước khi nào sẽ kết thúc, mà phải liên tục quan sát trạng thái hiện tại rồi mới quyết định có tiếp tục hay không. Đây là kiểu tư duy xuất hiện rất nhiều trong các thuật toán tìm kiếm, mô phỏng, tối ưu hóa và xử lý dữ liệu.

Vòng lặp vô hạn

Một trong những lỗi nghiêm trọng và cũng phổ biến nhất khi sử dụng `while` là tạo ra vòng lặp vô hạn. Điều này xảy ra khi điều kiện của vòng lặp luôn đúng và không bao giờ trở thành sai.

```
x = 1
```

```
2
3 while x > 0:
4     print("Chay mai...")
5     # Quen thay doi gia tri cua x
```

Trong đoạn chương trình trên, điều kiện `x > 0` luôn đúng vì giá trị của `x` không hề thay đổi trong suốt quá trình thực hiện. Sau mỗi lần in ra dòng chữ "Chay mai...", chương trình lại quay về kiểm tra điều kiện và tiếp tục thực hiện vòng lặp. Quá trình này sẽ lặp vô hạn cho đến khi người dùng chủ động dừng chương trình hoặc hệ điều hành buộc chương trình phải kết thúc.
Vòng lặp vô hạn không chỉ làm chương trình không thể hoàn thành nhiệm vụ mà còn có thể khiến máy tính tiêu tốn tài nguyên xử lý một cách không cần thiết. Vì vậy, mỗi khi viết một vòng lặp `while`, bạn luôn phải xác định rõ điều kiện nào sẽ làm cho vòng lặp kết thúc và điều kiện đó có thực sự thay đổi trong quá trình thực hiện hay không.

5.10.2 Sai lầm thường gặp với `while`

Đối với người mới học, vòng lặp `while` thường khó sử dụng hơn `for` vì người lập trình phải tự quản lý điều kiện lặp. Chỉ cần một sai sót nhỏ, chương trình có thể không hoạt động như mong muốn.

- Quên cập nhật biến quyết định điều kiện lặp, làm cho vòng lặp chạy mãi.
- Viết sai điều kiện khiến vòng lặp không được thực hiện lần nào.
- Cập nhật biến điều kiện không đúng vị trí, dẫn đến kết quả sai hoặc bỏ sót dữ liệu.
- Thay đổi sai giá trị của biến điều kiện làm cho vòng lặp kết thúc quá sớm hoặc quá muộn.

Để tránh những lỗi trên, mỗi khi viết một vòng lặp `while`, bạn nên tự đặt ra ba câu hỏi rất đơn giản nhưng quan trọng.

- Điều kiện nào quyết định việc tiếp tục hay dừng vòng lặp?
- Biến nào ảnh hưởng trực tiếp đến điều kiện đó?
- Trong mỗi lần lặp, biến này có thực sự được cập nhật để tiến gần đến điều kiện dừng hay không?

Nếu trả lời rõ ràng được ba câu hỏi này, bạn sẽ tránh được phần lớn các lỗi khi sử dụng `while`.

5.10.3 Tư duy kiểm tra liên tục

Điều làm cho vòng lặp `while` trở nên đặc biệt là nó mô phỏng rất tốt những quá trình "làm cho đến khi đạt mục tiêu". Đây là kiểu tư duy xuất hiện ở rất nhiều hoạt động trong học tập cũng như trong cuộc sống.

Khi giải một bài toán khó, bạn thường thử một cách giải, kiểm tra kết quả, nếu chưa đúng thì điều chỉnh rồi thử lại. Khi tìm kiếm một tài liệu trên Internet, bạn tiếp tục xem các kết quả cho đến khi tìm được thông tin cần thiết. Khi đăng nhập vào một hệ thống, bạn có thể phải nhập lại mật khẩu cho đến khi chính xác. Trong tất cả những tình huống đó, số lần thực hiện không được xác định trước; điều quan trọng chỉ là điều kiện kết thúc đã đạt hay chưa.

Có thể nói rằng, nếu vòng lặp `for` đại diện cho tư duy 'duyệt lần lượt", thì vòng lặp `while` đại diện cho tư duy 'kiểm tra liên tục". Hai cách tiếp cận này bổ sung cho nhau và xuất hiện ở hầu hết các thuật toán trong lập trình. Việc hiểu rõ bản chất của từng loại vòng lặp sẽ giúp bạn lựa chọn công cụ phù hợp và xây dựng chương trình một cách tự nhiên, hiệu quả hơn.

5.11 break và continue

Cho đến thời điểm này, các vòng lặp mà chúng ta đã học đều hoạt động theo một quy trình rất đơn giản. Sau khi bắt đầu, chương trình sẽ thực hiện lần lượt từng lần lặp cho đến khi hết dãy giá trị của `for` hoặc cho đến khi điều kiện của `while` trở thành sai. Tuy nhiên, trong nhiều bài toán thực tế, cách hoạt động này chưa đủ linh hoạt. Có những lúc chương trình đã tìm được kết quả mong muốn và không cần tiếp tục lặp nữa. Cũng có những trường hợp chỉ một vài dữ liệu là không hợp lệ, nhưng ta vẫn muốn bỏ qua chúng để tiếp tục xử lý các dữ liệu còn lại.

Để giải quyết những tình huống như vậy, Python cung cấp hai câu lệnh điều khiển đặc biệt là `break` và `continue`. Mặc dù chỉ gồm một từ khóa ngắn gọn, chúng cho phép người lập trình thay đổi luồng thực thi của vòng lặp theo những cách rất linh hoạt.

- `break`: kết thúc ngay lập tức vòng lặp hiện tại.
- `continue`: bỏ qua phần còn lại của lần lặp hiện tại và chuyển sang lần lặp tiếp theo.

Điểm khác biệt quan trọng là `break` làm cho vòng lặp chấm dứt hoàn toàn, trong khi `continue` chỉ ảnh hưởng đến một lần lặp duy nhất. Sau khi gặp `continue`, chương trình vẫn tiếp tục thực hiện các lần lặp còn lại nếu điều kiện cho phép.

Sử dụng break và continue

```
# break: Tim thay so 5 thi dung
for i in range(10):
    if i == 5:
        break
    print(i, end=" ")

print()

# continue: Bo qua so 5
for i in range(10):
```

```
    if i == 5:
        continue
    print(i, end=" ")
```

Ở vòng lặp đầu tiên, chương trình lần lượt in ra các số từ 0. Khi giá trị của `i` bằng 5, câu lệnh `break` được thực hiện và vòng lặp kết thúc ngay lập tức. Vì vậy, kết quả chỉ bao gồm các số từ 0 đến 4. Những giá trị phía sau như 5, 6, 7, 8 và 9 sẽ không bao giờ được xử lý.

Ở vòng lặp thứ hai, khi `i` bằng 5, câu lệnh `continue` được thực hiện. Thay vì kết thúc vòng lặp, chương trình chỉ bỏ qua câu lệnh `print()` của lần lặp hiện tại rồi chuyển ngay sang giá trị tiếp theo. Do đó, kết quả sẽ là các số từ 0 đến 9, nhưng thiếu số 5.

Qua ví dụ này, có thể thấy rằng cả hai câu lệnh đều làm thay đổi luồng thực thi của vòng lặp, nhưng theo hai cách hoàn toàn khác nhau. Hiểu rõ sự khác biệt này sẽ giúp bạn lựa chọn đúng công cụ cho từng bài toán.

5.11.1 Khi nào dùng `break`, khi nào dùng `continue`?

Trong nhiều thuật toán, chương trình không cần xử lý toàn bộ dữ liệu mà chỉ cần tìm được một kết quả đầu tiên thỏa mãn yêu cầu. Chẳng hạn, khi tìm một học sinh theo mã số, ngay sau khi tìm thấy học sinh cần tìm, chương trình không cần tiếp tục duyệt danh sách nữa. Nếu vẫn tiếp tục lặp, chương trình chỉ tốn thêm thời gian mà không mang lại thông tin mới. Trong trường hợp này, `break` là lựa chọn hợp lý vì nó giúp kết thúc vòng lặp ngay khi mục tiêu đã đạt được.

Ngược lại, có những bài toán yêu cầu xử lý toàn bộ dữ liệu nhưng một số phần tử không hợp lệ cần được bỏ qua. Ví dụ, khi tính điểm trung bình của một lớp học, nếu có một bản ghi bị thiếu dữ liệu, ta có thể bỏ qua bản ghi đó rồi tiếp tục xử lý các học sinh còn lại. Lúc này, `continue` sẽ phù hợp hơn vì nó chỉ bỏ qua một lần lặp mà không ảnh hưởng đến các lần lặp tiếp theo.

Có thể tóm tắt cách sử dụng hai câu lệnh này như sau:

- Dùng `break` khi mục tiêu của bài toán đã đạt được và không cần tiếp tục lặp.
- Dùng `continue` khi chỉ muốn bỏ qua một trường hợp đặc biệt nhưng vẫn tiếp tục xử lý các trường hợp còn lại.

Trong thực tế, việc sử dụng hợp lý `break` còn giúp chương trình chạy nhanh hơn. Nếu dữ liệu rất lớn, việc dừng sớm ngay khi đã tìm được kết quả sẽ tiết kiệm đáng kể thời gian xử lý. Đây là một ý tưởng thường gặp trong các thuật toán tìm kiếm và xử lý dữ liệu.

5.11.2 Hình ảnh tư duy

Một cách ghi nhớ đơn giản là liên hệ với việc đi trên một con đường có nhiều điểm dừng.

Nếu `break` xuất hiện, bạn có thể hình dung như đã đến đúng đích nên quyết định kết thúc chuyến đi ngay lập tức. Bạn không đi tiếp vì mục tiêu đã hoàn thành.

Ngược lại, `continue` giống như việc bạn đi ngang qua một điểm dừng nhưng quyết định không ghé vào. Bạn vẫn tiếp tục hành trình và sẽ ghé những điểm dừng tiếp theo như bình thường. Điều duy nhất bị bỏ qua chỉ là điểm dừng hiện tại.

Mặc dù đây chỉ là một hình ảnh minh họa, nhưng nó phản ánh rất đúng cách hoạt động của hai câu lệnh trong lập trình và thường giúp người mới học ghi nhớ dễ dàng hơn.

5.12 Ứng dụng của vòng lặp

Sau khi tìm hiểu các loại vòng lặp và cách điều khiển chúng, có thể thấy rằng vòng lặp không chỉ là một cấu trúc cú pháp của Python mà còn là nền tảng của rất nhiều thuật toán. Hầu như mọi chương trình có xử lý dữ liệu đều sử dụng vòng lặp ở một mức độ nào đó. Mỗi khi máy tính phải thực hiện cùng một thao tác nhiều lần với những dữ liệu khác nhau, vòng lặp gần như luôn xuất hiện.

Trong các chương trình đơn giản, vòng lặp thường được dùng để thực hiện các phép tính lặp lại hoặc hiển thị dữ liệu. Khi chương trình trở nên phức tạp hơn, vòng lặp còn được sử dụng để duyệt các cấu trúc dữ liệu, tìm kiếm thông tin, xử lý tệp, giao tiếp với người dùng hoặc thực hiện các thuật toán tính toán.

Một số ứng dụng tiêu biểu của vòng lặp gồm có:

- Tính tổng, tích hoặc các đại lượng được xác định từ nhiều số.
- Tính giai thừa, lũy thừa hoặc các dãy số quen thuộc trong Toán học.
- Lập bảng cửu chương và bảng giá trị của một hàm số.
- Duyệt qua danh sách, chuỗi ký tự hoặc các cấu trúc dữ liệu khác.
- Kiểm tra dữ liệu đầu vào cho đến khi người dùng nhập đúng.
- Tìm kiếm phần tử đầu tiên thỏa mãn một điều kiện.
- In các mẫu hình hoặc các quy luật trên màn hình.
- Mô phỏng các quá trình gồm nhiều bước lặp liên tiếp.

Đối với người học Toán, vòng lặp còn là cầu nối giữa tư duy toán học và tư duy thuật toán. Nhiều bài toán quen thuộc như tính tổng của một dãy số, lập bảng biến thiên đơn giản, khảo sát nhiều giá trị hoặc thử nhiều trường hợp đều có thể được mô tả rất tự nhiên bằng vòng lặp. Khi chuyển những ý tưởng đó sang Python, bạn không chỉ học thêm một cú pháp mới mà còn học cách mô hình hóa một quá trình tính toán bằng ngôn ngữ của máy tính.

Có thể nói rằng sức mạnh của máy tính không nằm ở việc thực hiện một phép tính đơn lẻ mà nằm ở khả năng lặp lại một quy trình với tốc độ rất cao và độ chính xác gần như tuyệt đối. Chỉ cần người lập trình mô tả đúng quy luật, máy tính có thể thực hiện hàng triệu hoặc hàng tỷ lần lặp mà vẫn duy trì kết quả chính xác. Đây chính là nền tảng của hầu hết các chương trình và thuật toán hiện đại.

5.13 Bài tập luyện tập

Sau khi hoàn thành chương này, bạn nên dành thời gian tự xây dựng các chương trình nhỏ để rèn luyện tư duy về vòng lặp. Trong giai đoạn đầu học lập trình, việc tự viết chương trình quan trọng hơn nhiều so với việc chỉ đọc ví dụ. Mỗi bài tập dưới đây đều hướng đến một kỹ năng cụ thể, từ việc sử dụng vòng lặp đơn giản cho đến kết hợp vòng lặp với điều kiện để giải quyết các bài toán thực tế.

1. Viết chương trình nhập vào một số nguyên dương `n` và in ra bảng cửu chương của số đó từ 1 đến 10.
2. Viết chương trình yêu cầu người dùng nhập mật khẩu. Nếu nhập sai, chương trình tiếp tục yêu cầu nhập lại. Khi nhập đúng mật khẩu `python123`, thông báo đăng nhập thành công và kết thúc chương trình.
3. Viết chương trình tính tổng tất cả các số lẻ từ 1 đến 100.
4. Viết chương trình in ra các số từ 1 đến 20, nhưng bỏ qua hai số 5 và 10.
5. Viết chương trình tìm số đầu tiên chia hết cho 9 trong khoảng từ 1 đến 100, sau đó kết thúc việc tìm kiếm ngay khi tìm thấy.
6. Viết chương trình nhập số nguyên dương `n` và tính tổng các số từ 1 đến `n` bằng vòng lặp `for`.
7. Viết chương trình nhập một số nguyên dương và sử dụng vòng lặp `while` để in các số từ 1 đến số đó.

5.14 Tóm tắt cuối chương

Qua chương này, chúng ta đã tìm hiểu một trong ba cấu trúc điều khiển quan trọng nhất của lập trình là vòng lặp. Nếu câu lệnh điều kiện giúp chương trình biết cách lựa chọn giữa nhiều hướng xử lý, thì vòng lặp giúp chương trình thực hiện một quy trình nhiều lần theo một quy luật xác định. Hai cấu trúc này kết hợp với nhau tạo thành nền tảng của hầu hết các thuật toán cơ bản.

Những ý chính cần ghi nhớ gồm có:

- Vòng lặp cho phép máy tính thực hiện cùng một công việc nhiều lần một cách tự động.
- Vòng lặp `for` thích hợp khi biết trước số lần lặp hoặc khi cần duyệt qua một dãy giá trị.
- Hàm `range()` giúp tạo ra các dãy số để sử dụng với vòng lặp `for`.
- Vòng lặp `while` phù hợp khi việc lặp phụ thuộc vào một điều kiện thay đổi trong quá trình thực hiện.
- Khi sử dụng `while`, cần đảm bảo điều kiện lặp sẽ thay đổi để tránh tạo ra vòng lặp vô hạn.

- Câu lệnh `break` dùng để kết thúc ngay vòng lặp, còn `continue` dùng để bỏ qua lần lặp hiện tại và chuyển sang lần lặp tiếp theo.
- Vòng lặp là nền tảng của rất nhiều thuật toán trong lập trình, từ các bài toán đơn giản đến những hệ thống phần mềm có quy mô lớn.

Góc nhìn cá nhân

Nếu câu lệnh điều kiện giúp chương trình biết cách đưa ra quyết định, thì vòng lặp lại giúp chương trình biết cách thực hiện một quy trình một cách có hệ thống. Đây là một bước chuyển rất quan trọng trong quá trình học lập trình. Bạn không còn viết từng hành động riêng lẻ nữa, mà bắt đầu suy nghĩ theo hướng mô tả một quy luật rồi giao cho máy tính thực hiện quy luật đó nhiều lần. Đó cũng chính là một trong những đặc điểm quan trọng nhất của tư duy thuật toán.

Đối với người học Toán, ý tưởng này không hề xa lạ. Những dãy số, các quy luật truy hồi, việc lập bảng giá trị, tính tổng nhiều số hạng, thử nhiều trường hợp hay lặp các phép biến đổi đều là những quá trình lặp đã xuất hiện từ rất sớm trong quá trình học tập. Python chỉ mang đến cho bạn một công cụ để diễn đạt những quy luật ấy dưới dạng chương trình.

Khi thực sự hiểu bản chất của vòng lặp, bạn sẽ nhận ra rằng lập trình không phải là ghi nhớ thật nhiều cú pháp, mà là học cách nhận diện quy luật của một bài toán rồi mô tả quy luật đó một cách chính xác. Một khi đã làm được điều đó, máy tính sẽ trở thành người cộng sự kiên nhẫn nhất, sẵn sàng thực hiện hàng triệu lần lặp với tốc độ và độ chính xác mà con người khó có thể đạt được.

Chương 6

Tổ chức mã nguồn - Hàm (Functions)

Giới thiệu ngắn

Ở những chương đầu tiên, các chương trình của chúng ta chỉ gồm vài dòng lệnh nên rất dễ đọc và dễ hiểu. Tuy nhiên, khi bài toán trở nên phức tạp hơn, số lượng câu lệnh cũng tăng lên nhanh chóng. Một chương trình có thể phải nhập dữ liệu, kiểm tra điều kiện, thực hiện nhiều phép tính khác nhau, xử lý kết quả rồi mới hiển thị ra màn hình. Nếu tất cả các công việc đó đều được viết liên tiếp trong một khối mã nguồn duy nhất, chương trình sẽ trở nên dài, khó theo dõi và rất khó sửa đổi khi có yêu cầu mới.

Một vấn đề khác cũng thường xuyên xuất hiện là sự lặp lại của mã nguồn. Có những đoạn chương trình được sử dụng ở nhiều nơi với nội dung gần như giống hệt nhau. Chẳng hạn, bạn có thể phải tính diện tích hình chữ nhật ở nhiều vị trí khác nhau, hoặc phải kiểm tra nhiều lần xem một số có phải số nguyên tố hay không. Nếu mỗi lần cần sử dụng đều sao chép lại toàn bộ đoạn mã, chương trình sẽ ngày càng cồng kềnh. Đến khi cần sửa một lỗi hoặc thay đổi thuật toán, bạn sẽ phải tìm và chỉnh sửa ở rất nhiều vị trí khác nhau, rất dễ bỏ sót và phát sinh lỗi mới.

Trong kỹ thuật phần mềm có một nguyên tắc nổi tiếng mang tên **DRY** (*Don't Repeat Yourself*), nghĩa là *đừng lặp lại chính mình*. Ý tưởng của nguyên tắc này rất đơn giản: nếu một công việc được sử dụng nhiều lần thì chỉ nên viết một lần, sau đó tái sử dụng ở mọi nơi cần thiết. Đây là một trong những nguyên tắc quan trọng giúp chương trình trở nên gọn gàng, dễ đọc và dễ bảo trì.

Python giải quyết vấn đề đó bằng **hàm** (*function*). Một hàm cho phép bạn gom nhiều câu lệnh thực hiện cùng một nhiệm vụ vào trong một đơn vị có tên. Sau khi định nghĩa xong, bạn chỉ cần gọi tên hàm mỗi khi cần sử dụng. Thay vì quan tâm đến từng câu lệnh bên trong, bạn chỉ cần biết hàm đó thực hiện công việc gì và cung cấp cho nó những dữ liệu cần thiết.

Có thể xem hàm như một chiếc máy trong dây chuyền sản xuất. Bạn đưa nguyên liệu vào, chiếc máy xử lý theo đúng quy trình đã được thiết kế, sau đó tạo ra kết quả. Mỗi lần cần thực hiện cùng một công việc, bạn chỉ cần sử dụng lại chiếc máy đó mà không phải chế tạo một chiếc máy mới.

Đối với người học Toán, ý tưởng này cũng rất quen thuộc. Trong Toán học, khi định

nghĩa một hàm số

$$f(x) = x^2 + 2x + 1,$$

bạn không cần viết lại biểu thức mỗi khi muốn tính giá trị tại một số cụ thể. Chỉ cần viết

$$f(3), \qquad f(5), \qquad f(10),$$

là mọi người đều hiểu phải thay giá trị tương ứng vào công thức đã định nghĩa. Hàm trong lập trình cũng hoạt động theo tinh thần đó. Bạn định nghĩa cách xử lý một lần, sau đó có thể sử dụng lại với rất nhiều dữ liệu khác nhau.

Trong chương này, chúng ta sẽ tìm hiểu cách xây dựng các hàm trong Python, cách truyền dữ liệu vào hàm thông qua tham số, cách nhận kết quả bằng câu lệnh `return`, đồng thời hình thành một tư duy rất quan trọng trong lập trình: chia một bài toán lớn thành nhiều bài toán nhỏ, mỗi bài toán nhỏ được giải quyết bởi một hàm riêng. Đây là nền tảng của việc xây dựng những chương trình có cấu trúc rõ ràng và dễ mở rộng.

Mục tiêu học tập

Sau khi học xong chương này, bạn cần đạt được những mục tiêu sau:

- Hiểu được khái niệm hàm và vai trò của hàm trong việc tổ chức mã nguồn.
- Hiểu vì sao hàm giúp chương trình ngắn gọn, dễ đọc, dễ kiểm thử và dễ bảo trì.
- Biết cách định nghĩa hàm bằng từ khóa `def`.
- Phân biệt được tham số (*parameter*) và đối số (*argument*).
- Biết cách truyền dữ liệu vào hàm thông qua tham số.
- Hiểu ý nghĩa của câu lệnh `return` và biết cách sử dụng giá trị mà hàm trả về.
- Phân biệt được hàm chỉ thực hiện công việc và hàm trả về kết quả để tiếp tục tính toán.
- Bước đầu hình thành tư duy chia nhỏ bài toán thành các hàm độc lập để xây dựng chương trình có cấu trúc.

6.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một bài tính điểm tuyển sinh có các bước chuẩn hóa điểm, tính tổng theo tổ hợp, cộng ưu tiên và sinh thông báo. Viết tất cả trong một khối khiến mọi thay đổi ảnh hưởng lẫn nhau. Tư duy hàm yêu cầu tách những phép biến đổi có tên, chỉ rõ đầu vào, đầu ra và điều kiện đúng của từng phép biến đổi trước khi ghép chúng.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **hàm như hợp đồng, phép biến đổi và công cụ chia nhỏ bài toán**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

6.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tham số, miền dữ liệu, giá trị trả về, tác dụng phụ, tiền điều kiện và hậu điều kiện.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

6.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$R(x) = f_4(f_3(f_2(f_1(x)))), \qquad f_i : \mathcal{D}_i \to \mathcal{D}_{i+1}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: đối số nhận được, biến cục bộ và kết quả trung gian của lần gọi hiện tại?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

6.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Một hàm phải thực hiện đúng trách nhiệm đã đặt tên; với cùng đầu vào hợp lệ, hàm thuần phải trả cùng kết quả và không âm thầm sửa dữ liệu bên ngoài.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

6.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **hàm như hợp đồng, phép biến đổi và công cụ chia nhỏ bài toán** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

6.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Tách bộ tính điểm thành `kiem_tra_diem`, `tinh_tong`, `cong_uu_tien` và `xep_ket_qua`. Mỗi hàm được kiểm tra riêng bằng ví dụ tay; sau đó mới kiểm tra phép ghép toàn bộ. Khi sai, ta xác định được hợp đồng nào bị vi phạm.

6.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

6.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

6.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

6.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

6.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

6.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

6.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Nhầm giá trị được `return` với nội dung được `print`.
- Để hàm vừa tính toán, vừa nhập dữ liệu và vừa trình bày kết quả.
- Sửa biến toàn cục khiến kết quả phụ thuộc thứ tự gọi hàm.

- Đặt tên hàm theo cách thực hiện thay vì theo trách nhiệm của nó.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

6.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

6.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết hợp đồng cho hàm tính diện tích tam giác theo công thức Heron, gồm tiền điều kiện.
2. Tách một chương trình đổi đơn vị thành ba hàm thuần và một hàm giao tiếp.
3. Dựng sơ đồ lời gọi cho bài kiểm tra số nguyên tố và truy vết giá trị trả về.
4. Tìm tác dụng phụ trong một đoạn hàm cho sẵn và đề xuất cách loại bỏ.
5. Thiết kế kiểm thử đơn vị cho hàm phân loại điểm tại các giá trị sát ngưỡng.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

6.8 Ý nghĩa của hàm

Trong những chương trước, các chương trình mà chúng ta viết còn khá ngắn nên việc đặt toàn bộ câu lệnh trong một tệp duy nhất không gây nhiều khó khăn. Tuy nhiên, khi chương trình ngày càng lớn, số lượng câu lệnh tăng lên nhanh chóng. Một bài toán hoàn chỉnh có thể bao gồm nhiều bước khác nhau: nhập dữ liệu, kiểm tra tính hợp lệ, thực hiện tính toán, xử lý kết quả và cuối cùng là hiển thị thông tin cho người dùng. Nếu tất cả những công việc

này đều được viết liên tiếp trong một khối mã nguồn duy nhất, chương trình sẽ trở nên dài, khó đọc và rất khó xác định vị trí xảy ra lỗi.

Một trong những nguyên nhân khiến chương trình trở nên phức tạp là sự lặp lại của mã nguồn. Hãy tưởng tượng bạn cần tính diện tích hình chữ nhật ở nhiều vị trí khác nhau trong cùng một chương trình. Nếu mỗi lần đều viết lại công thức `chieu_dai * chieu_rong`, chương trình không chỉ dài hơn mà còn khó bảo trì. Đến một lúc nào đó, nếu công thức hoặc cách tính cần thay đổi, bạn sẽ phải tìm và sửa ở tất cả những nơi đã sao chép đoạn mã đó. Chỉ cần bỏ sót một vị trí, chương trình có thể cho ra kết quả không nhất quán.

Hàm được tạo ra để giải quyết chính vấn đề này. Thay vì viết đi viết lại cùng một nhóm câu lệnh, ta đóng gói chúng thành một đơn vị có tên. Mỗi đơn vị như vậy thực hiện một nhiệm vụ cụ thể và có thể được sử dụng nhiều lần ở bất kỳ đâu trong chương trình. Khi cần thực hiện công việc đó, ta chỉ việc gọi tên hàm mà không cần quan tâm đến toàn bộ chi tiết xử lý bên trong.

Một chương trình được chia thành nhiều hàm thường có cấu trúc rõ ràng hơn rất nhiều. Mỗi hàm chỉ chịu trách nhiệm cho một nhiệm vụ riêng biệt. Chẳng hạn, một hàm dùng để nhập dữ liệu, một hàm dùng để kiểm tra điều kiện, một hàm dùng để tính toán và một hàm khác dùng để hiển thị kết quả. Khi đọc chương trình, bạn không còn phải theo dõi hàng trăm dòng lệnh liên tiếp, mà chỉ cần hiểu vai trò của từng hàm rồi xem cách chúng phối hợp với nhau. Đây cũng là cách mà hầu hết các phần mềm thực tế được xây dựng.

Không chỉ giúp chương trình ngắn gọn hơn, hàm còn giúp việc kiểm tra và sửa lỗi trở nên đơn giản. Nếu chương trình cho kết quả sai, ta có thể kiểm tra từng hàm riêng lẻ để xác định nguyên nhân thay vì phải đọc lại toàn bộ mã nguồn. Điều này đặc biệt quan trọng khi chương trình có hàng nghìn hoặc hàng chục nghìn dòng lệnh.

Một lợi ích khác của hàm là khả năng tái sử dụng. Sau khi viết xong một hàm, bạn có thể gọi lại nó nhiều lần trong cùng một chương trình hoặc thậm chí sử dụng lại ở những dự án khác. Đây là một trong những nguyên tắc cơ bản của lập trình hiện đại: thay vì liên tục viết mã mới, hãy xây dựng những thành phần có thể dùng lại nhiều lần.

Đối với người học Toán, tư duy này hoàn toàn không xa lạ. Khi gặp biểu thức

$$x^2 + 2x + 1,$$

nếu phải sử dụng nhiều lần, bạn thường đặt

$$f(x) = x^2 + 2x + 1.$$

Kể từ đó, thay vì phải viết lại toàn bộ biểu thức, bạn chỉ cần viết $f(2)$, $f(5)$ hoặc $f(a)$. Việc đặt tên cho một công thức giúp lời giải ngắn gọn và dễ theo dõi hơn. Hàm trong lập trình cũng hoạt động theo cách tương tự. Thay vì lặp đi lặp lại nhiều câu lệnh, ta đặt cho chúng một cái tên để có thể gọi lại bất cứ khi nào cần.

Có thể hình dung một hàm như một chiếc máy trong dây chuyền sản xuất. Bạn đưa dữ liệu đầu vào vào chiếc máy, chiếc máy thực hiện một quy trình đã được thiết kế sẵn, rồi tạo ra kết quả đầu ra. Người sử dụng không nhất thiết phải biết mọi chi tiết bên trong chiếc

máy; điều quan trọng là biết nó nhận dữ liệu gì và tạo ra kết quả gì. Chính cách tổ chức này giúp các chương trình lớn trở nên có cấu trúc, dễ mở rộng và dễ bảo trì hơn.

6.8.1 Tư duy chia nhỏ bài toán

Một trong những kỹ năng quan trọng nhất của người lập trình là biết chia một bài toán lớn thành nhiều bài toán nhỏ hơn. Thay vì cố gắng giải quyết toàn bộ vấn đề trong một đoạn mã dài, ta tách chương trình thành nhiều phần, mỗi phần đảm nhận đúng một nhiệm vụ. Sau đó, các phần này phối hợp với nhau để tạo nên lời giải hoàn chỉnh.

Ví dụ, hãy xét bài toán quản lý điểm của một học sinh. Nếu viết toàn bộ chương trình trong một khối lệnh, bạn sẽ phải thực hiện liên tiếp việc nhập điểm, kiểm tra dữ liệu, tính điểm trung bình, xếp loại và in kết quả. Khi chương trình dài ra, việc đọc và sửa đổi sẽ trở nên rất khó khăn.

Ngược lại, nếu chia chương trình thành các hàm, cấu trúc sẽ rõ ràng hơn rất nhiều. Một hàm chỉ nhập dữ liệu, một hàm tính điểm trung bình, một hàm xếp loại học lực và một hàm hiển thị kết quả. Mỗi hàm chỉ giải quyết một nhiệm vụ duy nhất. Nhờ vậy, chương trình không chỉ dễ đọc mà còn dễ kiểm tra và mở rộng khi cần bổ sung chức năng mới.

Đây cũng là cách làm việc phổ biến trong các dự án phần mềm chuyên nghiệp. Một chương trình lớn không được xây dựng bởi một đoạn mã khổng lồ, mà được ghép từ rất nhiều hàm nhỏ, mỗi hàm giống như một viên gạch trong toàn bộ công trình. Khi nắm được tư duy này, bạn sẽ không chỉ học cách viết hàm, mà còn học được cách tổ chức chương trình một cách khoa học và có hệ thống.

6.9 Định nghĩa hàm với `def`

Sau khi hiểu vì sao cần sử dụng hàm, chúng ta sẽ tìm hiểu cách tạo một hàm trong Python. Mỗi hàm được định nghĩa bằng từ khóa `def`, viết tắt của từ *definition* (định nghĩa). Sau từ khóa này là tên hàm, cặp dấu ngoặc đơn `()` và dấu hai chấm `:`. Phía dưới là khối lệnh được thụt lề, chứa các câu lệnh mà hàm sẽ thực hiện.

Cấu trúc tổng quát của một hàm như sau:

```
def ten_ham():
    # Cac cau lenh cua ham
    ...
```

Điều quan trọng cần hiểu là việc **định nghĩa hàm chưa làm cho các câu lệnh bên trong được thực thi**. Khi Python đọc đến câu lệnh `def`, nó chỉ ghi nhớ rằng từ thời điểm đó trở đi sẽ tồn tại một hàm có tên như vậy. Chỉ khi bạn gọi hàm, các câu lệnh bên trong mới được thực hiện.

Đây là điểm mà nhiều người mới học thường nhầm lẫn. Họ nghĩ rằng chỉ cần viết hàm là chương trình sẽ tự chạy các câu lệnh bên trong. Thực tế không phải như vậy. Định nghĩa hàm giống như việc viết ra một công thức hoặc một quy trình xử lý. Công thức chỉ được áp dụng khi ta thực sự sử dụng nó.

Tạo hàm đơn giản

```
def chao_hoi():
    print("Xin chao cac ban!")
    print("Chuc mot ngay tot lanh!")

# Goi ham
chao_hoi()
```

Khi chương trình chạy đến dòng

```
def chao_hoi():
```

Python chỉ ghi nhớ sự tồn tại của hàm `chao_hoi`. Hai câu lệnh `print()` bên trong chưa được thực hiện ngay. Chỉ đến khi gặp dòng

```
chao_hoi()
```

chương trình mới chuyển quyền thực thi vào bên trong hàm, lần lượt thực hiện hai câu lệnh `print()`, sau đó quay trở lại vị trí ngay sau lời gọi hàm để tiếp tục chạy các câu lệnh còn lại.

6.9.1 Luồng thực thi của chương trình

Để hiểu rõ hơn, hãy quan sát quá trình thực thi của ví dụ trên.

1. Python đọc câu lệnh `def` và tạo ra hàm `chao_hoi`.
2. Chương trình tiếp tục đọc các dòng tiếp theo.
3. Khi gặp lời gọi `chao_hoi()`, Python chuyển sang thực hiện toàn bộ các câu lệnh bên trong hàm.
4. Sau khi thực hiện xong câu lệnh cuối cùng của hàm, chương trình quay trở lại vị trí sau lời gọi và tiếp tục chạy.

Có thể hình dung hàm giống như một căn phòng trong ngôi nhà. Khi chương trình gặp lời gọi hàm, nó bước vào căn phòng đó, thực hiện mọi công việc bên trong, rồi bước ra để tiếp tục hành trình của mình.

6.9.2 Quy tắc đặt tên hàm

Tên hàm nên phản ánh đúng nhiệm vụ mà hàm thực hiện. Một tên hàm rõ ràng giúp người đọc hiểu ngay mục đích của chương trình mà không cần xem toàn bộ mã nguồn.

Trong Python, tên hàm thường được viết bằng chữ thường, các từ được nối với nhau bằng dấu gạch dưới (_). Đây là quy ước được khuyến nghị trong tài liệu chính thức của Python.

Một số ví dụ về tên hàm phù hợp là:

- `tinh_tong()`
- `kiem_tra_so_nguyen_to()`
- `tinh_dien_tich()`
- `xep_loai_hoc_luc()`

Ngược lại, những tên quá ngắn hoặc không có ý nghĩa như

```
def abc():
    ...

def x():
    ...

def ham1():
    ...
```

không giúp người đọc hiểu hàm dùng để làm gì. Khi chương trình có hàng chục hoặc hàng trăm hàm, việc đặt tên không rõ ràng sẽ khiến mã nguồn rất khó bảo trì.

6.9.3 Mỗi hàm nên thực hiện một nhiệm vụ

Một nguyên tắc quan trọng trong lập trình là mỗi hàm chỉ nên đảm nhận một công việc chính. Khi một hàm vừa nhập dữ liệu, vừa tính toán, vừa in kết quả, vừa ghi tệp, nó sẽ trở nên dài và khó sửa đổi.

Ngược lại, nếu mỗi hàm chỉ thực hiện một nhiệm vụ, chương trình sẽ dễ đọc hơn rất nhiều. Chẳng hạn:

- Một hàm chỉ dùng để nhập dữ liệu.
- Một hàm chỉ dùng để tính toán.
- Một hàm chỉ dùng để kiểm tra điều kiện.
- Một hàm chỉ dùng để hiển thị kết quả.

Cách tổ chức này giúp mỗi hàm trở thành một ”viên gạch” nhỏ trong toàn bộ chương trình. Khi cần thay đổi một chức năng, ta chỉ cần chỉnh sửa đúng hàm liên quan mà không ảnh hưởng đến các phần khác.

6.9.4 Liên hệ với Toán học

Trong Toán học, khi định nghĩa

$$f(x) = x^2 + 1,$$

ta chưa hề tính toán bất kỳ giá trị nào. Ta chỉ đang nói rằng từ bây giờ, ký hiệu f sẽ đại diện cho quy tắc ”lấy bình phương rồi cộng thêm 1”.

Chỉ khi viết

$$f(3),$$

ta mới thực sự áp dụng quy tắc đó để thu được kết quả bằng 10.

Hàm trong Python cũng hoạt động theo đúng tinh thần này. Lệnh `def` chỉ định nghĩa cách xử lý. Việc tính toán chỉ diễn ra khi chương trình gọi hàm. Hiểu được sự khác biệt giữa *định nghĩa* và *gọi hàm* sẽ giúp bạn tránh được rất nhiều nhầm lẫn khi bắt đầu học lập trình.

6.10 Tham số (Parameters) và Đối số (Arguments)

Ở phần trước, chúng ta đã tạo một hàm không nhận dữ liệu đầu vào. Mỗi lần gọi hàm, chương trình luôn thực hiện cùng một công việc và tạo ra cùng một kết quả. Tuy nhiên, trong thực tế, các hàm thường cần làm việc với nhiều dữ liệu khác nhau. Chẳng hạn, nếu muốn viết một hàm chào hỏi người dùng, ta không thể tạo riêng một hàm cho từng người như `chao_an()`, `chao_binh()` hay `chao_lan()`. Thay vào đó, ta chỉ cần xây dựng một hàm tổng quát, rồi truyền tên của người cần chào vào mỗi lần gọi.

Đó chính là vai trò của **tham số** (*parameter*) và **đối số** (*argument*). Hai khái niệm này rất dễ bị nhầm lẫn đối với người mới học, nhưng thực chất chúng xuất hiện ở hai thời điểm khác nhau.

Tham số là các biến được khai báo trong phần định nghĩa của hàm. Chúng đóng vai trò như những ”ô trống” để chờ nhận dữ liệu khi hàm được gọi. Ngược lại, đối số là những giá trị thực tế được truyền vào các ô trống đó trong lúc chương trình thực thi.

Nói cách khác, tham số xuất hiện khi bạn **xây dựng** hàm, còn đối số xuất hiện khi bạn **sử dụng** hàm.

Hàm có một tham số

```
def chao_ten(ten):
    print(f"Xin chao, {ten}!")

chao_ten("An")
chao_ten("Binh")
chao_ten("Chi")
```

Trong ví dụ trên, biến `ten` là tham số vì nó được khai báo trong phần định nghĩa hàm. Khi gọi

```
chao_ten("An")
```

thì chuỗi `"An"` là đối số. Python sẽ gán giá trị này cho tham số `ten`, sau đó thực hiện các câu lệnh bên trong hàm. Tương tự, ở hai lần gọi tiếp theo, tham số `ten` lần lượt nhận giá trị `"Binh"` và `"Chi"`.

Điều đáng chú ý là chúng ta chỉ viết hàm đúng một lần, nhưng có thể sử dụng với rất nhiều dữ liệu khác nhau. Đây chính là sức mạnh của hàm trong việc tái sử dụng mã nguồn.

6.10.1 Hàm có nhiều tham số

Một hàm không bị giới hạn chỉ có một tham số. Tùy vào yêu cầu của bài toán, bạn có thể khai báo nhiều tham số để nhận nhiều dữ liệu đầu vào.

Hàm tính tổng hai số

```
def tinh_tong(a, b):
    print(f"Tong la: {a + b}")

tinh_tong(5, 10)
tinh_tong(12, 8)
```

Ở đây, `a` và `b` đều là tham số. Trong lời gọi

```
tinh_tong(5, 10)
```

giá trị `5` được gán cho `a`, còn giá trị `10` được gán cho `b`. Sau khi việc gán hoàn tất, hàm thực hiện phép cộng và hiển thị kết quả.

Bạn có thể hình dung quá trình này giống như việc điền thông tin vào một biểu mẫu. Các tham số là những ô cần điền, còn đối số là nội dung thực tế mà người dùng nhập vào từng ô.

6.10.2 Sự tương ứng giữa tham số và đối số

Thông thường, số lượng đối số phải khớp với số lượng tham số đã khai báo. Nếu hàm cần hai tham số nhưng bạn chỉ truyền vào một giá trị, Python sẽ không biết phải gán giá trị còn thiếu cho biến nào và sẽ sinh lỗi.

Ví dụ sau đây không hợp lệ:

```
def tinh_tong(a, b):
    print(a + b)

tinh_tong(5)
```

Trong trường hợp này, tham số `b` không nhận được giá trị nên chương trình không thể thực hiện lời gọi hàm.

Ngược lại, nếu truyền quá nhiều đối số so với số tham số được khai báo, chương trình cũng sẽ báo lỗi vì Python không biết phải sử dụng giá trị dư thừa như thế nào.

Vì vậy, khi gọi một hàm, bạn cần chú ý truyền đúng số lượng dữ liệu mà hàm yêu cầu.

6.10.3 Luồng truyền dữ liệu vào hàm

Có thể mô tả quá trình hoạt động của một hàm theo bốn bước liên tiếp:

1. Chương trình gặp lời gọi hàm.
2. Các đối số được truyền vào và gán cho các tham số tương ứng.
3. Hàm thực hiện các câu lệnh bên trong bằng các giá trị vừa nhận.
4. Sau khi hoàn thành, chương trình quay trở lại vị trí ngay sau lời gọi hàm.

Việc hiểu rõ luồng thực thi này sẽ giúp bạn dễ dàng theo dõi chương trình khi các hàm bắt đầu gọi lẫn nhau ở những chương sau.

6.10.4 Liên hệ với hàm số trong Toán học

Khái niệm tham số trong lập trình rất gần với biến số trong Toán học.

Khi viết

$$f(x) = 2x + 3,$$

ký hiệu x chỉ là một biến đại diện cho mọi giá trị có thể. Chỉ khi tính

$$f(5),$$

ta mới thay 5 vào vị trí của x để nhận được kết quả.

Trong Python, tham số cũng đóng vai trò tương tự. Khi định nghĩa

```
def nhan_doi(x):
    print(x * 2)
```

thì `x` chỉ là một biến đại diện. Khi gọi

```
nhan_doi(7)
```

giá trị `7` được thay vào vị trí của `x`, sau đó hàm mới bắt đầu thực hiện phép tính. Chính cơ chế này làm cho một hàm có thể xử lý vô số trường hợp khác nhau mà không cần viết lại mã nguồn.

6.10.5 Vì sao nên sử dụng tham số?

Nếu không có tham số, mỗi khi dữ liệu thay đổi bạn sẽ phải sửa trực tiếp bên trong hàm hoặc viết thêm một hàm mới. Điều đó làm chương trình mất đi tính tổng quát và rất khó bảo trì.

Nhờ có tham số, cùng một hàm có thể được sử dụng với nhiều dữ liệu khác nhau mà không cần thay đổi cấu trúc của nó. Đây là một trong những yếu tố quan trọng giúp chương

trình trở nên linh hoạt, tái sử dụng được và dễ mở rộng khi bài toán ngày càng phức tạp.

6.11 Lệnh return – Trả về giá trị

Ở phần trước, các hàm của chúng ta chủ yếu sử dụng `print()` để hiển thị kết quả ra màn hình. Cách làm này phù hợp khi mục đích duy nhất là thông báo cho người dùng. Tuy nhiên, trong lập trình thực tế, kết quả của một phép tính thường không chỉ để xem, mà còn cần được sử dụng tiếp trong các phép tính khác. Đó là lý do câu lệnh `return` trở thành một trong những thành phần quan trọng nhất của hàm.

Có thể hiểu đơn giản rằng `return` là cách để hàm gửi kết quả trở lại nơi đã gọi nó. Sau khi nhận được giá trị này, chương trình có thể lưu vào biến, tiếp tục tính toán, truyền cho hàm khác hoặc sử dụng trong các biểu thức phức tạp hơn.

Nói cách khác, nếu xem một hàm như một chiếc máy, thì tham số là nguyên liệu đầu vào, còn `return` chính là sản phẩm đầu ra của chiếc máy đó.

Sử dụng return

```
1 def nhan_doi(so):
2     return so * 2
3
4 ket_qua = nhan_doi(5)
5
6 print(ket_qua)
7 print(nhan_doi(10) + 5)
```

Trong ví dụ trên, khi gọi

```
1 nhan_doi(5)
```

Python thực hiện phép nhân

$$5 \times 2 = 10.$$

sau đó câu lệnh

```
1 return so * 2
```

trả giá trị `10` về nơi gọi hàm. Giá trị này được gán cho biến `ket_qua`, vì vậy chương trình có thể tiếp tục sử dụng ở những dòng lệnh phía sau.

Ở lời gọi thứ hai,

```
1 print(nhan_doi(10) + 5)
```

hàm nhận đối số `10`, thực hiện phép tính

$$10 \times 2 = 20,$$

rồi trả về giá trị `20`. Sau đó Python mới thực hiện phép cộng với `5` để thu được kết quả cuối cùng là `25`. Điều này cho thấy giá trị trả về của hàm hoàn toàn có thể tham gia vào các biểu thức tính toán giống như một biến thông thường.

6.11.1 `print()` và `return` khác nhau như thế nào?

Đây là điểm khiến rất nhiều người mới học nhầm lẫn.

Lệnh `print()` chỉ có nhiệm vụ hiển thị thông tin ra màn hình. Sau khi hiển thị xong, giá trị đó không được gửi trở lại nơi gọi hàm. Nếu muốn tiếp tục sử dụng kết quả để tính toán, bạn sẽ không thể lấy trực tiếp từ `print()`.

Ngược lại, `return` không nhất thiết phải hiển thị bất kỳ thông tin nào. Nó chỉ gửi kết quả trở lại chương trình để các câu lệnh khác tiếp tục sử dụng.

Hãy so sánh hai hàm dưới đây.

```
def tong_1(a, b):
    print(a + b)

def tong_2(a, b):
    return a + b
```

Nếu gọi

```
tong_1(3, 4)
```

màn hình sẽ xuất hiện số `7`, nhưng bạn không thể lấy kết quả đó để cộng tiếp với một số khác.

Trong khi đó,

```
tong_2(3, 4)
```

không tự in ra màn hình, nhưng lại trả về giá trị `7`. Vì vậy, bạn hoàn toàn có thể viết

```
x = tong_2(3, 4)
print(x * 10)
```

hoặc

```
print(tong_2(3, 4) + tong_2(5, 6))
```

Chính khả năng kết hợp linh hoạt này khiến `return` được sử dụng rất phổ biến trong các chương trình thực tế.

6.11.2 Hàm kết thúc ngay khi gặp return

Một đặc điểm quan trọng của return là ngay khi được thực hiện, hàm sẽ kết thúc lập tức. Mọi câu lệnh phía sau return trong cùng một hàm sẽ không được thực thi.

Hàm dừng tại return

```
def vi_du():
    print("Bat dau")
    return
    print("Dong nay khong bao gio duoc thuc hien")

vi_du()
```

Kết quả chỉ là

```
Bat dau
```

Dòng cuối cùng không được thực hiện vì chương trình đã rời khỏi hàm ngay sau khi gặp return.

Đặc điểm này thường được sử dụng để kết thúc sớm việc xử lý khi đã tìm được kết quả hoặc khi phát hiện một điều kiện đặc biệt.

6.11.3 Một hàm có thể trả về nhiều kiểu dữ liệu

Giá trị được trả về không nhất thiết phải là số. Trong Python, hàm có thể trả về bất kỳ kiểu dữ liệu nào mà bạn đã học.

Ví dụ:

- Một số nguyên.
- Một số thực.
- Một chuỗi ký tự.
- Một giá trị logic True hoặc False.

Ví dụ:

```
def la_so_chan(n):
    return n % 2 == 0
```

Ở đây, hàm không in ra màn hình mà trả về một giá trị logic. Khi gọi

```
print(la_so_chan(8))
```

kết quả sẽ là

```
True
```

Kiểu thiết kế này rất phổ biến trong lập trình vì nó giúp các hàm có thể phối hợp với nhau để xây dựng những thuật toán lớn hơn.

6.11.4 Khi nào nên dùng `return`?

Trong hầu hết các trường hợp, nếu một hàm thực hiện tính toán thì nên sử dụng `return`. Điều này giúp kết quả có thể được sử dụng lại ở nhiều vị trí khác nhau trong chương trình.

Ngược lại, `print()` chỉ nên dùng khi mục đích của hàm là hiển thị thông tin cho người dùng, chẳng hạn như thông báo, hướng dẫn hoặc kết quả cuối cùng của chương trình.

Có thể ghi nhớ nguyên tắc đơn giản sau:

`print()` hay `return`?

- Dùng `print()` khi chỉ muốn hiển thị thông tin ra màn hình.
- Dùng `return` khi muốn gửi kết quả về để tiếp tục xử lý.
- Trong các chương trình thực tế, các hàm tính toán hầu như luôn sử dụng `return`.

6.12 Bài tập luyện tập

1. Viết hàm `la_so_chan(n)` nhận vào một số nguyên, trả về `True` nếu là số chẵn, `False` nếu là số lẻ.
2. Viết hàm `tinh_dt_hinh_chu_nhat(chieu_dai, chieu_rong)` trả về diện tích.
3. Viết hàm `xin_chao(ten, tuoi)` in ra câu: ”Tôi tên là [ten], năm nay [tuoi] tuổi”.

Lời bình Phần 1

Trong phần đầu của cuốn sách, bạn đã làm quen với những nền tảng quan trọng nhất của lập trình Python. Từ việc cài đặt môi trường làm việc, viết chương trình đầu tiên, sử dụng biến để lưu trữ dữ liệu, đến các kiểu dữ liệu cơ bản, phép toán, nhập xuất dữ liệu và những cấu trúc điều khiển đầu tiên, mỗi chương đều góp phần xây dựng một tư duy lập trình có hệ thống.

Bạn đã học cách để chương trình lưu giữ thông tin bằng biến, thực hiện các phép tính với toán tử, giao tiếp với người dùng thông qua hàm `input()` và `print()`, đồng thời biết cách sử dụng các phép so sánh và toán tử logic để xây dựng điều kiện. Trên cơ sở đó, bạn đã tiếp cận các cấu trúc rẽ nhánh bằng `if`, `if-else` và `if-elif-else`, giúp chương trình có khả năng đưa ra quyết định dựa trên dữ liệu đầu vào.

Tiếp theo, bạn đã tìm hiểu vòng lặp `for` và `while`, hai công cụ cho phép máy tính thực hiện những công việc lặp đi lặp lại một cách tự động. Bạn cũng biết cách điều khiển quá trình lặp bằng `break` và `continue`, đồng thời hiểu được tầm quan trọng của việc xây dựng

điều kiện dừng để tránh vòng lặp vô hạn.

Chương cuối của phần này giới thiệu khái niệm hàm (*function*), một trong những công cụ quan trọng nhất của lập trình hiện đại. Bạn đã biết cách định nghĩa hàm bằng từ khóa `def`, truyền dữ liệu thông qua tham số và đối số, cũng như sử dụng `return` để trả kết quả về cho chương trình. Từ đây, bạn không còn chỉ viết những đoạn mã rời rạc, mà đã bắt đầu biết tổ chức chương trình thành các khối chức năng độc lập, dễ đọc và có khả năng tái sử dụng.

Nếu nhìn lại toàn bộ nội dung đã học, bạn sẽ nhận thấy mỗi chương đều giải quyết một câu hỏi rất cơ bản trong lập trình. Biến trả lời cho câu hỏi *dữ liệu được lưu ở đâu*; phép toán trả lời *dữ liệu được xử lý như thế nào*; câu lệnh điều kiện trả lời *khi nào nên thực hiện một hành động*; vòng lặp trả lời *làm thế nào để thực hiện một công việc nhiều lần*; còn hàm trả lời *làm thế nào để tổ chức chương trình thành những đơn vị rõ ràng và có thể sử dụng lại*. Khi kết hợp những kiến thức này với nhau, bạn đã có thể xây dựng được nhiều chương trình hoàn chỉnh để giải quyết các bài toán ở mức cơ bản.

Quan trọng hơn cả, phần đầu của cuốn sách không chỉ giúp bạn học cú pháp Python mà còn hình thành tư duy lập trình. Bạn đã bắt đầu biết phân tích một bài toán thành các bước nhỏ, xác định dữ liệu đầu vào và đầu ra, xây dựng các điều kiện xử lý, nhận ra những thao tác lặp lại và đóng gói chúng thành các hàm. Đây chính là cách suy nghĩ mà mọi lập trình viên đều cần rèn luyện, bất kể họ sử dụng Python hay bất kỳ ngôn ngữ lập trình nào khác.

Những kiến thức trong phần này sẽ là nền tảng cho toàn bộ các chương tiếp theo. Khi bước sang phần hai, bạn sẽ không còn chỉ làm việc với các giá trị đơn lẻ, mà sẽ học cách quản lý nhiều dữ liệu cùng lúc bằng các cấu trúc dữ liệu của Python như danh sách (`list`), bộ (`tuple`), tập hợp (`set`) và từ điển (`dictionary`). Những công cụ đó sẽ giúp chương trình xử lý các bài toán thực tế hiệu quả hơn và mở ra cánh cửa đến những lĩnh vực cao hơn như phân tích dữ liệu, trí tuệ nhân tạo, phát triển ứng dụng và khoa học máy tính.

Phần II

Từ cú pháp đến cách giải một bài toán

Chương 7

Tư duy thuật toán - Trước code là một bản thiết kế

Giới thiệu ngắn

Biết cú pháp chưa đồng nghĩa biết giải bài toán. Chương này xây cây cầu từ lời kể đời sống sang một quá trình hữu hạn mà người khác có thể làm theo, kiểm tra và chuyển sang Python.

Mục tiêu học tập

- Phân biệt bài toán, mô hình, thuật toán và chương trình.
- Viết input, output, tiền điều kiện và hậu điều kiện.
- Mô tả thuật toán bằng pseudocode và bảng truy vết.
- Nhận diện tính đúng và tính dừng ở mức nhập môn.

Hành trang trước khi học

Một xe buýt cần tính giá vé theo quãng đường và nhóm hành khách. Đề bài bằng lời có ngoại lệ, đơn vị và biên. Nếu viết if ngay, ta dễ bỏ sót trường hợp. Tư duy thuật toán bắt đầu bằng phân rã input–process–output, lập bảng quyết định, tạo ví dụ tay và chứng minh quá trình dừng.

7.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một xe buýt cần tính giá vé theo quãng đường và nhóm hành khách. Đề bài bằng lời có ngoại lệ, đơn vị và biên. Nếu viết if ngay, ta dễ bỏ sót trường hợp. Tư duy thuật toán bắt đầu bằng phân rã input–process–output, lập bảng quyết định, tạo ví dụ tay và chứng minh quá trình dừng.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **tư duy thuật toán như nghệ thuật biến yêu cầu thành dãy bước hữu hạn**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

7.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: đầu vào, đầu ra, tiền điều kiện, dãy bước, trạng thái trung gian, điều kiện dừng và tiêu chí đúng.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

7.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{output} = A(\text{input}; \text{rules}), \qquad A \text{ hữu hạn và xác định}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: bước hiện tại, dữ liệu đã xác nhận và kết quả trung gian?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

7.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi bước phải rõ người thực hiện, dữ liệu dùng và kết quả tạo; với mọi đầu vào hợp lệ, thuật toán dừng và trả đúng một kết quả theo quy tắc.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

7.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **tư duy thuật toán như nghệ thuật biến yêu cầu thành dãy bước hữu hạn** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng

có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

7.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế thuật toán tính cước xe theo quãng đường bậc thang. Ta vẽ đoạn trên trục số, tính tay tại biên, viết pseudocode bằng động từ rồi mới chuyển thành hàm.

7.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

7.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

7.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay

đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

7.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

7.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

7.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

7.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Nhảy thẳng từ đề bài sang code.
- Dùng tên câu lệnh thay cho mô tả hành động.
- Không xác định điều kiện dừng.
- Chỉ thử một dữ liệu thuận lợi.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

7.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

7.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết thuật toán pha nước theo đơn vị đo.
2. Phân rã bài tính cước theo bậc.
3. Chứng minh thuật toán Euclid dừng.
4. Truy vết một thuật toán trên bảng.
5. So sánh hai lời giải theo số bước.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

7.8 Phòng thực hành sau khi đã hoàn tất mô hình

Ví dụ cuối: thuật toán Euclid sau khi đã truy vết

Đoạn mã dưới đây chỉ xuất hiện sau khi bài toán đã được phát biểu, mô hình hóa, truy vết và kiểm thử bằng tay. Hãy đọc nó như một bản triển khai của các bước đã chứng minh, không đọc như một mẫu để chép.

```
def ucln(a, b):
    a = abs(a)
    b = abs(b)
    while b != 0:
        a, b = b, a % b
    return a

print(ucln(84, 30))
```

7.9 Tóm tắt cuối chương

Bản đồ kiến thức

Thuật toán là lời giải độc lập với cú pháp. Trước khi viết Python, cần xác định hợp đồng, mô hình, bất biến, điều kiện dừng và các ca biên.

Chương 8

Truy vết, gỡ lỗi và kiểm thử - Học cách chứng minh chương trình

Giới thiệu ngắn

Người mới thường sửa lỗi bằng cách thử ngẫu nhiên. Chương này thay thói quen ấy bằng quy trình khoa học: dự đoán, đo, so sánh, thu hẹp và lưu lại một phép thử.

Mục tiêu học tập

- Lập bảng truy vết biến và nhánh.
- Thiết kế test từ miền tương đương và giá trị biên.
- Tìm dữ liệu gây lỗi nhỏ nhất.
- Dùng assert để giữ một tính chất của chương trình.

Hành trang trước khi học

Một chương trình tính nghiệm phương trình bậc hai trả sai khi delta bằng 0. Nhìn code từ đầu đến cuối nhiều lần không đủ; ta cần ghi trạng thái sau từng bước và so với kết quả toán học đã biết. Truy vết tìm vị trí lệch đầu tiên, còn kiểm thử giữ lỗi ấy không quay lại.

8.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một chương trình tính nghiệm phương trình bậc hai trả sai khi delta bằng 0. Nhìn code từ đầu đến cuối nhiều lần không đủ; ta cần ghi trạng thái sau từng bước và so với kết quả toán học đã biết. Truy vết tìm vị trí lệch đầu tiên, còn kiểm thử giữ lỗi ấy không quay lại.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **truy vết và kiểm thử như cách biến trực giác thành bằng chứng**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

8.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: dữ liệu thử, expected, observed, trạng thái từng dòng, nhánh đã đi và sai khác đầu tiên.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

8.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\Delta = b^2 - 4ac, \qquad N(\Delta) = \begin{cases} 0 & \Delta < 0 \\ 1 & \Delta = 0 \\ 2 & \Delta > 0 \end{cases}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng

thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: giá trị biến sau mỗi phép gán và tập giả thuyết về nguyên nhân?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

8.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Bảng truy vết phải tái tạo đúng luồng thực thi; mỗi lỗi đã sửa có một ca kiểm thử thất bại trước và thành công sau sửa.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

8.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **truy vết và kiểm thử như cách biến trực giác thành bằng chứng** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong

Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

8.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Kiểm tra bộ phân loại nghiệm tại delta âm, dương, bằng 0 và rất gần 0. Ta tính expected bằng tay, ghi bảng biến a, b, c, Δ, rồi so với chương trình.

8.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

8.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

8.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

8.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

8.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

8.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

8.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Chỉ in thêm dữ liệu mà không có giả thuyết.
- Chọn expected sau khi đã xem output.
- Không thử đúng tại biên.
- Sửa nhiều dòng cùng lúc.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

8.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

8.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Truy vết vòng lặp tính tổng.
2. Lập test table cho phương trình bậc hai.
3. Tìm minimal failing input.
4. Viết regression test bằng assert.
5. Phân biệt lỗi mô hình và lỗi cài đặt.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

8.8 Phòng thực hành sau khi đã hoàn tất mô hình

Ví dụ cuối: phân loại nghiệm có kiểm thử biên

Đoạn mã dưới đây chỉ xuất hiện sau khi bài toán đã được phát biểu, mô hình hóa, truy vết và kiểm thử bằng tay. Hãy đọc nó như một bản triển khai của các bước đã chứng minh, không đọc như một mẫu để chép.

```
def so_nghiem(a, b, c):
    if a == 0:
        raise ValueError("a phai khac 0")
    delta = b * b - 4 * a * c
    if delta < 0:
        return 0
    if delta == 0:
        return 1
    return 2

assert so_nghiem(1, 0, 1) == 0
assert so_nghiem(1, 2, 1) == 1
assert so_nghiem(1, 0, -1) == 2
```

8.9 Tóm tắt cuối chương

Bản đồ kiến thức

Debug không phải đoán. Hãy tái tạo lỗi, viết expected trước, truy vết điểm lệch đầu tiên, sửa tối thiểu và thêm regression test.

Chương 9

Mô hình hóa từ Toán học đến Python

Giới thiệu ngắn

Toán học nén một quan hệ thành ký hiệu; chương trình triển khai quan hệ ấy trên nhiều dữ liệu. Chương này dạy cách giữ nguyên ý nghĩa, đơn vị và giới hạn khi đi từ công thức sang mã.

Mục tiêu học tập

- Xác định biến, tham số và đơn vị.
- Kiểm tra thứ nguyên và miền áp dụng.
- Tách sai số đo khỏi sai số mô hình.
- Dùng Python để khảo sát nhiều kịch bản có kiểm chứng.

Hành trang trước khi học

Một bạn muốn ước lượng quãng đường chạy, năng lượng tiêu thụ và tiến độ theo tuần. Công thức chỉ hữu ích khi đơn vị, giả định và miền áp dụng rõ ràng. Python giúp tính lặp nhiều kịch bản, nhưng không thể tự biết công thức có đại diện đúng thực tế hay không.

9.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một bạn muốn ước lượng quãng đường chạy, năng lượng tiêu thụ và tiến độ theo tuần. Công thức chỉ hữu ích khi đơn vị, giả định và miền áp dụng rõ ràng. Python giúp tính lặp nhiều kịch bản, nhưng không thể tự biết công thức có đại diện đúng thực tế hay không.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **mô hình hóa Toán học như chiếc cầu giữa đại lượng thật và chương trình**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

9.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: đại lượng độc lập/phụ thuộc, tham số, đơn vị, giả định, miền áp dụng, sai số và dữ liệu kiểm chứng.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

9.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$y = f(x;\theta) + \varepsilon, \qquad \varepsilon = \text{sai số mô hình và đo lường}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: bộ tham số, dữ liệu đầu vào đã chuẩn hóa và sai số giữa dự đoán–quan sát?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

9.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi phép cộng/trừ phải cùng thứ nguyên; kết luận chỉ có hiệu lực trong miền và giả định đã công bố.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

9.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **mô hình hóa Toán học như chiếc cầu giữa đại lượng thật và chương trình** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

9.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Mô hình quãng đường chuyển động đều và chi phí chạy bộ theo một hệ số ước lượng. Ta kiểm tra thứ nguyên, đổi phút sang giờ, so dự đoán với một quan sát và giải thích phần sai lệch.

9.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

9.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

9.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

9.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

9.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

9.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

9.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng công thức đúng trong sai miền.
- Trộn đơn vị.
- Hiểu kết quả nhiều chữ số là chính xác.

- Che giả định trong một hằng số không tên.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

9.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

9.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Kiểm tra thứ nguyên năm công thức.
2. Mô hình lãi đơn/lãi kép.
3. Mô phỏng cấp số cộng.
4. Đo residual của mô hình.
5. Viết báo cáo có giả định và giới hạn.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

9.8 Phòng thực hành sau khi đã hoàn tất mô hình

Ví dụ cuối: mô hình chuyển động đều có kiểm tra đơn vị

Đoạn mã dưới đây chỉ xuất hiện sau khi bài toán đã được phát biểu, mô hình hóa, truy vết và kiểm thử bằng tay. Hãy đọc nó như một bản triển khai của các bước đã chứng minh, không đọc như một mẫu để chép.

```python
def quang_duong_km(van_toc_km_h, thoi_gian_phut):
    if van_toc_km_h < 0 or thoi_gian_phut < 0:
        raise ValueError("Dai luong phai khong am")
    thoi_gian_gio = thoi_gian_phut / 60
    return van_toc_km_h * thoi_gian_gio

assert quang_duong_km(12, 30) == 6
```

9.9 Tóm tắt cuối chương

Bản đồ kiến thức

Một mô hình tốt ghi rõ đại lượng, đơn vị, tham số, giả định, miền áp dụng và sai số. Code chỉ là phép triển khai mô hình đã được kiểm tra.

Chương 10

Chia nhỏ bài toán - Từ một khối lớn đến các phần có hợp đồng

Giới thiệu ngắn

Chương này không thêm nhiều cú pháp. Nó thay đổi cách nhìn: một chương trình là mạng những trách nhiệm nhỏ, mỗi trách nhiệm có dữ liệu vào, dữ liệu ra và cách kiểm tra riêng.

Mục tiêu học tập

- Phân rã theo trách nhiệm thay vì số dòng.
- Vẽ luồng dữ liệu giữa các hàm.
- Giảm phụ thuộc vào biến toàn cục.
- Kiểm thử từng phần rồi kiểm thử phép ghép.

Hành trang trước khi học

Một chương trình tính kết quả học tập phải nhận điểm, kiểm tra, tính trung bình, xếp loại và trình bày. Nếu coi đây là một việc duy nhất, mọi thay đổi làm ta phải đọc toàn bộ. Phân rã biến hệ thống lớn thành các hộp nhỏ có hợp đồng, rồi ghép chúng theo luồng dữ liệu.

10.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một chương trình tính kết quả học tập phải nhận điểm, kiểm tra, tính trung bình, xếp loại và trình bày. Nếu coi đây là một việc duy nhất, mọi thay đổi làm ta phải đọc toàn bộ. Phân rã biến hệ thống lớn thành các hộp nhỏ có hợp đồng, rồi ghép chúng theo luồng dữ liệu.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **phân rã bài toán như cách quản lý độ phức tạp**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

10.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: trách nhiệm, đầu vào/đầu ra từng khối, phụ thuộc, dữ liệu chung và điểm ghép.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

10.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$F = f_n \circ f_{n-1} \circ \cdots \circ f_1$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: kết quả trung gian hợp lệ đi từ khối trước sang khối sau?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

10.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi khối có một lý do chính để thay đổi; ghép các khối không làm mất tiền điều kiện/hậu điều kiện của chúng.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

10.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **phân rã bài toán như cách quản lý độ phức tạp** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích

nhiều lỗi tưởng như bí ẩn.

10.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Phân rã bộ xử lý điểm thành nhận, parse, validate, calculate, classify và present. Ta viết test cho calculate/classify mà không cần input hoặc print.

10.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

10.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

10.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

10.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

10.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

10.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

10.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Tách hàm theo số dòng.
- Để hàm nhỏ phụ thuộc biến toàn cục.
- Truyền quá nhiều tham số rời không có cấu trúc.

- Tạo hàm tên mơ hồ như process.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

10.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

10.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ cây phân rã máy tính BMI.
2. Viết hợp đồng từng hàm.
3. Tách core khỏi I/O.
4. Phát hiện coupling ẩn.
5. Ghép pipeline và test từng điểm.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

10.8 Phòng thực hành sau khi đã hoàn tất mô hình

Ví dụ cuối: pipeline xử lý điểm

Đoạn mã dưới đây chỉ xuất hiện sau khi bài toán đã được phát biểu, mô hình hóa, truy vết và kiểm thử bằng tay. Hãy đọc nó như một bản triển khai của các bước đã chứng minh, không đọc như một mẫu để chép.

```python
def hop_le(diem):
    return 0 <= diem <= 10

def trung_binh(a, b, c):
    return (a + b + c) / 3

def xep_loai(tb):
    if tb >= 8:
        return "Gioi"
    if tb >= 6.5:
        return "Kha"
    return "Can co gang"

assert xep_loai(trung_binh(8, 9, 7)) == "Gioi"
```

10.9 Tóm tắt cuối chương

Bản đồ kiến thức

Phân rã tốt làm cho mỗi phần dễ gọi tên, dễ kiểm tra và dễ thay đổi. Luồng dữ liệu rõ quan trọng hơn việc tạo thật nhiều hàm nhỏ.

Chương 11

Dữ liệu hợp lệ - Đừng để lỗi đi sâu vào chương trình

Giới thiệu ngắn

Một chương trình đáng tin không chỉ chạy với dữ liệu đẹp. Chương này xây cổng kiểm soát ở biên, để phần tính toán bên trong được làm việc với những giả định rõ ràng.

Mục tiêu học tập

- Phân biệt chuẩn hóa, parse và validation.
- Kiểm tra cú pháp, miền và quan hệ chéo.
- Thiết kế thông báo lỗi có ích.
- Không để dữ liệu bẩn đi vào lõi chương trình.

Hành trang trước khi học

Một biểu mẫu tuổi, điểm và email có thể nhận dữ liệu trống, sai kiểu hoặc đúng cú pháp nhưng vô nghĩa. Validation phải diễn ra theo lớp: có dữ liệu, parse được, nằm trong miền và nhất quán với các trường khác. Thông báo phải giúp người dùng sửa đúng.

11.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một biểu mẫu tuổi, điểm và email có thể nhận dữ liệu trống, sai kiểu hoặc đúng cú pháp nhưng vô nghĩa. Validation phải diễn ra theo lớp: có dữ liệu, parse được, nằm trong miền và nhất quán với các trường khác. Thông báo phải giúp người dùng sửa đúng.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **xác thực dữ liệu như bảo vệ ranh giới của mô hình**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

11.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: dữ liệu thô, cú pháp, kiểu, miền, quan hệ chéo, lỗi và giá trị đã chuẩn hóa.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

11.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{valid}(x) = S(x) \wedge T(x) \wedge R(x) \wedge C(x)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: trường đã hợp lệ, lỗi tích lũy và dữ liệu sạch chờ xử lý?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

11.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Dữ liệu chưa hợp lệ không được đi vào core; validation không âm thầm thay ý nghĩa đầu vào.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

11.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **xác thực dữ liệu như bảo vệ ranh giới của mô hình** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích

nhiều lỗi tưởng như bí ẩn.

11.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Xác thực đăng ký câu lạc bộ gồm tuổi, lớp và email. Ta tách strip, parse, range, relation; thu lỗi theo trường và chỉ tạo bản ghi khi không còn lỗi.

11.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

11.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

11.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

11.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

11.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

11.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

11.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Ép kiểu và bắt mọi lỗi bằng một câu chung.
- Chỉ kiểm tra kiểu mà bỏ miền.
- Tự sửa dữ liệu mơ hồ.

- Để core lặp lại validation ở nhiều nơi.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

11.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

11.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Lập ma trận lỗi cho ba trường.
2. Thiết kế thông báo có hành động sửa.
3. Tách sanitize và validate.
4. Kiểm tra ràng buộc chéo.
5. Thử dữ liệu Unicode và khoảng trắng.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

11.8 Phòng thực hành sau khi đã hoàn tất mô hình

Ví dụ cuối: nhập điểm có vòng phản hồi

Đoạn mã dưới đây chỉ xuất hiện sau khi bài toán đã được phát biểu, mô hình hóa, truy vết và kiểm thử bằng tay. Hãy đọc nó như một bản triển khai của các bước đã chứng minh, không đọc như một mẫu để chép.

```python
def nhap_diem():
    while True:
        raw = input("Nhap diem 0..10: ").strip()
        try:
            diem = float(raw)
        except ValueError:
            print("Hay nhap mot con so.")
            continue
        if not 0 <= diem <= 10:
            print("Diem phai nam trong 0..10.")
            continue
        return diem
```

11.9 Tóm tắt cuối chương

Bản đồ kiến thức

Validation là hợp đồng ở ranh giới. Hãy tách dữ liệu thô, dữ liệu đã parse và dữ liệu hợp lệ; báo đúng nguyên nhân thay vì che lỗi.

Chương 12

Dự án cuối Tập 1 - Máy luyện Toán trong terminal

Giới thiệu ngắn

Dự án cuối không nhằm khoe nhiều câu lệnh. Nó chứng minh bạn có thể đi trọn một chu kỳ: hiểu người dùng, mô hình hóa trạng thái, viết thuật toán, triển khai từng phần, kiểm thử và giải thích giới hạn.

Mục tiêu học tập

- Viết yêu cầu và tiêu chí chấp nhận.
- Thiết kế trạng thái cùng bất biến điểm số.
- Phát triển theo các milestone nhỏ.
- Tự kiểm thử và trình bày bằng chứng.

Hành trang trước khi học

Một máy luyện Toán tạo phép tính, nhận đáp án, chấm điểm và kết thúc sau số câu đã chọn. Dù nhỏ, dự án có trạng thái, vòng lặp, điều kiện, hàm, dữ liệu sai và tiêu chí hoàn thành. Làm dự án tốt nghĩa là xây từng lát mỏng có thể chạy và kiểm tra.

12.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một máy luyện Toán tạo phép tính, nhận đáp án, chấm điểm và kết thúc sau số câu đã chọn. Dù nhỏ, dự án có trạng thái, vòng lặp, điều kiện, hàm, dữ liệu sai và tiêu chí hoàn thành. Làm dự án tốt nghĩa là xây từng lát mỏng có thể chạy và kiểm tra.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **dự án nhập môn như phép tổng hợp mô hình, thuật toán và kiểm thử**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

12.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: số câu, câu hiện tại, toán hạng, đáp án đúng, câu trả lời, điểm và trạng thái kết thúc.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

12.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{score}_{k+1} = \text{score}_k + \mathbf{1}[\text{answer}_k = \text{expected}_k]$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: chỉ số câu, điểm hiện tại và thống kê đúng/sai?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

12.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Số câu đã xử lý cộng số câu còn lại bằng tổng; điểm không vượt số câu và mỗi câu chỉ được chấm một lần.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

12.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **dự án nhập môn như phép tổng hợp mô hình, thuật toán và kiểm thử** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng

có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

12.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Xây máy luyện phép nhân theo bốn milestone: một câu cố định, nhiều câu, xử lý nhập sai và báo cáo cuối. Mỗi milestone có test tay và tiêu chí xong trước khi thêm tính năng.

12.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

12.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

12.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay

đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

12.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

12.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

12.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

12.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Viết menu đầy đủ trước khi core đúng.
- Thêm tính năng giữa lúc chưa có test.
- Trộn sinh câu, nhập và chấm trong một hàm.
- Không định nghĩa khi nào dự án hoàn thành.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

12.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

12.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết user story.
2. Chia milestone theo lát dọc.
3. Lập state table.
4. Thiết kế test cho điểm.
5. Viết retrospective sau demo.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

12.8 Phòng thực hành sau khi đã hoàn tất mô hình

Phiên bản lõi sau khi hoàn tất thiết kế

Đoạn mã dưới đây chỉ xuất hiện sau khi bài toán đã được phát biểu, mô hình hóa, truy vết và kiểm thử bằng tay. Hãy đọc nó như một bản triển khai của các bước đã chứng minh, không đọc như một mẫu để chép.

```
def cham_cau(expected, raw_answer):
    try:
        answer = int(raw_answer)
    except ValueError:
        return False
    return answer == expected

def bao_cao(score, total):
    ti_le = 100 * score / total if total else 0
    return f"Ket qua: {score}/{total} ({ti_le:.1f}%)"

assert cham_cau(12, "12")
assert not cham_cau(12, "muoi hai")
assert bao_cao(4, 5) == "Ket qua: 4/5 (80.0%)"
```

12.9 Tóm tắt cuối chương

Bản đồ kiến thức

Một dự án hoàn chỉnh phải có yêu cầu, mô hình, bất biến, phiên bản chạy được, ca kiểm thử và lời giải thích. Code là một phần của hồ sơ bằng chứng.

Phụ lục A

Xưởng 24 bài toán - Đi từ Toán học và đời sống đến Python

Phần này không phải tuyển tập đáp án ngắn. Mỗi bài là một bài tập về cách biến tình huống thành mô hình, tạo phép tính tay độc lập, thiết kế thuật toán, lập ca kiểm thử rồi mới triển khai. Người học nên dành một trang giấy cho mỗi bài và giữ lại các bản truy vết; đó là bằng chứng về quá trình suy nghĩ.

Quy tắc của xưởng

Không mở trình soạn thảo code trong mười phút đầu. Hãy viết đại lượng, đơn vị, công thức hoặc quy tắc, một ví dụ tay và các giá trị biên. Chỉ khi một người khác có thể làm theo bản thiết kế ấy, bạn mới chuyển sang Python.

A.1 Điểm trung bình có trọng số

Tình huống

Ba thành phần có trọng số khác nhau và tổng trọng số phải bằng 1.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$\bar{x} = \sum_i w_i x_i, \quad \sum_i w_i = 1$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: nhân từng điểm với trọng số, cộng các tích và chỉ làm tròn khi hiển thị.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: trọng số thiếu 0.01, điểm đúng tại 0 và 10.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.2 Chuyển động thẳng đều

Tình huống

Tính quãng đường khi vận tốc dùng km/h nhưng thời gian nhập bằng phút.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$s = vt$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: đổi phút sang giờ trước, rồi nhân vận tốc với thời gian đã chuẩn hóa.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: thời gian bằng 0, vận tốc âm và 30 phút ở 12 km/h.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.3 Phương trình bậc hai

Tình huống

Phân loại và tính nghiệm thực của phương trình với hệ số đầu khác 0.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$\Delta = b^2 - 4ac$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: tính delta, phân nhánh theo dấu rồi mới áp công thức nghiệm.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: delta âm, bằng 0, dương và $a = 0$.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.4 Tam giác hợp lệ và diện tích

Tình huống

Ba cạnh chỉ tạo tam giác khi tổng hai cạnh bất kỳ lớn hơn cạnh còn lại.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$p = \frac{a+b+c}{2}, \quad S = \sqrt{p(p-a)(p-b)(p-c)}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: kiểm tra cạnh dương và bất đẳng thức tam giác trước công thức Heron.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: cạnh 3–4–5, 1–2–3 và một cạnh âm.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.5 Tiền điện theo bậc

Tình huống

Đơn giá thay đổi theo từng đoạn sản lượng, không phải toàn bộ kWh theo bậc cuối.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$C(x) = \sum_{j} p_j \, \min(\max(x - b_j, 0), h_j)$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: tách phần tiêu thụ thuộc từng bậc rồi cộng chi phí.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: đúng tại mọi mốc bậc và ngay hai phía.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.6 Chỉ số BMI

Tình huống

Khối lượng dùng kilogram, chiều cao thường nhập centimet nhưng công thức cần mét.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$BMI = \frac{m}{h^2}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: đổi chiều cao sang mét, kiểm tra dương, tính rồi phân loại bằng ngưỡng.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: chiều cao 0, 170 cm và giá trị sát ngưỡng.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.7 Lãi kép

Tình huống

Số dư sau mỗi kỳ trở thành vốn của kỳ kế tiếp.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$B_n = B_0(1 + r)^n$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: tính cả công thức đóng và vòng lặp để kiểm chứng chéo.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: lãi 0, một kỳ và mười hai kỳ.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.8 Tiết kiệm đến mục tiêu

Tình huống

Tìm tháng đầu tiên số dư đạt mốc khi mỗi tháng gửi thêm một khoản.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$B_{k+1} = B_k(1 + r) + p$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: lặp theo trạng thái, tăng tháng đúng một lần và kiểm tra điều kiện dừng.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: mục tiêu đã đạt từ đầu và khoản gửi bằng 0.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.9 Kiểm tra số nguyên tố

Tình huống

Một số lớn hơn 1 là nguyên tố nếu không có ước từ 2 đến căn bậc hai của nó.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$d \le \sqrt{n}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: loại $n < 2$, thử ước đến khi $d^2 > n$ hoặc tìm thấy ước.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: 0, 1, 2, số chính phương và số nguyên tố.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.10 Thuật toán Euclid

Tình huống

Tìm ước chung lớn nhất bằng cách thay cặp số bằng số thứ hai và phần dư.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: truy vết các phần dư cho đến khi số thứ hai bằng 0.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: hai số bằng nhau, một số 0 và hai số nguyên tố cùng nhau.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.11 Đổi số thập phân sang nhị phân

Tình huống

Các phần dư khi chia liên tiếp cho 2 tạo bit theo thứ tự ngược.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$n = 2q + r, \quad r \in \{0, 1\}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc

lập với chương trình. Cách tính được dùng ở đây: ghi phần dư, cập nhật thương, rồi đảo thứ tự các bit.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: 0, 1, lũy thừa của 2 và số lẻ.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.12 Dãy Collatz và điều kiện dừng

Tình huống

Nếu số chẵn chia 2, nếu số lẻ nhân 3 cộng 1; quan sát đến khi gặp 1 trong phạm vi thử.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$a_{k+1} = \begin{cases} a_k/2 & a_k \text{ chẵn} \\ 3a_k + 1 & a_k \text{ lẻ} \end{cases}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: giữ số bước và giới hạn an toàn thay vì giả định vô hạn.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: đầu vào 1, số chẵn, số lẻ và giới hạn bước.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.13 Máy tính bốn phép

Tình huống

Người dùng chọn phép toán và hai toán hạng; phép chia cần đường lỗi riêng.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$y = f_{\mathrm{op}}(a, b)$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: xác thực tên phép toán rồi phân nhánh, kiểm tra mẫu trước khi chia.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: chia cho 0, phép toán không tồn tại và số âm.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.14 Kiểm tra mật khẩu theo quy tắc

Tình huống

Một mật khẩu cần độ dài tối thiểu và nhiều nhóm ký tự; đây là kiểm tra tiêu chí, không phải đánh giá an ninh tuyệt đối.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$valid = L \wedge U \wedge D \wedge S$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: duyệt ký tự, cập nhật cờ tiêu chí và giải thích tiêu chí thiếu.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: chuỗi rỗng, chỉ chữ, đủ nhóm và Unicode.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.15 Trò chơi đoán số

Tình huống

Sau mỗi lần đoán, khoảng ứng viên thu hẹp theo phản hồi cao/thấp.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$[L, R] \to [L, g-1] \text{ hoặc } [g+1, R]$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: giữ số lượt và kiểm tra input nằm trong khoảng.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: đoán đúng lần đầu, nhập ngoài khoảng và hết lượt.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.16 Độ cao vật ném theo thời gian

Tình huống

Mô hình lý tưởng bỏ lực cản chỉ hợp lệ trong miền thời gian trước khi vật chạm đất.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$h(t) = h_0 + v_0 t - \frac{1}{2} g t^2$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: chọn bước thời gian, tính dãy giá trị và dừng khi độ cao không dương.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: vận tốc đầu 0, bước thời gian quá lớn và thời điểm chạm đất.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.17 Điểm hòa vốn

Tình huống

Lợi nhuận bằng doanh thu trừ chi phí cố định và chi phí biến đổi.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$P(q) = pq - (F + cq)$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: giải bất phương trình và xác nhận bằng tìm kiếm nguyên nhỏ nhất.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: giá bán không lớn hơn chi phí biến đổi.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.18 Trung bình và phương sai

Tình huống

Hai lớp có thể cùng trung bình nhưng độ phân tán khác nhau.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$\sigma^2 = \frac{1}{n}\sum_i (x_i - \bar{x})^2$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: tính mean trước, rồi sai lệch bình phương; nêu rõ dùng population hay sample.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: một phần tử, mọi phần tử bằng nhau và dữ liệu rỗng.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.19 Năm nhuận

Tình huống

Quy tắc lịch có ngoại lệ theo 100 năm và ngoại lệ của ngoại lệ theo 400 năm.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$leap = (y \bmod 400 = 0) \lor [(y \bmod 4 = 0) \land (y \bmod 100 \neq 0)]$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: viết bảng chân trị cho các trường hợp chia hết.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: 1900, 2000, 2024 và 2025.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.20 Khoản trả góp cố định

Tình huống

Khoản trả phụ thuộc lãi suất theo kỳ, số kỳ và dư nợ; lãi suất 0 là nhánh đặc biệt.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$A = P\frac{r(1+r)^n}{(1+r)^n - 1}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: đổi lãi năm sang lãi tháng và kiểm tra tổng tiền bằng mô phỏng dư nợ.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: lãi 0, một kỳ và số kỳ không dương.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.21 Mô phỏng bể nước

Tình huống

Mực nước mới bằng mực cũ cộng dòng vào trừ dòng ra nhưng bị giới hạn bởi dung tích.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$h_{k+1} = \min(H, \max(0, h_k + q_{in} - q_{out}))$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: cập nhật theo bước thời gian và ghi sự kiện tràn/cạn.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: bể rỗng, bể đầy và dòng ra lớn hơn hiện có.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.22 Logic đèn giao thông

Tình huống

Hai hướng xung đột không được cùng xanh; pha vàng là trạng thái riêng.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$\neg(G_A \wedge G_B)$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: liệt kê trạng thái hợp lệ rồi định nghĩa chuyển tiếp theo thời gian.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: reset ở mỗi pha và yêu cầu đổi hướng sát deadline.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.23 Lịch học theo quỹ thời gian

Tình huống

Phân bổ số phút hữu hạn cho các môn theo ưu tiên nhưng không vượt thời gian rảnh.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$\sum_i t_i \le T, \quad t_i \ge m_i$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: cấp mức tối thiểu trước, rồi phân bổ phần còn lại theo trọng số.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: tổng mức tối thiểu vượt quỹ và một môn trọng số 0.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

A.24 Báo cáo tiến độ tuần

Tình huống

Tiến độ cần dựa trên nhiệm vụ hoàn thành có bằng chứng, không chỉ số giờ ngồi.

1. Gọi tên đại lượng

Trước khi tính, ta lập bảng đại lượng gồm dữ liệu đã biết, dữ liệu cần tìm, đơn vị và miền hợp lệ. Mọi giá trị người dùng nhập vẫn là dữ liệu thô; chỉ sau khi parse, đổi đơn vị và kiểm tra, nó mới được phép tham gia phép tính. Ở bài này, quan hệ trung tâm là:

$$p = \frac{\sum_i w_i c_i}{\sum_i w_i}$$

2. Tính một trường hợp bằng tay

Chọn một bộ số nhỏ, thay lần lượt vào mô hình và ghi từng kết quả trung gian. Không gộp nhiều phép biến đổi trong một bước, bởi mục tiêu của phép tính tay là tạo một oracle độc lập với chương trình. Cách tính được dùng ở đây: xác định trọng số trước tuần, ghi trạng thái nhị phân và giải thích nhiệm vụ chưa đạt.

3. Viết thuật toán bằng lời

1. Nhận dữ liệu và ghi rõ đơn vị.
2. Kiểm tra tiền điều kiện; nếu sai, dừng với thông báo có thể hành động.
3. Chuẩn hóa dữ liệu về cùng một hệ biểu diễn.
4. Thực hiện từng phép biến đổi theo đúng thứ tự của mô hình.
5. Kiểm tra hậu điều kiện và trình bày kết quả kèm đơn vị.

4. Truy vết và kiểm thử

Ta không chỉ thử bộ số vừa tính. Tập kiểm thử phải có dữ liệu bình thường, giá trị tại biên, giá trị ngay hai phía của biên và dữ liệu không hợp lệ. Ca kiểm thử bắt buộc của bài là: không có nhiệm vụ, mọi nhiệm vụ xong và nhiệm vụ trọng số lớn chưa xong.. Với mỗi ca, hãy ghi expected trước khi chạy.

Hồ sơ phải nộp

Một lời giải đầy đủ gồm mô hình, bảng đơn vị, phép tính tay, pseudocode, code Python, ít nhất bốn ca kiểm thử và một đoạn giải thích giới hạn. Nếu chỉ có code và ảnh output, bài toán chưa được giải thích.

Tài liệu tham khảo

1. Python Software Foundation. (2023). *Python 3 Documentation*. https://docs.python.org/3/
2. Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.
3. Sweigart, A. (2015). *Automate the Boring Stuff with Python*. No Starch Press.
4. Shaw, Z. A. (2017). *Learn Python 3 the Hard Way*. Addison-Wesley.

Nguồn chính thống dùng để kiểm chứng bản sâu

1. Python Software Foundation. *The Python Tutorial*, Python 3.14: https://docs.python.org/3/tutorial/
2. Python Software Foundation. *Python Data Model*: https://docs.python.org/3/reference/datamodel.html
3. Python Software Foundation. *Built-in Types*: https://docs.python.org/3/library/stdtypes.html
4. Python Software Foundation. *Errors and Exceptions*: https://docs.python.org/3/tutorial/errors.html
5. Python Software Foundation. *Input and Output*: https://docs.python.org/3/tutorial/inputoutput.html